



1st Edition

REACT 19 COMPLETE COURSE

An In-depth Journey into Building Powerful
Interactive, and Highly Maintainable Web
Applications for the Modern Era

Frank Wells

CURSO COMPLETO DE REACT

Un viaje en profundidad hacia la creación de aplicaciones interactivas potentes y
Aplicaciones web de alto mantenimiento para la era moderna

Frank Wells

Derechos de autor

Todos los derechos reservados. Ninguna parte de esta publicación podrá ser reproducida, distribuida o transmitida en ninguna forma ni por ningún medio, incluyendo fotocopia, grabación u otros métodos electrónicos o mecánicos, sin la autorización previa por escrito del editor, excepto en el caso

de citas breves incorporadas en reseñas críticas y ciertos otros usos no comerciales permitidos por la ley de derechos de autor.

Derechos de autor © (Frank Wells), (2024).

Tabla de contenido

Capítulo 1.

Introducción a React 19

- ¿Qué es React?
- Características clave de React 19
- Beneficios de usar React para el desarrollo web moderno
- Comprender el ecosistema React

Capítulo 2.

Configuración de su entorno

- Instalación de Node.js y npm -
- Creación de un proyecto React con Create React App • Estructura
de carpetas y anatomía del proyecto
- Configuración de herramientas de desarrollo (IDE, extensiones y linters)

Capítulo 3.

Conceptos básicos de React •

- JSX: Extensión de sintaxis de JavaScript -
- Componentes: Componentes funcionales y de clase • Props: Pasar
datos a componentes
- Conceptos básicos de gestión estatal

Capítulo 4.

Nuevas funciones de React 19

- Actualizaciones clave en React 19
- Mejoras en el rendimiento y la representación • Manejo de
componentes del servidor
- Mejoras en las herramientas para desarrolladores de React

Capítulo 5.

Componentes de React en profundidad

- Comprensión del ciclo de vida de los componentes
- Escritura de componentes reutilizables y modulares

- [Componentes con estado y sin estado](#)
- [Mejores prácticas para el diseño de componentes](#)

[Capítulo 6.](#)

[Gestión del estado y del ciclo de vida •](#)

[Gestión del estado en React](#)

- [Uso de ganchos \(useState, useEffect, useReducer\)](#) • [Métodos](#)
- [de ciclo de vida de componentes de clase](#) • [Mejores prácticas para actualizaciones de estado](#)

[Capítulo 7.](#)

[Manejo de eventos y formularios](#)

- [Agregar oyentes de eventos en React](#)
- [Trabajar con formularios y componentes controlados](#) • [Técnicas de validación de formularios](#) •
- [Integración de bibliotecas de formularios de terceros](#)

[Capítulo 8.](#)

[Enrutamiento y navegación de React](#)

- [Introducción a React Router](#)
- [Configuración de la](#)
- [navegación](#) • [Enrutamiento dinámico y parámetros de ruta](#)
- [Manejo de rutas anidadas](#)

[Capítulo 9.](#)

[Dar estilo a las aplicaciones React](#)

- [Estilos en línea vs CSS en JS](#) • [Uso de módulos CSS](#) •
- [Componentes con estilo y emoción](#)
- [Tematización de su aplicación](#) _ _

[React Capítulo 10.](#)

[Obtención de datos y API](#)

- [Obtener datos con Fetch y Axios](#) • [Usar React](#)
- [Query para la gestión del estado del servidor](#)

- [Manejo de carga, errores y almacenamiento en caché](#)
- [Integración de API RESTful](#)

[Capítulo 11.](#)

[Gestión de estados más allá de React • API](#)

- [de contexto para la gestión de estados globales](#)
- [Kit de herramientas Redux: un enfoque simplificado de Redux](#)
- [Comparación de Redux con Context API](#)
- [Gestión de estados complejos con MobX](#)

[Capítulo 12. Optimización del rendimiento](#)

- [React.memo y ganchos useMemo](#)
- [Carga diferida de componentes con React.lazy](#)
- [Optimización de la renderización y reducción de re-renderizaciones](#)
- [Creación de perfiles de rendimiento con React DevTools](#)

[Capítulo 13.](#)

[Pruebas de aplicaciones React •](#)

- [Introducción a la biblioteca de pruebas de React](#)
- [Escritura de pruebas unitarias para componentes](#)
- [Pruebas instantáneas](#)
- [Pruebas de extremo a extremo](#)

[con Cypress Capítulo 14.](#)

[Reaccionar con TypeScript](#)

- [Introducción a TypeScript en proyectos React](#)
- [Escritura de propiedades y estados en React](#)
- [Uso de TypeScript con ganchos](#)
- [Mejores prácticas para TypeScript y React](#)

[Capítulo 15.](#)

[Temas avanzados en React](#)

- [Representación del lado del servidor \(SSR\) con Next.js](#)
- [Generación de sitios estáticos \(SSG\) con Next.js](#)
- [Integración con React y GraphQL](#)

- Datos en tiempo real con WebSockets

Capítulo 16.

Implementación de aplicaciones React

- Preparación de su aplicación para producción •

Opciones de alojamiento: Vercel, Netlify, AWS y Heroku

- Implementación continua con acciones de GitHub • Monitoreo

y depuración en producción Capítulo 17.

Ecosistema y bibliotecas de React

- Explorando las bibliotecas populares de React

- Bibliotecas de componentes (Material-UI, Ant Design)

- Animaciones con Framer Motion

- React y bibliotecas de gráficos Capítulo

18.

Problemas comunes de React y sus soluciones

- Depuración de aplicaciones React

- Manejo de problemas de compatibilidad •

Consejos para escribir código limpio y mantenible Capítulo 19.

Construyendo un proyecto React en el mundo real

- Configuración del proyecto

- Creación de una aplicación web rica en funciones

- Pruebas e implementación •

Reflexiones sobre el proceso de desarrollo • Reflexiones

sobre el proceso de desarrollo

Conclusión

- Resumen de conceptos y aprendizajes clave •

Mantenerse actualizado con React

- Palabras finales del autor

Capítulo 1.

Introducción a React 19

- es React? ?

React es una popular biblioteca JavaScript de código abierto que se utiliza para crear interfaces de usuario, en particular aplicaciones de página única (SPA), donde el contenido se actualiza dinámicamente sin necesidad de recargar la página completa. Fue desarrollada por Facebook (ahora Meta) en 2013 y, desde entonces, se ha convertido en una de las herramientas más utilizadas para el desarrollo front-end. React se centra principalmente en la "vista" de la arquitectura Modelo-Vista-Controlador (MVC), lo que permite a los desarrolladores crear aplicaciones web dinámicas e interactivas con facilidad.

En esencia, React ayuda a los desarrolladores a crear componentes de interfaz de usuario reutilizables, que son los pilares de las interfaces web modernas. Estos componentes se pueden combinar, anidar y gestionar eficientemente, lo que promueve la modularidad y la reutilización en el diseño de aplicaciones. Al descomponer interfaces complejas en partes más pequeñas y manejables, React simplifica el desarrollo, mejora la mantenibilidad y garantiza la consistencia en toda la aplicación.

Cómo funciona React

React aprovecha el concepto del DOM Virtual (Modelo de Objetos del Documento). A diferencia del DOM tradicional, que manipula directamente la estructura de la página web, el DOM Virtual es una representación en memoria del DOM real. Cuando un usuario interactúa con una aplicación React, los cambios se aplican primero al DOM Virtual. React calcula entonces el conjunto mínimo de cambios necesarios para actualizar el DOM real, garantizando un rendimiento óptimo y una experiencia de usuario fluida.

Este proceso, conocido como reconciliación, hace que las aplicaciones React sean más rápidas y eficientes que los frameworks JavaScript tradicionales. Elimina la necesidad de volver a renderizarlas innecesariamente, lo cual puede ralentizar las aplicaciones, especialmente aquellas con contenido complejo y dinámico.

Arquitectura basada en componentes de React

La arquitectura basada en componentes de React es una de sus características destacadas. Los componentes son piezas de código independientes y reutilizables que definen cómo debe verse y comportarse una parte específica de la interfaz de usuario. Pueden administrar su propio estado y recibir datos a través de props (abreviatura de propiedades).

Por ejemplo, un componente de React podría representar un botón, un formulario o incluso un panel complejo. Al combinar estos componentes, los desarrolladores pueden crear interfaces de usuario sofisticadas sin repetir código. React también admite la anidación, lo que permite estructurar los componentes jerárquicamente.

Este enfoque ha revolucionado la forma en que los desarrolladores piensan sobre el desarrollo de UI, cambiando el foco de las páginas a los componentes.

Desarrollo con React vs. JavaScript tradicional

Antes de React, los desarrolladores usaban bibliotecas como jQuery para manipular el DOM directamente. Si bien era eficaz para proyectos pequeños, este enfoque se volvió engorroso y propenso a errores a medida que las aplicaciones se volvían más complejas. Mantener una separación clara entre la lógica y la interfaz de usuario se volvió cada vez más difícil.

React aborda estos problemas ofreciendo un enfoque declarativo para el desarrollo de la interfaz de usuario. En lugar de describir explícitamente cada paso necesario para actualizar la interfaz de usuario, los desarrolladores simplemente definen el estado deseado y React se encarga del resto. Este cambio de paradigma ha convertido a React en la opción preferida para crear aplicaciones web modernas.

Popularidad y adopción

La simplicidad, escalabilidad y rendimiento de React lo han convertido en uno de los favoritos tanto de desarrolladores como de empresas. Impulsa el frontend de numerosas aplicaciones de alto perfil, como Facebook, Instagram, Airbnb y Netflix. Su comunidad activa, su sólido ecosistema y sus actualizaciones periódicas garantizan que se mantenga a la vanguardia de las tecnologías de desarrollo frontend.

Ventajas de React

- **Sintaxis declarativa:** React simplifica el desarrollo de la interfaz de usuario mediante el uso de código declarativo, lo que permite a los desarrolladores describir cómo debería verse la interfaz de usuario en función del estado de la aplicación.
- **Componentes reutilizables:** React promueve la reutilización, reduciendo el tiempo de desarrollo y garantizando la coherencia entre los proyectos.
- **DOM virtual:** al optimizar las actualizaciones del DOM real, React mejora el rendimiento y minimiza los cálculos innecesarios.
- **Flujo de datos unidireccional:** el enlace de datos unidireccional de React garantiza un código predecible y manejable, lo que facilita la depuración.
- **Ecosistema rico:** con herramientas como React Router, Redux y una amplia biblioteca de componentes de terceros, React es altamente extensible.
- **Desarrollo multiplataforma:** React Native, un derivado de React, permite a los desarrolladores crear aplicaciones móviles utilizando los mismos principios y conocimientos.

La amplia adopción de React también se ve respaldada por su sólida comunidad. Los desarrolladores pueden acceder a innumerables tutoriales, plugins y foros, lo que facilita el aprendizaje y la resolución de desafíos.

• Características clave de React 19

React 19 presenta mejoras y refinamientos significativos, consolidando su posición como una biblioteca líder para la creación de aplicaciones web.

Aplicaciones. Estas actualizaciones se centran en mejorar el rendimiento, simplificar los flujos de trabajo de desarrollo y garantizar la compatibilidad con los estándares web modernos.

1. Representación simultánea

Una de las características más esperadas de React 19 es el renderizado concurrente, que permite a React priorizar tareas y renderizar componentes de forma más eficiente. Esta función mejora la experiencia del usuario al garantizar que las actualizaciones de alta prioridad, como la entrada del usuario, se procesen primero y se pospongan las actualizaciones menos críticas.

Por ejemplo, en una aplicación con animaciones o transiciones, la renderización simultánea garantiza que estos elementos permanezcan fluidos y respondan, incluso durante cálculos pesados o la obtención de datos.

2. Componentes del servidor

React 19 mejora su compatibilidad con componentes de servidor, lo que permite a los desarrolladores renderizar componentes en el servidor y transmitirlos al cliente. Este enfoque reduce la cantidad de JavaScript necesario en el lado del cliente, lo que resulta en tiempos de carga más rápidos y un mejor rendimiento, especialmente en dispositivos o redes más lentas.

Los componentes del servidor permiten a los desarrolladores dividir la lógica de la aplicación entre el servidor y el cliente sin problemas, optimizando el uso de recursos sin comprometer la interactividad.

3. Herramientas de desarrollo mejoradas

La extensión React Developer Tools se ha actualizado en React 19 para ofrecer una mejor comprensión de las jerarquías de componentes, la gestión de estados y el rendimiento. Las nuevas funciones incluyen:

- Perfilado mejorado para identificar el rendimiento de los cuellos de botella.
- Información de depuración más clara para ganchos y efectos.
- Herramientas para visualizar el impacto de la renderización concurrente.

Estas actualizaciones permiten a los desarrolladores depurar y optimizar sus aplicaciones de manera más efectiva.

4. Procesamiento automático de lotes

En React 19, el procesamiento automático por lotes se aplica a todas las actualizaciones asincrónicas, como tiempos de espera, solicitudes de red y ganchos personalizados. Esta función minimiza la cantidad de nuevas representaciones al agrupar las actualizaciones, lo que garantiza que la aplicación mantenga su rendimiento incluso durante operaciones complejas.

5. Manejo mejorado de errores

React 19 introduce límites de error mejorados, lo que facilita la gestión de excepciones y la recuperación sin problemas. Los desarrolladores ahora pueden detectar errores en código asíncrono, como controladores de eventos o efectos, sin recurrir a soluciones alternativas.

Esta mejora mejora la estabilidad de las aplicaciones React y proporciona a los usuarios una mejor experiencia general.

6. Soporte de API modernizado

React 19 continúa perfeccionando su API, ofreciendo mayor compatibilidad con las funciones modernas de JavaScript. Esto incluye una mejor integración con los módulos ES, mejores capacidades de tree-shaking y una agrupación más eficiente para compilaciones de producción.

7. Rendimiento optimizado para aplicaciones grandes

Con las actualizaciones del algoritmo de conciliación de React y las optimizaciones del DOM virtual, React 19 ofrece un rendimiento más rápido para aplicaciones a gran escala. Los desarrolladores que crean paneles complejos, aplicaciones en tiempo real o interfaces con uso intensivo de datos pueden esperar mejoras notables en la capacidad de respuesta.

8. Funciones de accesibilidad mejoradas

La accesibilidad es una consideración fundamental para el desarrollo web y React 19 mejora la compatibilidad con los atributos ARIA y el teclado

Navegación. Estas actualizaciones simplifican el proceso de creación de aplicaciones inclusivas y compatibles con los estándares de accesibilidad web.

9. Integración con cadenas de herramientas modernas

React 19 se integra perfectamente con herramientas de compilación modernas como Vite y Webpack 5. Esta compatibilidad garantiza flujos de trabajo de desarrollo más rápidos, tiempos de compilación reducidos y una experiencia de desarrollador más fluida.

10. Compatibilidad con versiones posteriores

React 19 está diseñado con la compatibilidad hacia adelante en mente, lo que permite a los desarrolladores adoptar nuevas funciones gradualmente sin modificar el código base existente. Esto garantiza una transición fluida para los equipos que actualizan desde versiones anteriores de React.

Estas mejoras en React 19 no solo abordan problemas comunes, sino que también preparan a los desarrolladores para el futuro del desarrollo web.

Al adoptar React 19, los desarrolladores obtienen acceso a un conjunto de herramientas de vanguardia que prioriza el rendimiento, la escalabilidad y la experiencia del desarrollador.

• Beneficios de usar React para la web moderna

Desarrollo

React se ha convertido en la biblioteca de referencia para el desarrollo front-end, ofreciendo beneficios incomparables para crear aplicaciones web modernas.

Sus características únicas y su robusta arquitectura permiten a los desarrolladores crear interfaces de usuario escalables, eficientes e interactivas que satisfacen las necesidades de las experiencias web contemporáneas. A continuación, exploramos las principales ventajas que hacen de React una excelente opción para el desarrollo web moderno.

1. Paradigma de programación declarativa

React utiliza un enfoque declarativo para describir las interfaces de usuario, lo que simplifica el proceso de desarrollo. Los desarrolladores especifican qué es la interfaz de usuario.

Cómo debería verse la interfaz para un estado determinado, y React se encarga de renderizar los cambios correspondientes en la interfaz. Esto contrasta con la programación imperativa, donde los desarrolladores describen manualmente los pasos necesarios para actualizar la interfaz.

Al abstraer la lógica de renderizado, React reduce la complejidad de mantenimiento y depuración de aplicaciones, lo que permite a los desarrolladores centrarse en la lógica de la aplicación en lugar de en la mecánica de la manipulación del DOM.

2. Arquitectura basada en componentes

La arquitectura basada en componentes de React fomenta la modularidad y la reutilización. Los componentes son unidades de código autónomas que gestionan su propia lógica, estilo y estado. Esta estructura permite a los desarrolladores:

- Reutilice componentes en múltiples proyectos, ahorrando tiempo y esfuerzo.
- Administre y actualice fácilmente los elementos de la interfaz de usuario sin afectar toda la aplicación.
- Cree una apariencia consistente en todo el solicitud.

Por ejemplo, un desarrollador puede crear un componente de botón y reutilizarlo en todo el proyecto, personalizando su apariencia y funcionalidad con accesorios.

3. Optimización del rendimiento con DOM virtual

El uso del DOM Virtual por parte de React supone un cambio radical para el rendimiento web. La manipulación tradicional del DOM puede ser lenta, especialmente en aplicaciones con actualizaciones frecuentes. React aborda este desafío utilizando el DOM Virtual para calcular la cantidad mínima de cambios necesarios para actualizar el DOM real.

Este eficiente proceso de renderizado garantiza que las aplicaciones sigan respondiendo y funcionando bien, incluso bajo intensa interacción del usuario o cambios en los datos.

4. Flujo de datos unidireccional

React emplea un flujo de datos unidireccional, donde los datos fluyen en una sola dirección, desde los componentes principales a los secundarios. Este diseño simplifica la gestión de estados y garantiza un comportamiento predecible, lo que facilita la depuración y el mantenimiento de aplicaciones complejas.

Para una gestión de estados más amplia, herramientas como Redux o Context API se integran perfectamente con React, lo que proporciona a los desarrolladores opciones para gestionar datos de toda la aplicación de manera eficiente.

5. Flexibilidad y ecosistema

React es una biblioteca, no un framework completo, lo que ofrece a los desarrolladores la libertad de elegir herramientas y bibliotecas adicionales según los requisitos del proyecto. Ya sea para el enrutamiento, la gestión de estados o las pruebas, React se integra a la perfección con herramientas de terceros, lo que permite a los desarrolladores crear soluciones personalizadas.

Esta flexibilidad contrasta con los marcos rígidos y hace que React sea adecuado para una amplia gama de proyectos, desde pequeños sitios web hasta aplicaciones de nivel empresarial.

6. Fuerte adopción por parte de la comunidad y la industria

La popularidad de React se traduce en una amplia comunidad de desarrolladores que contribuyen a su ecosistema. Este sistema de soporte ofrece:

- Acceso a amplia documentación, tutoriales y Recursos en línea.

- Una amplia gama de bibliotecas y complementos de terceros para mejorar el desarrollo.

- Un grupo de desarrolladores experimentados, lo que facilita la contratación de talento.

Además, su adopción generalizada por parte de empresas como Facebook, Instagram, Netflix y Airbnb demuestra la confiabilidad y escalabilidad de React para aplicaciones del mundo real.

7. Integración perfecta con herramientas modernas

React funciona a la perfección con herramientas y frameworks de desarrollo modernos. Es compatible con Webpack, Babel, Vite y otras herramientas que optimizan los flujos de trabajo de desarrollo. Estas integraciones permiten a los desarrolladores optimizar el rendimiento de su código, gestionar la agrupación de módulos y ser compatible con la sintaxis moderna de JavaScript.

El soporte de React para las características de ES6+ y TypeScript mejora aún más la productividad y la calidad del código.

8. Desarrollo multiplataforma con React Native

Los principios de React se extienden más allá de la web a través de React Native, un framework para crear aplicaciones móviles. Los desarrolladores familiarizados con React pueden usar las mismas habilidades para crear aplicaciones móviles para iOS y Android, aprovechando una base de código compartida para mayor eficiencia.

Esta capacidad multiplataforma hace de React una opción atractiva para las empresas que buscan llegar a usuarios tanto web como móviles.

9. Accesibilidad y soporte SEO

React ofrece un soporte sólido para la creación de aplicaciones web accesibles. Características como los atributos ARIA, los componentes semánticos y la navegación de teclado mejorada facilitan la creación de aplicaciones inclusivas.

Para SEO, las capacidades de renderizado del lado del servidor de React (habilitadas a través de marcos como Next.js) garantizan que los motores de búsqueda puedan indexar el contenido de manera efectiva, mejorando la capacidad de descubrimiento.

10. Innovación continua

El equipo de desarrollo activo de React introduce constantemente nuevas características y mejoras, como la renderización concurrente y los componentes de servidor en React 19. Este compromiso con la innovación garantiza que React siga siendo una herramienta de vanguardia para el desarrollo web.

• Comprender el ecosistema React

El ecosistema React se refiere a la amplia colección de herramientas, bibliotecas y marcos que amplían la funcionalidad de React, lo que permite a los desarrolladores crear aplicaciones ricas en funciones con facilidad.

Comprender el ecosistema es crucial para aprovechar al máximo el potencial de React, ya que permite a los desarrolladores afrontar diversos desafíos de forma eficiente. A continuación, se presenta un análisis detallado de los elementos clave del ecosistema de React.

1. Biblioteca principal de React

En el corazón del ecosistema se encuentra la biblioteca principal de React, que proporciona los componentes esenciales para crear componentes, gestionar el estado y renderizar la interfaz de usuario. La naturaleza modular de React le permite integrarse a la perfección con otras herramientas, sirviendo como base del ecosistema.

2. Bibliotecas de Gestión Estatal

Administrar el estado de la aplicación es un aspecto fundamental del desarrollo web moderno, y React ofrece múltiples soluciones:

- Redux: Una popular biblioteca de gestión de estados que proporciona un almacenamiento centralizado para los datos de la aplicación. Redux simplifica las actualizaciones de estado y garantiza la coherencia entre los componentes.
- API de contexto: Una función integrada de React para gestionar el estado global sin depender de bibliotecas de terceros. Ideal para aplicaciones pequeñas y medianas.
- React Query: una herramienta para administrar el estado del lado del servidor, manejar el almacenamiento en caché y sincronizar datos con API de manera eficiente.

3. Enrutamiento con React Router

React Router es la biblioteca de facto para implementar el enrutamiento en aplicaciones React. Permite a los desarrolladores:

- Cree aplicaciones dinámicas de varias páginas con facilidad.
- Manejar rutas anidadas, redirecciones y protectores de rutas.
- Mejore la experiencia del usuario con una navegación fluida.

4. Soluciones de estilo

El ecosistema React ofrece una variedad de opciones de estilo, lo que permite a los desarrolladores elegir la que mejor se adapte a sus proyectos:

- Bibliotecas CSS-in-JS: Herramientas como Styled Components y Emotion permiten a los desarrolladores escribir CSS directamente en JavaScript.
- Módulos CSS:

Un enfoque modular para el estilo que evita conflictos de nombres.

- Bibliotecas de utilidad: Tailwind CSS y herramientas similares proporcionan Clases de utilidad prediseñadas para un desarrollo rápido de UI.

5. Marcos y extensiones

Marcos como Next.js y Gatsby amplían las capacidades de React, permitiendo funciones avanzadas:

- Next.js: ofrece renderizado del lado del servidor, generación de sitios estáticos y rutas API para mejorar el rendimiento y el SEO.
- Gatsby: un generador de sitios estáticos optimizado para crear sitios web rápidos y basados en contenido.

6. Herramientas de desarrollo

El ecosistema React incluye herramientas para mejorar la productividad de los desarrolladores:

- React Developer Tools: una extensión del navegador para inspeccionar componentes React y depurar aplicaciones.
- Storybook: una herramienta para desarrollar y probar la interfaz de usuario componentes de forma aislada.
- CodeSandbox: un IDE en línea para crear prototipos de React aplicaciones.

7. Marcos de prueba

El ecosistema de React admite flujos de trabajo de pruebas integrales:

- Biblioteca de pruebas de React: se centra en probar aplicaciones comportamiento desde la perspectiva del usuario.
- Jest: Un potente marco de pruebas para unidades y pruebas de integración.
- Cypress: ideal para pruebas de extremo a extremo de React aplicaciones.

8. Integración de backend

React se integra sin problemas con varias tecnologías de backend:

- GraphQL: Herramientas como Apollo Client simplifican los datos Obtención y gestión de estados con API GraphQL.
- API REST: Bibliotecas como Axios manejan HTTP solicitudes de manera eficiente.

9. Bibliotecas de animación

Para agregar efectos dinámicos, el ecosistema ofrece bibliotecas como:

- Framer Motion: una potente biblioteca para crear animaciones y transiciones.
- React Spring: una biblioteca de animación basada en física para movimiento de fluido.

El ecosistema React es amplio y está en constante evolución, ofreciendo herramientas para cada etapa del desarrollo de aplicaciones. Al dominar estas herramientas, Los desarrolladores pueden crear aplicaciones robustas y de alto rendimiento que satisfacer las demandas web modernas.

Capítulo 2.

Configuración de su entorno

- Instalación de Node.js y npm

Muéstrame cómo hacer esto de manera efectiva en un contenido de 1200 palabras.

- Crear un proyecto React con Create React App

Instalación de Node.js y npm

Node.js y npm (Node Package Manager) son herramientas esenciales para desarrollar aplicaciones React. Node.js es un entorno de ejecución que permite ejecutar JavaScript fuera del navegador, mientras que npm es el gestor de paquetes predeterminado de Node.js, utilizado para gestionar bibliotecas y dependencias en los proyectos. A continuación, se incluye una guía detallada sobre cómo instalar Node.js y npm eficazmente en diferentes sistemas operativos.

Paso 1 Verifique la instalación existente

Antes de instalar Node.js, compruebe si ya está instalado en su sistema. Abra una terminal o un símbolo del sistema y ejecute:

```
nodo -v  
npm -v
```

Si los comandos devuelven números de versión, Node.js y npm ya están instalados. De lo contrario, continúe con los pasos a continuación.

Paso 2 Descargar Node.js .

1. Seleccione la versión correcta
 - LTS (soporte a largo plazo): recomendado para la mayoría

usuarios, especialmente para entornos de producción.

- Versión actual: incluye las últimas funciones, pero podría no sea tan estable.

2. Descargue el instalador

Dependiendo de su sistema operativo (Windows, macOS o Linux), Descargue el instalador de Node.js apropiado.

Paso 3. Instalación en Windows

1. Ejecute el instalador

Haga doble clic en el archivo .msi y siga las instrucciones de instalación.

2. Configuración de instalación

- Acepte el acuerdo de licencia.
- Seleccione el directorio de instalación (el predeterminado es recomendado).
- Asegúrese de que la opción para agregar Node.js a PATH esté activada. comprobado.

3. Verificar la instalación

Abra el símbolo del sistema y escriba:

```
nodo -v
```

```
npm -v
```

Esto debería mostrar las versiones instaladas de Node.js y npm.

Paso 4. Instalación en macOS

1. Ejecute el instalador

Ejecute el archivo .pkg y siga las instrucciones para completar la instalación.

2. Uso de un administrador de paquetes (opcional)

Alternativamente, puede utilizar un administrador de paquetes para instalar Node.js. Para Por ejemplo, con Homebrew:

```
nodo de instalación de brew
```

3. Verificar la instalación

Abra la terminal y escriba:

```
nodo -v
```

```
npm -v
```

Paso 5 Instalación en Linux

1. Uso de un administrador de paquetes

Para sistemas basados en Debian:

```
sudo apt update
```

```
sudo apt install nodejs npm
```

Para sistemas basados en Red Hat:

```
sudo dnf install nodejs
```

2. Uso del Administrador de versiones de nodo (nvm)

Si necesita varias versiones de Node.js, utilice nvm:

```
curl -o- <URL del script de instalación de nvm> | bash
```

```
source ~/.bashrc
```

```
nvm install --lts
```

3. Verificar la instalación

Después de la instalación, ejecute:

```
nodo -v
```

```
npm -v
```

Paso 6 Actualización de Node.js y npm

1. Actualizar npm Para

asegurarse de tener la última versión de npm:

```
npm install -g npm
```

2. Actualizar Node.js Si

usa nvm, puede actualizar Node.js con:

```
nvm instalar nodo
```

```
nvm usar nodo
```

Paso 7 Instalar paquetes npm globales

Una vez instalado npm, considere instalar herramientas globales útiles como:

- Crear una aplicación React:

```
npm install -g create-react-app
```

- npm-check-actualizaciones:

```
npm install -g npm-check-updates
```

Estas herramientas simplifican el desarrollo de React y la gestión de dependencias.

- Creación de un proyecto React con Create React App

Después de instalar Node.js y npm, puedes crear un proyecto React usando Create React App, una herramienta que configura un proyecto React con una configuración mínima.

Paso 1 ¿Qué es Create React App? ?

Create React App es una herramienta con soporte oficial para crear aplicaciones React. Preconfigura herramientas esenciales como Babel y Webpack, lo que proporciona una experiencia de desarrollo optimizada sin necesidad de configuración manual.

Paso 2 Instalar la aplicación Create React

1. Abra su terminal o símbolo del sistema.
2. Ejecute el siguiente comando para instalar globalmente Create React App:

```
npm install -g create-react-app
```

3. Verifique la instalación escribiendo:

```
crear-react-app --versión
```

Esto debería mostrar el número de versión.

Paso **3** crear un nuevo proyecto React

1. Navegue hasta el directorio de su proyecto:

```
cd ruta/a/su/directorio
```

2. Crea un nuevo proyecto React:

```
npx create-react-app mi-aplicación
```

Reemplaza "my-app" por el nombre que desees para tu proyecto. Usar npx garantiza usar la última versión de Create React App.

3. Espere a que finalice la instalación. La herramienta creará la carpeta del proyecto e instalará todas las dependencias necesarias.

Paso **4** Explorar la estructura del proyecto

Navegue hasta el directorio del proyecto:

```
cd mi-aplicación
```

La estructura del proyecto incluye:

- node_modules/: Contiene dependencias instaladas.

- public/: Recursos estáticos como HTML e imágenes. src/:
- Directorio principal para el código de la aplicación.
- package.json: Archivo de configuración que enumera las dependencias.

y guiones.

Paso 5 Iniciar el servidor de desarrollo

Para iniciar el servidor de desarrollo y ver su aplicación en un navegador, ejecute:

```
inicio de npm
```

Esto iniciará la aplicación en su navegador predeterminado en localhost:3000. El servidor de desarrollo se actualiza automáticamente cada vez que realiza cambios en su código.

Paso 6 Modificar el componente de la aplicación

1. Abra el proyecto en un editor de código (por ejemplo, Visual Studio Código).

Localice el archivo App.js dentro del directorio src/. 2.

3. Reemplace el código predeterminado con:

```
función App()  
{ return  
  ( <div>  
    <h1>¡Hola, React!</h1>  
    <p>Este es mi primer proyecto React.</p> </  
    div> ); }  
  
exportar aplicación predeterminada;
```

4. Guarde el archivo y observe los cambios en el navegador.

Paso 7 Agregar funciones adicionales

Ahora que la configuración básica está completa, puede comenzar a crear componentes, diseñar su aplicación y conectar API para crear una aplicación completamente

aplicación funcional.

Paso 8: Prepara la aplicación para producción

Cuando su aplicación esté lista para su implementación, cree una versión de producción optimizada utilizando:

```
npm ejecutar compilación
```

Esto genera una carpeta build/ que contiene la aplicación minimizada y optimizada, lista para su implementación en un servidor web.

Mejores prácticas para usar Create React App

1. Organice los componentes: mantenga sus componentes bien organizados en subdirectorios para una mejor capacidad de mantenimiento.
2. Utilice el control de versiones: realice un seguimiento de los cambios mediante Git u otro sistema de control de versiones.
3. Actualice periódicamente las dependencias: mantenga las dependencias actualizadas para garantizar la seguridad y el rendimiento.

Siguiendo estos pasos, tendrás una base sólida para desarrollar aplicaciones React de forma eficiente. ¡Avísame si necesitas más información!

• Estructura de carpetas y anatomía del proyecto

Una estructura de carpetas bien organizada es esencial para gestionar y escalar aplicaciones React. Comprender la estructura de carpetas predeterminada que ofrece Create React App (CRA) y adaptarla a las necesidades de su proyecto hará que su proceso de desarrollo sea más eficiente y fácil de mantener. Esta guía ofrece un análisis detallado de la estructura predeterminada y las mejores prácticas para organizar su proyecto React.

1. Estructura de carpetas predeterminada

Cuando creas un proyecto React usando Create React App, se genera la siguiente estructura:

2. Descripción general de cada carpeta y archivo

- Contiene todas las dependencias y módulos instalados a través de npm o yarn.
- Se gestiona automáticamente, por lo que no es necesario modificar nada aquí.
- Esta carpeta puede crecer mucho, pero es esencial para la funcionalidad del proyecto.

público/

- Almacena archivos estáticos y activos que no se procesarán por webpack.
- Los archivos en esta carpeta se sirven tal como están, sin modificaciones.

Archivos clave:

- index.html: El archivo HTML principal de tu aplicación. React monta su aplicación en `<div id="root"></div>` dentro de este archivo.
 - favicon.ico: El icono que se muestra en la pestaña del navegador.
 - manifest.json: Metadatos para aplicaciones web progresivas (PWA).
 - robots.txt: proporciona instrucciones a los rastreadores web para fines SEO.
-

fuentes/

- El núcleo de tu aplicación. Aquí es donde se encuentran todas tus componentes, estilos y lógica de React residen.

Archivos clave:

- App.js: El componente principal que sirve como entrada punto para la interfaz de usuario de su aplicación.
- App.css: Estilos específicos del componente App.js.
- index.js: el archivo JavaScript raíz que representa su aplicación al DOM usando ReactDOM.render().
- index.css: Estilos globales para su aplicación.
- reportWebVitals.js: Un archivo de monitoreo de rendimiento que registra métricas.
- setupTests.js: Se utiliza para configurar pruebas configuraciones.

Otros archivos

- `.gitignore`: especifica los archivos y directorios que Git debe ignorar.
- `package.json`: Contiene metadatos sobre su proyecto, incluyendo dependencias, scripts y detalles del proyecto.
 - `README.md`: Un archivo markdown con instrucciones para tu proyecto.

3. Mejores prácticas para la personalización de la estructura de carpetas

A medida que su proyecto crece, la estructura predeterminada podría no ser suficiente.

A continuación se muestran las mejores prácticas para organizar su proyecto:

Organización de carpetas basada en componentes

Cree una carpeta 'componentes/' para almacenar componentes de IU reutilizables.

fuentes/

```
|— componentes/
| |— Botón/
| | |— Botón.js
| | |— Botón.css
| | |— Botón.test.js
| |— Encabezado/
| | |— Encabezado.js
| |— Encabezado.css
```

Cada componente debe tener su propia carpeta que contenga:

- Archivo JavaScript: define el componente.
 - Archivo CSS: contiene estilos específicos del componente.
 - Archivo de prueba: incluye pruebas para el componente.
- funcionalidad.

Separación de funciones y páginas

Organice las funciones y páginas de la aplicación en carpetas dedicadas.

fuentes/

```
├── características/
|   ├── Aut/
|       ├── Login.js
|       ├── Registro.js
|       └── authSlice.js
|   ├── Tablero/
|       ├── Dashboard.js
|   └── Panel de control.css
```

Gestión estatal centralizada

Si utiliza una biblioteca de gestión de estados como Redux o Context API,
Crea una carpeta dedicada para la lógica relacionada con el estado.

fuentes/

```
├── estado/
|   ├── tienda.js
|   ├── acciones.js
|   └── reducers.js
```

Utilidades y ayudantes

Almacene las funciones de utilidad y los scripts auxiliares en una carpeta separada.

fuentes/

```
├── utilidades/
|   ├── api.js
|   └── formatDate.js
```

4. Ejemplo de estructura de carpetas avanzada

A continuación se muestra un ejemplo de una estructura de carpetas avanzada para una aplicación escalable.
Aplicación React:

fuentes/

—	activos/	# Activos estáticos como imágenes y fuentes
—	componentes/	# Componentes de interfaz de usuario reutilizables
—	características/	# Componentes y lógica específicos de cada función
—	ganchos/	# Ganchos de React personalizados
—	diseños/	# Componentes de diseño (por ejemplo, encabezado, pie de página)
—	páginas/	# Componentes de la página (por ejemplo, Inicio, Acerca de)
—	rutas/	# Lógica de enrutamiento de la aplicación
—	servicios/	# Servicios API
—	estilos/	— # Estilos globales y basados en temas
pruebas/		# Pruebas unitarias y de integración
—	utilidades/	# Funciones de utilidad y ayudantes

• Configuración de herramientas de desarrollo (IDE, Extensiones y linters)

Configurar un entorno de desarrollo eficiente es crucial para una experiencia de desarrollo de React fluida. Esto implica elegir el entorno de desarrollo integrado (IDE) adecuado, instalar extensiones para aumentar la productividad y configurar linters para el mantenimiento de la calidad del código. A continuación, se muestra una guía detallada paso a paso.

1. Elección de un IDE

Si bien hay varios IDE disponibles, Visual Studio Code (VS Code) es la opción más popular entre los desarrolladores de React debido a sus características y una amplia biblioteca de extensiones.

Pasos para instalar y configurar VS Code

1. Descargue e instale VS Code desde su fuente oficial.
2. Abra VS Code y configure su tema y preferencias

Para la mejor experiencia.

3. Instale las siguientes extensiones de clave (explicadas a continuación).

2. Extensiones esenciales para el desarrollo de React

Las extensiones mejoran la productividad al automatizar tareas repetitivas y Mejorando la legibilidad del código.

1. Fragmentos de código de React/Redux/React-Native de ES7+
 - Proporciona fragmentos de código para componentes y ganchos de React.
- y Redux.
- Uso: Escriba rafce para generar un componente funcional
- plantilla.
2. Prettier - Formateador de código
 - Formatea automáticamente su código según lo definido
- normas.
- Ayuda a mantener un estilo de codificación consistente.
3. Herramientas para desarrolladores de React
 - Depure componentes de React directamente desde su navegador.
4. Colorizador de pares de soportes
 - Resalta los corchetes coincidentes, mejorando el código
- legibilidad.
5. Path Intellisense
 - Completa automáticamente las rutas de archivos en las declaraciones de importación.
6. GitLens
 - Mejora la integración de Git, mostrando cambios de código y
- Historial en línea.
7. Tema de icono de material
 - Agrega íconos visualmente atractivos para diferentes tipos de archivos.

3. Configuración de linters y formateadores

Los linters y formateadores garantizan que su código cumpla con las mejores prácticas y está libre de errores de sintaxis.

Instalación de ESLint

1. Instalar ESLint globalmente usando npm:

```
npm install eslint --save-dev
```

2. Inicialice ESLint en su proyecto:

```
npx eslint --init
```

Elija las siguientes opciones durante la configuración:

- Utilice una guía de estilo popular (por ejemplo, Airbnb).
- Utilice módulos JavaScript (importación/exportación).
- Instalar las dependencias necesarias.

3. Cree un archivo `.eslintrc.json` para definir sus reglas de pelusa.

Configurando Prettier

1. Instalar Prettier:

```
npm install --save-dev más bonito
```

2. Cree un archivo `.prettierrc` para definir reglas de formato, como

como:

```
{ "semi":  
  verdadero, "singleQuote":  
  verdadero, "trailingComma":  
  "es5" }
```

Integración de ESLint y Prettier

1. Instalar el complemento necesario:

```
npm install eslint-config-prettier eslint-plugin-prettier --save-dev
```

2. Actualice `.eslintrc.json` para incluir:

```
{ "extiende": ["eslint:recommended", "plugin:react/recommended", "más bonito"] }
```



```
}
```

4. Configuración de la depuración en VS Code

1. Abra el panel de depuración y haga clic en “crear un archivo launch.json”.
 2. Seleccione “Node.js” como entorno.
 3. Agregue la siguiente configuración:
-

```
{ "tipo": "nodo",  
  "solicitud": "iniciar",  
  "nombre": "Iniciar programa",  
  "skipFiles": ["<node_internals>/**"],  
  "programa": "${workspaceFolder}/src/index.js" }
```

5. Integración de Git para el control de versiones

1. Inicialice Git en su proyecto:

```
git init
```

2. Agregue un archivo .gitignore para excluir archivos innecesarios, como

como:

```
/módulos_de_nodo/ /
```

```
compilación/ /.env
```

3. Realice cambios periódicamente para realizar un seguimiento del progreso.

Siguiendo estos pasos, tendrás un entorno de desarrollo robusto, adaptado al desarrollo con React. Un IDE correctamente configurado, extensiones esenciales y linters no solo aumentarán la productividad, sino que también garantizarán que tu código sea limpio y fácil de mantener.

Capítulo 3.

Conceptos básicos de React

- **JSX: Extensión de sintaxis de JavaScript**

JSX (Extensión de Sintaxis de JavaScript o XML de JavaScript) es una sintaxis utilizada en React para describir la apariencia de la interfaz de usuario (IU). Permite a los desarrolladores escribir código HTML directamente en JavaScript, lo que facilita la definición y visualización de componentes. JSX conecta las capas de lógica y presentación, proporcionando una experiencia de desarrollo fluida.

En esencia, JSX es una sintaxis simplificada para la función `React.createElement()`. Esto significa que, en segundo plano, el código JSX se convierte en JavaScript simple, que React utiliza para crear y gestionar los elementos del DOM.

Sintaxis básica de JSX

Aquí hay un ejemplo de JSX:

```
const elemento = <h1>¡Hola, mundo!</h1>;
```

Este fragmento de código JSX crea un elemento React que muestra "¡Hola mundo!" en la pantalla. Tras bambalinas, se traduce a:

```
const elemento = React.createElement('h1', null, '¡Hola, mundo!');
```

Características principales de JSX

1. Incrustar expresiones JSX le permite incrustar expresiones JavaScript dentro de llaves `{}`.

Ejemplo:

```
const nombre = 'React';  
const elemento = <h1>¡Bienvenido a {name}!</h1>;
```

2. Atributos en JSX

Los atributos en JSX se parecen mucho a los atributos HTML, pero con algunas diferencias debido a las reglas de sintaxis de JavaScript.

Ejemplo:

```
elemento constante = ;
```

Diferencias: :

- clase se convierte en className.
- for se convierte en htmlFor.

3. Elementos anidados

JSX le permite anidar elementos, lo que posibilita la creación de interfaces de usuario complejas.

Ejemplo:

```
const element =  
  ( <div>  
    <h1>¡Bienvenido!</h1>  
    <p>Esta es una aplicación React.</p> </  
    div> );
```

4. Etiquetas de cierre

automático Al igual que HTML, algunas etiquetas en JSX son de cierre automático.

Ejemplo:

```
const elemento = <input type="text" placeholder="Ingresar texto" />;
```

5. Condicionales con JSX

Puedes usar la lógica de JavaScript para representar elementos condicionalmente.

Ejemplo:

```
const isLoggedIn = true; const  
element = isLoggedIn ? <h1>¡Bienvenido de nuevo!</h1> : <h1>Por favor, inicia sesión.</h1>;
```

¿Por qué utilizar JSX?

1. Legibilidad mejorada JSX hace que el código sea más legible y expresivo al combinar HTML y JavaScript en un solo archivo.
2. Depuración más sencilla JSX proporciona mensajes de error claros, lo que facilita la depuración de problemas en el código.
3. Integración con componentes React JSX se integra perfectamente con los componentes React, lo que le permite definir y administrar las interfaces de usuario de forma más eficiente.

Conceptos avanzados en JSX

1. Elementos de estilo JSX admite estilos en línea utilizando un objeto JavaScript.
Ejemplo:

```
const style = { color: 'azul', tamaño de fuente: '20px' }; const  
element = <h1 style={style}>Texto con estilo</h1>;
```

2. Fragmentos Los fragmentos le permiten agrupar múltiples elementos sin agregar nodos adicionales al DOM.

Ejemplo:

```
elemento constante = (  
  <>  
    <h1>Título</h1>  
    <p>Párrafo</p> </> );
```

3. Manejo de eventos JSX

admite controladores de eventos escritos en sintaxis camelCase.

Ejemplo:

```
const handleClick = () => alert('¡Botón hecho clic!'); const  
element = <button onClick={handleClick}>Haz clic aquí</button>;
```

Limitaciones de JSX

1. Curva de aprendizaje

A los principiantes puede resultarles desconocida la sintaxis JSX al principio, ya que mezcla HTML y JavaScript.

Requiere compilación 2.

JSX debe compilarse en JavaScript simple utilizando herramientas como Babel, agregando un paso de compilación al proceso de desarrollo.

3. Verbosidad

Las estructuras JSX complejas pueden volverse verbosas y más difíciles de administrar.

- Componentes: Componentes funcionales y de clase

¿Qué son los componentes? ?

En React, los componentes son los componentes básicos de la interfaz de usuario (IU). Son fragmentos de código independientes y reutilizables que definen la apariencia y el comportamiento de una parte específica de la aplicación. Los componentes se pueden clasificar en componentes funcionales y componentes de clase, cada uno con características distintivas.

Componentes funcionales

Los componentes funcionales son funciones de JavaScript que devuelven JSX. Son simples, livianos y adecuados para la mayoría de las aplicaciones React modernas.

Ejemplo de un componente funcional

```
función Bienvenido(props) {  
  devolver <h1>¡Hola, {props.name}!</h1>;  
}
```

Aquí, el componente de bienvenida toma props como argumento y devuelve JSX.

Características de los componentes funcionales

1. Sin estado vs. con estado
 - Inicialmente, los componentes funcionales no tenían estado.
 - Con la introducción de React Hooks, los recursos funcionales

Los componentes ahora pueden gestionar el estado y los efectos secundarios.

Ejemplo con ganchos:

```
importar { useState } de 'react';
```

```
función Contador() {  
  const [count, setCount] = useState(0);  
  
  devolver (  
    <div>  
      <p>Conde: {count}</p>  
      <botón onClick={() => setCount(contar +  
1)}}>Incrementar</button>  
    </div>  
  );  
}
```

2. Sencillez

Los componentes funcionales son más fáciles de leer, probar y depurar debido a su estructura simple.

3. Rendimiento

Los componentes funcionales son un poco más rápidos porque no tienen la sobrecarga de los métodos del ciclo de vida de React.

Componentes de clase

Los componentes de clase son clases ES6 que extienden `React.Component`. Son más ricos en características que los componentes funcionales y eran la forma principal de administrar los métodos de estado y ciclo de vida antes de que se introdujeran los Hooks.

Ejemplo de un componente de clase

```
importar React, { Componente } de 'react';

clase Bienvenido extiende Componente { render()
  { return
    <h1>¡Hola, {this.props.name}!</h1>; } }
```

Características de los componentes de clase

1. Los componentes de la clase Gestión de estado gestionan el estado mediante el objeto `this.state`.

Ejemplo:

```
clase Contador extiende Componente
  { constructor(props)
    { super(props);
      this.state = { count: 0 }; }

  incremento = () =>
    { this.setState({ count: this.state.count + 1 }); };

  render()
    { return
      ( <div>
        <p>Conteo: {this.state.count}</p> <button
          onClick={this.increment}>Incremento</button> </div> ); } }
```

```
}
```

2. Métodos de ciclo de vida

Los componentes de clase tienen métodos de ciclo de vida como `componentDidMount`, `componentDidUpdate` y `componentWillUnmount`. Estos métodos permiten a los desarrolladores ejecutar lógica durante fases específicas del ciclo de vida de un componente.

Ejemplo:

```
clase Timer extiende Component
```

```
  { componentDidMount()  
    { console.log('¡Componente montado!'); }  
  }
```

```
  componentWillUnmount()  
    { console.log('¡El componente se desmontará!'); }  
  }
```

```
  render()  
    { return <h1>Temporizador</h1>; } }
```

```
h1>; } }
```

3. Complejidad Los

componentes de clase son más complejos debido al uso de este y de los métodos de ciclo de vida.

Cuándo utilizar cada uno

- Componentes funcionales: se utilizan en la mayoría de los escenarios, especialmente al crear aplicaciones React modernas con Hooks.
- Componentes de clase: Úselo solo cuando trabaje con bases de código más antiguas o cuando se requieran métodos de ciclo de vida heredados.

Transición a componentes funcionales

Con la introducción de los Hooks en React 16.8, el enfoque se ha desplazado hacia los componentes funcionales. Los Hooks proporcionan equivalentes

funcionalidad a los componentes de clase pero con una API más simple e intuitiva.

Tanto JSX como los componentes de React (funcionales y de clase) constituyen la base del desarrollo de React. JSX optimiza el proceso de creación de interfaces de usuario (UI) al combinar la potencia de JavaScript y HTML, mientras que los componentes permiten a los desarrolladores crear bases de código modulares, reutilizables y fáciles de mantener. Los componentes funcionales, potenciados por Hooks, son ahora el enfoque preferido, pero comprender los componentes de clase sigue siendo valioso para trabajar con bases de código heredadas. Al dominar estos conceptos fundamentales, los desarrolladores pueden crear aplicaciones React dinámicas y eficientes.

• Props: Pasar datos a componentes

En React, las props (abreviatura de "propiedades") son un mecanismo para pasar datos de un componente principal a un componente secundario. Las props hacen que los componentes sean dinámicos y reutilizables, ya que permiten que reciban datos externos y adapten su comportamiento o apariencia en consecuencia. Esta guía ofrece una explicación paso a paso de las props, su funcionamiento y las mejores prácticas para usarlas eficazmente en tus aplicaciones React.

1. ¿Qué son las propiedades en React?

Las propiedades son objetos que se pasan a los componentes como argumentos. Contienen datos o atributos a los que un componente puede acceder y mostrar.

A diferencia del estado, que es local y mutable dentro de un componente, las propiedades son inmutables y administradas externamente por el componente principal.

Características principales de los accesorios:

- Solo lectura: los accesorios no se pueden modificar dentro del niño componente.
- Flujo de padre a hijo: los datos fluyen unidireccionalmente desde El componente padre al componente hijo.

- Reutilizable: Los accesorios permiten reutilizar los componentes con diferentes datos.

2. Definición de propiedades en componentes funcionales

Las propiedades se pasan a los componentes funcionales como parámetros. Veamos cómo definirlos y usarlos:

Paso 1: Crear un componente funcional

Un componente funcional toma propiedades como argumento.

```
función Saludo(props) { devolver  
  <h1>¡Hola, {props.name}!</h1>; }
```

Paso 2: Pasar accesorios de un componente principal

Los accesorios se pasan a un componente secundario como atributos.

```
función App() { return  
  <Saludo nombre="Juan" />; }
```

En este ejemplo: :

- La propiedad `name` se pasa del componente `App` al componente `Saludo`.
- El componente `Saludo` usa `props.name` para mostrar `"Hola, John!"`.

3. Definición de propiedades en componentes de clase

Para los componentes de clase, se accede a las propiedades mediante `this.props`.

Paso 1: Crear un componente de clase

Utilice `React.Component` para crear un componente basado en clases.

```
importar React, { Component } de 'react';
```

```
clase Saludo extiende Componente { render()
  { return
    <h1>¡Hola, {this.props.name}!</h1>; } }
```

Paso 2 Pasar accesorios

Los accesorios se pasan del mismo modo que en los componentes funcionales.

```
función App()
  { return <Saludo nombre="Juan" />; }
```

4. Pasar múltiples accesorios

Puede pasar múltiples propiedades a un componente agregando múltiples atributos.

```
función UserCard(props) { return
  ( <div>

    <h2>{props.name}</h2>
    <p>Edad: {props.age}</p> </
  div> ); }
```

```
función App()
  { return <UserCard name="Alice" age={25} />; }
```

El componente UserCard recibe tanto el nombre como la edad como accesorios y los representa en consecuencia.

5. Pasar valores dinámicos como propiedades

Las propiedades pueden aceptar datos dinámicos, como variables, expresiones o funciones.

Ejemplo 1 Usar variables

```
const nombreUsuario = "Charlie";  
función App()  
  { return <Saludo nombre={nombreUsuario} />; }
```

Ejemplo 2 Usar expresiones

```
función App()  
  { return <UserCard name="Eva" age={2024 - 1990} />; }
```

Ejemplo 3 Usar funciones

```
función Botón(props) { return  
  <button onClick={props.onClick}>{props.label}</button>; }  
  
función App()  
  { const handleClick = () => alert("¡Botón hecho clic!"); return  
    <Button label="Click Me" onClick={handleClick} />; }
```

6 Propiedades predeterminadas

Las propiedades predeterminadas proporcionan un valor de respaldo cuando no se pasa ninguna propiedad a un componente. Se pueden definir propiedades predeterminadas tanto en componentes funcionales como de clase.

Ejemplo en componentes funcionales

```
función Saludo(props) { devolver  
  <h1>¡Hola, {props.name}!</h1>; }
```

```
}
```

```
Saludo.defaultProps = { nombre:  
  "Invitado", };
```

```
función App() { return  
  <Saludo />; }
```

Si no se pasa ninguna propiedad de nombre, se utiliza el valor predeterminado "Invitado".

Ejemplo en componentes de clase

```
clase Saludo extiende React.Component { static  
  defaultProps = { nombre:  
    "Invitado", };  
  
  render()  
    { return <h1>¡Hola, {this.props.name}!</h1>; } }
```

7. Validación de propiedades con PropTypes

Para garantizar que los componentes reciban el tipo correcto de datos, puede utilizar PropTypes para la validación.

Paso 1 **Importar** PropTypes

```
importar PropTypes desde 'prop-types';
```

Paso 2 **Definir** PropTypes

```
función UserCard(props) { return  
  ( <div>  
  
    <h2>{props.name}</h2>
```

```
    <p>Edad: {props.age}</p> </div> ); }
```

```
UserCard.propTypes = { nombre:  
  PropTypes.string.isRequired, edad:  
  PropTypes.number, };
```

En este ejemplo:

- El nombre debe ser una cadena y es obligatorio.
- La edad es opcional, pero debe ser un número si se proporciona.

8. Pasarles accesorios a los niños

A veces, puede que necesites pasar datos a componentes secundarios. La propiedad `props.children` permite anidar componentes.

Ejemplo: Uso de `props.children`

```
función Tarjeta(props)  
  { devolver <div className="tarjeta">{props.niños}</div>; }
```

```
función App() { return  
  ( <Card>  
    <h1>Título  
      de la tarjeta</h1> <p>Este  
        es el contenido de la tarjeta.</p> </Card> ); }
```

Aquí, el componente `Tarjeta` envuelve a sus hijos (los elementos `h1` y `p`).

9. Pasar objetos como accesorios

En lugar de pasar múltiples propiedades individuales, puedes pasar un objeto y desestructurarlo.

Ejemplo: Pasar un objeto

```
función Perfil({ usuario }) { return  
  ( <div>  
  
    <h2>{usuario.nombre}</h2>  
    <p>Edad: {usuario.edad}</p> </  
    div> ); }
```

```
función App() { const  
  userInfo = { nombre: "Dave", edad: 30 }; return <Perfil  
  usuario={userInfo} />; }
```

10. Representación condicional con accesorios

Puedes usar accesorios para renderizar componentes o elementos de forma condicional.

Ejemplo: Representación condicional

```
función StatusMessage(props) { return  
  ( <div>  
  
    {props.isLoggedIn ? <h1>¡Bienvenido de nuevo!</h1> : <h1>Inicie sesión</h1>} </  
    div> ); }
```

```
función App() { return  
  <MensajeDeEstadoisIniciado={verdadero} />; }
```

11. Pasar accesorios en bucles

Al representar listas de componentes, puedes pasar propiedades dinámicamente usando bucles como `map()`.

Ejemplo: Representación de una lista

```
función UserList(props) { return
  ( <ul>

    {props.users.map((usuario, índice) => ( <li
      key={índice}>{usuario}</li> ))} </

    ul> ); }
```

```
función App() { const
  nombresDeUsuario = ["Alice", "Bob", "Charlie"]; return
  <ListaDeUsuarios usuarios={NombresDeUsuario} />; }
```

12. Mejores prácticas para el uso de accesorios

1. Utilice `PropTypes` para la validación: valide siempre los props para evitar errores de tiempo de ejecución.
2. Evite mutar las propiedades: las propiedades son de solo lectura; modifique el estado para lograr un comportamiento dinámico.
3. Mantenga los accesorios al mínimo: pase solo los datos requeridos por el componente para evitar una complejidad innecesaria.
4. Use la desestructuración: Desestructura los accesorios para un mayor limpieza. código.

Ejemplo:

```
función Saludo({ nombre }) { devolver
  <h1>¡Hola, {nombre}!</h1>; }
```

5. Propiedades del componente del documento: describa claramente las propiedades esperadas en los comentarios o la documentación.

13. Errores comunes y cómo evitarlos

1. Acceso a propiedades no definidas: defina siempre propiedades predeterminadas para evitar acceder a valores no definidos.

Tipos de datos incorrectos: utilice PropTypes para garantizar que se pase el 2. tipo de datos correcto.

3. Sobrecarga de accesorios: evite pasar accesorios innecesarios para mantener el componente enfocado.

Las props son uno de los conceptos fundamentales de React, ya que permiten que los componentes sean reutilizables, dinámicos y versátiles. Al comprender cómo definir, pasar y validar props, se pueden crear aplicaciones React más robustas y fáciles de mantener. Dominar las props sienta las bases para conceptos más avanzados, como la gestión de estados y la API de contexto.

• Conceptos básicos de gestión estatal

El estado es uno de los conceptos fundamentales de React, ya que permite a los componentes mantener y gestionar datos dinámicos. Mientras que las propiedades permiten el flujo de datos de los componentes principales a los secundarios, el estado permite que los componentes gestionen sus propios datos locales y vuelvan a renderizar la interfaz de usuario cuando estos cambian. La gestión del estado es esencial para crear aplicaciones interactivas y responsivas. Esta guía abarca los fundamentos de la gestión del estado en React, explicando cómo definir, actualizar y utilizar el estado eficazmente.

1. ¿Qué es el estado en React? ?

En React, el estado se refiere a un objeto JavaScript que contiene datos dinámicos o información sobre un componente. El estado es:

- Local del Componente: El Estado es privado y totalmente controlado por el componente en el que está definido.

- Mutable: a diferencia de las propiedades, el estado se puede actualizar utilizando las funciones de gestión de estado de React.
- Desencadena re-renderizados: cuando el estado cambia, React vuelve a renderizar automáticamente el componente para reflejar los datos actualizados.

Ejemplo: Estado en un componente funcional

```
importar React, { useState } de 'react';
```

```
función Contador() { const
```

```
  [count, setCount] = useState(0);
```

```
  return
```

```
    ( <div>
```

```
      <p>Recuento actual: {count}</p>
```

```
      <button      onClick={()      =>      setCount(contar      +
```

```
1))>Incremento</button> </
```

```
div> ); }
```

En este ejemplo:

- El gancho useState inicializa el estado con un valor de 0.
- La función setCount actualiza el estado.
- La interfaz de usuario se actualiza automáticamente cuando cambia el estado.

2. Cómo funciona el estado en React

La gestión estatal implica tres pasos clave:

1. Definición de estado: inicialice el estado utilizando useState (componentes funcionales) o this.state (componentes de clase).
2. Estado de lectura: accede al valor del estado actual.
3. Actualización de estado: utilice las funciones de gestión de estado para actualizar el estado.

3. Gestión de estados en componentes funcionales

Los componentes funcionales utilizan el gancho useState para la gestión del estado.

Paso 1 Importar useState

El gancho useState es parte de React y debe importarse:

```
importar React, { useState } de 'react';
```

Paso 2 Definir el estado

Llamar a useState con el valor del estado inicial:

```
const [estado, establecerEstado] = usarEstado(valorInicial);
```

- estado: El valor del estado actual.
- setState: Una función para actualizar el estado.

Paso 3 Actualizar el estado

Llama a setState con el nuevo valor:

```
setState(nuevoValor);
```

Ejemplo: Un botón de alternancia simple

```
función ToggleButton() { const  
  [isOn, setIsOn] = useState(false);  
  
  devolver  
  ( <button onClick={() => setIsOn(!isOn)}> {isOn ?  
    'ON' : 'OFF'} </button> ); }
```

4. Gestión de estados en componentes de clase

Los componentes de clase utilizan el objeto `this.state` para administrar el estado.

Paso 1: Inicializar el estado

El estado se inicializa en el constructor:

```
class Contador extiende React.Component
{ constructor(props)
  { super(props);
    this.state = { count: 0 }; } }
```

Paso 2: Leer el estado

Acceda al estado usando `this.state`:

```
render()
{ return <p>Conteo: {this.state.count}</p>; }
```

Paso 3: Actualizar el estado

Utilice `this.setState()` para actualizar el estado:

```
este.setState({ count: este.estado.count + 1 });
```

Ejemplo: Contador en un componente de clase

```
class Counter extiende React.Component
{ constructor(props)
  { super(props);
    this.state = { count: 0 }; }

  incremento = () =>
    { this.setState({ count: this.state.count + 1 }); }
```

```
render()
{ return
  ( <div>
    <p>Recuento actual: {this.state.count}</p> <button
    onClick={this.increment}>Incremento</button> </div> ); } }
```

5. Casos de uso comunes para el Estado

1. Entradas de formulario: administre los valores de entrada del usuario para los formularios.
2. Alternar elementos de la interfaz de usuario: mostrar u ocultar botones modales, menús desplegables o barras laterales.
3. Contadores: rastrea valores numéricos (por ejemplo, me gusta, votos positivos).
4. Listas dinámicas: agregue, elimine o modifique elementos de la lista de forma dinámica.
5. Datos API: almacene y administre datos obtenidos de las API.

6. Actualización correcta del estado

Utilice actualizaciones funcionales para el estado dependiente

Cuando el nuevo estado depende del estado anterior, pasa una función a setState:

```
setCount((prevCount) => prevCount + 1);
```

Evite la mutación de estado directa

Nunca modifique el estado directamente, ya que puede generar un comportamiento impredecible:

// Incorrecto

```
this.state.count = this.state.count + 1;
```

// Corrija

```
esto.setState({ count: this.state.count + 1 });
```

7. Gestión de estados complejos

A veces, el estado puede incluir varios valores relacionados. En tales casos, se pueden usar objetos o matrices para gestionar el estado.

Ejemplo: uso de un objeto para el estado

```
función Perfil() { const
  [usuario, setUser] = useState({ nombre: "", edad: "" });

  const updateName = (e) =>
    { setUser((prevUser) => ({ ...prevUser, nombre: e.target.value })); };

  devolver
  ( <div>
    <input type="text" onChange={updateName}
placeholder="Nombre" />
    <p>Nombre: {usuario.nombre}</p>
  </div> ); }
```

8. Elevando el Estado

Cuando varios componentes necesitan compartir el estado, puedes “elevar” el estado a un componente principal común.

Ejemplo: Elevación del Estado

```
función Padre() { const
  [estadoCompartido, establecerEstadoCompartido] = usarEstado("");

  devolver
  ( <div>
    <Estado Niño1={estadocompartido} setEstado={estadoCompartido} />
    <Child2 estado={estadocompartido} /> </div> ); }
```

```

}

función Child1({ estado, setState }) { return
  ( <valor
    de
      entrada={estado}
      onChange={(e) => setState(e.target.value)}
      placeholder="Escribe algo" /> ); }

```

```

función Child2({ estado })
  { devolver <p>Estado compartido: {estado}
  </p>; }

```

9. Gestión del Estado Global

Para aplicaciones más grandes, gestionar el estado localmente dentro de los componentes se vuelve un desafío. Las herramientas de gestión de estado global, como Context API, Redux o MobX, permiten gestionar el estado compartido de forma más eficaz.

Uso de la API de contexto para el estado global

```

importar React, { createContext, useContext, useState } de 'react';

const UserContext = crearContexto();

función App()
  { const [usuario, setUser] = useState({ nombre: 'Alice', edad: 25 });

  devolver
    ( <UserContext.Provider valor={usuario}>
      <Perfil /> </
      UserContext.Provider> ); }

```

```

función Perfil() {

```

```
const usuario = useContext(UserContext); return  
<p>Nombre: {usuario.nombre}</p>; }
```

10. Mejores prácticas para la gestión estatal

1. Mantener el estado mínimo: almacene únicamente los datos que sean necesarios para la representación.
2. Utilice el estado derivado con moderación: evite duplicar valores de estado que puedan derivarse de un estado existente.
3. Evite la superposición de estados: asegúrese de que los valores de estado no superposición entre componentes.
4. Separe la interfaz de usuario y la lógica empresarial: mantenga su estado lógica organizada y reutilizable.

11. Estado de depuración

Utilice las herramientas para desarrolladores de React

Instale la extensión React Developer Tools para inspeccionar y depurar el estado del componente.

Agregar registros para actualizaciones de estado

Registre los cambios de estado para comprender cómo y cuándo ocurren:

```
useEffect(() =>  
  { console.log('Estado actualizado:', estado); },  
[estado]);
```

La gestión de estados es un aspecto fundamental del desarrollo con React, ya que permite crear aplicaciones dinámicas, responsivas e interactivas. Al comprender cómo definir, actualizar y compartir estados eficazmente, se pueden gestionar fácilmente incluso escenarios complejos.

Si bien el estado local es adecuado para aplicaciones pequeñas, herramientas como Context API o Redux se vuelven esenciales a medida que la aplicación crece. Dominar la gestión de estados sienta las bases para crear proyectos React escalables y fáciles de mantener.

Capítulo 4.

Nuevas funciones de React 19

• Actualizaciones clave en React 19

React 19 presenta numerosas características y mejoras diseñadas para optimizar el rendimiento, simplificar los flujos de trabajo de desarrollo y proporcionar a los desarrolladores mejores herramientas para crear aplicaciones escalables y eficientes. Estas actualizaciones forman parte del compromiso continuo de React de modernizar la biblioteca y adaptarla a las necesidades del desarrollo web actual.

1. Representación simultánea

El renderizado concurrente es una de las actualizaciones más importantes de React 19. Permite a React trabajar en múltiples tareas simultáneamente, priorizando las interacciones del usuario y posponiendo las actualizaciones menos críticas. Esta función mejora la capacidad de respuesta al garantizar que la interfaz de usuario siga siendo interactiva, incluso durante cálculos pesados o la obtención de datos.

Beneficios clave:

- Experiencia de usuario fluida: prioriza los aspectos de alta prioridad tareas como el manejo de entrada.
- Programación mejorada: permite pausar, continuar o cancelar tareas de renderizado.
- Utilización eficiente de recursos: administra la carga de trabajo de forma más efectiva, garantizando que las actualizaciones críticas se procesen primero.

2. Componentes del servidor

React 19 mejora la compatibilidad con componentes de servidor, una función que permite a los desarrolladores renderizar componentes en el servidor y transmitirlos al cliente. Este enfoque reduce la cantidad de JavaScript.

se envían al navegador, lo que genera tiempos de carga más rápidos y un rendimiento mejorado, especialmente en dispositivos más lentos.

Características principales:

- Integración perfecta: los componentes del servidor funcionan junto con los componentes del cliente.
- Reducción de la sobrecarga del lado del cliente: descarga el trabajo pesado al servidor.
- Obtención de datos simplificada: los componentes del servidor pueden acceder directamente a los recursos del backend, lo que elimina la necesidad de llamadas API adicionales.

3. Procesamiento automático de lotes

React 19 introduce el procesamiento por lotes automático para actualizaciones de estado en operaciones asíncronas, como controladores de eventos, tiempos de espera o solicitudes de red. Anteriormente, los desarrolladores debían encapsular manualmente las actualizaciones en `ReactDOM.flushSync()` u otros métodos para procesarlas por lotes.

Ejemplo: :

función `App()`

```
{ const [count, setCount] = useState(0); const  
[texto, setText] = useState("");
```

```
const handleClick = () =>  
  { setCount((c) => c + 1);  
    setText('¡Actualizado!'); };
```

```
regresar <button onClick={handleClick}>Haz clic en mí</button>; }
```

Tanto `setCount` como `setText` ahora se procesan en lotes de forma automática, lo que da como resultado una única nueva representación en lugar de dos.

4. Mejoras de React DevTools

React DevTools en React 19 viene con varias actualizaciones para ayudar
Los desarrolladores depuran las aplicaciones de forma más eficaz. Estos
Las mejoras proporcionan información más profunda sobre el comportamiento de los componentes y
cuellos de botella en el rendimiento.

Nuevas funciones:

- Perfilador mejorado: visualizaciones detalladas de la representación
procesos.
- Razones para la representación de componentes: comprenda por qué
Componentes específicos re-renderizados.
- Depuración de ganchos: información más clara sobre el comportamiento
de ganchos personalizados.

5. Manejo de errores mejorado

El manejo de errores se ha mejorado significativamente en React 19, lo que lo hace
es más fácil detectar y gestionar errores en todos los componentes.

Mejoras clave :

- Límites de error para código asíncrono: React now
Admite la captura de errores en operaciones asíncronas como efectos y
controladores de eventos.
- Seguimiento de pila detallado: mensajes de error mejorados con
Mejor información de depuración.

6. Modo estricto mejorado

El modo estricto de React 19 se ha actualizado para ayudar a los desarrolladores a identificar
Problemas potenciales en su código durante el desarrollo. Introduce
controles adicionales para detectar métodos de ciclo de vida inseguros, efectos secundarios,
y otros posibles peligros.

7. API modernizadas

React 19 moderniza las API existentes para alinearse con JavaScript
avances y optimizar los flujos de trabajo de los desarrolladores.

Actualizaciones clave:

- **React.lazy con Suspense para componentes de servidor:**
Optimiza la división de código y la representación del lado del servidor.
- **Soporte mejorado para módulos ES:** mejora compatibilidad con los empaquetadores de JavaScript modernos.

8. Compatibilidad con versiones posteriores

React 19 se diseñó con la compatibilidad hacia adelante en mente, lo que permite a los desarrolladores adoptar nuevas funciones gradualmente sin modificar el código base existente. Esto garantiza una transición fluida para los equipos que actualizan desde versiones anteriores.

Las actualizaciones de React 19 representan un gran avance, centrándose en el rendimiento, la experiencia del desarrollador y la escalabilidad. Características como el renderizado concurrente, los componentes de servidor y el procesamiento automático por lotes simplifican el proceso de creación de aplicaciones intuitivas y de alto rendimiento. Al adoptar React 19, los desarrolladores pueden aprovechar estas innovaciones para crear aplicaciones web modernas y responsivas.

• Mejoras en el rendimiento y la representación

El rendimiento es fundamental en React, y con React 19, se han introducido mejoras significativas para optimizar el renderizado y la gestión de recursos. Estas actualizaciones solucionan problemas comunes en el desarrollo web moderno, garantizando que las aplicaciones se mantengan rápidas y con capacidad de respuesta, incluso con cargas de trabajo elevadas.

1. Renderizado concurrente: un cambio radical en el rendimiento

El renderizado concurrente es la principal mejora de rendimiento de React 19. Permite a React dividir las tareas de renderizado en unidades más pequeñas y procesarlas sin bloqueos. Esto garantiza que las actualizaciones de alta prioridad, como las interacciones del usuario, se gestionen con prontitud.

Características principales:

- Segmentación de tiempo: divide la renderización en fragmentos que pueden pausarse y reanudarse.
- Priorización: se centra primero en las tareas críticas, como la entrada del usuario, y pospone las tareas no esenciales.
- Animaciones y transiciones fluidas: evita los bloqueos garantizando que la representación no interfiera con las animaciones.

Ejemplo: :

Imagine un escenario en el que una aplicación obtiene datos mientras el usuario escribe en un cuadro de búsqueda. Con la renderización concurrente, la interacción al escribir se mantiene fluida, incluso mientras la aplicación procesa la obtención de datos en segundo plano.

2. Componentes del servidor: representación cliente-servidor optimizada

Los componentes de servidor de React 19 mejoran el rendimiento al transferir las tareas de renderizado pesadas al servidor. Esto reduce la cantidad de JavaScript enviado al cliente, lo que permite tiempos de carga iniciales más rápidos y un mejor rendimiento en dispositivos de bajo consumo.

Beneficios: :

- Carga útil de JavaScript reducida: solo se envía al cliente el JavaScript esencial.
- Renderizado en streaming: los componentes se transmiten en tiempo real desde el servidor, permitiendo que la interfaz de usuario se cargue progresivamente.
- SEO mejorado: el contenido renderizado por el servidor es más fácilmente indexado por los motores de búsqueda.

Ejemplo: :

Imagine una página de listado de productos. Con los Componentes de Servidor, el servidor puede pre-renderizar los detalles del producto, enviando componentes ligeros al cliente y reduciendo así el tiempo de carga.

3. Procesamiento automático por lotes para actualizaciones de estado

React 19 introduce el procesamiento automático por lotes de actualizaciones de estado, lo que reduce la cantidad de repeticiones de renderizados y mejora la eficiencia.

Cómo funciona: :

Anteriormente, las actualizaciones de eventos asincrónicos (p. ej., solicitudes de red) activaban múltiples re-renderizados. Con el procesamiento por lotes automático, React agrupa estas actualizaciones en un único ciclo de re-renderizado.

Ejemplo: :

```
función App()
{ const [count, setCount] = useState(0); const
  [texto, setText] = useState("");

  const handleClick = () =>
    { setCount((c) => c + 1);
      setText('Actualizado'); };

  regresar <button onClick={handleClick}>Haz clic en mí</button>; }
```

Ambas actualizaciones de estado (setCount y setText) se realizan por lotes, lo que da como resultado una sola representación en lugar de dos.

4. Suspensión mejorada para la obtención de datos

React 19 se basa en la API Suspense, lo que mejora el modo en que las aplicaciones manejan la obtención y representación asincrónica de datos.

Mejoras clave: :

- Representación de respaldo: muestra una IU de respaldo (por ejemplo, una cargador) mientras se obtienen los datos.
- Obtención paralela: obtiene datos para varios componentes simultáneamente.

Ejemplo:

```
const PerfilUsuario = React.lazy(() => import('./PerfilUsuario'));

función App() {
```

```
return  
( <Suspense fallback={<div>Cargando...</div>}>  
  <PerfilDeUsuario /  
> </Suspense> ); }
```

5. Gestión de memoria y recursos

React 19 introduce optimizaciones para el uso de memoria y recursos, lo que garantiza un rendimiento más fluido en dispositivos con recursos limitados.

Mejoras: :

- Eficiencia de recolección de basura: React libera automáticamente memoria para los componentes no utilizados.
- Reconciliación eficiente: las actualizaciones están optimizadas para minimizar las manipulaciones innecesarias del DOM.

6. React DevTools: creación de perfiles de rendimiento

El perfilador actualizado de React DevTools proporciona información detallada sobre los cuellos de botella de rendimiento. Los desarrolladores ahora pueden identificar y optimizar renderizados ineficientes con mayor eficacia.

Características:

- Render Flamegraph: Visualiza el tiempo empleado
- Componentes de renderizado.
- Seguimiento de interacciones: comprenda cómo las interacciones del usuario afectan el rendimiento.

7. Carga diferida para tiempos de carga mejorados

La carga diferida se mejora aún más en React 19, lo que permite a los desarrolladores dividir su código en paquetes más pequeños y cargar solo los componentes necesarios para una vista específica.

Ejemplo:

```
constante LazyComponent = React.lazy(() => import('./LazyComponent'));  
  
función App() { return  
  
  ( <Suspense fallback={<div>Cargando...</div>}>  
    <LazyComponent /> </  
    Suspense> ); }  
  


---


```

8. Priorización de eventos

React 19 garantiza que eventos como escribir y hacer clic se prioricen sobre otras tareas, lo que mejora la capacidad de respuesta. Esto es especialmente beneficioso para aplicaciones complejas con un procesamiento intensivo de datos.

Las mejoras de rendimiento y renderizado de React 19 suponen un avance significativo, abordando los desafíos críticos que enfrentan las aplicaciones web modernas. Características como el renderizado concurrente, los componentes de servidor, el procesamiento por lotes automático y la función Suspense mejorada permiten a los desarrolladores crear aplicaciones más rápidas y con mayor capacidad de respuesta. Al adoptar estas actualizaciones, los desarrolladores pueden crear experiencias de usuario fluidas y optimizar el uso de recursos.

• Manejo de componentes del servidor

Los componentes de servidor React (RSC) son una característica poderosa que permite que los componentes se representen en el servidor y se transmitan al cliente.

Este enfoque mejora el rendimiento al reducir la cantidad de JavaScript enviado al navegador y permite tiempos de carga de página más rápidos, especialmente para aplicaciones complejas. A continuación, se muestra una guía paso a paso para comprender e implementar componentes de servidor en React 19.

Paso 1 **Comprender** los conceptos básicos de los componentes del servidor

¿Qué son los componentes del servidor?

Los componentes de servidor son un nuevo tipo de componente de React que se renderizan en el servidor en lugar del cliente. A diferencia de los componentes tradicionales, estos:

- No incluya JavaScript para el navegador.
- Puede acceder directamente a los recursos del servidor como bases de datos o API.
- Trabaje sin problemas con los componentes del cliente para crear aplicaciones React de pila completa.

Beneficios de los componentes de servidor: :

1. Tamaño reducido del paquete de JavaScript: solo lo necesario
Se envía JavaScript al cliente.
2. Rendimiento mejorado: los componentes se procesan más rápido en el servidor.
Acceso directo a los datos: no se necesitan API adicionales para 3.
obtener datos.

Paso 2. Configurar un proyecto React con componentes de servidor

Para utilizar componentes de servidor, necesita un proyecto React configurado con un marco de representación del lado del servidor como Next.js.

Paso 2.1. Instalar Next.js .

```
npx crear-siguiente-aplicación@última-aplicación mi-aplicación
cd mi-aplicación
```

Paso 2.2. Habilitar los componentes de servidor

Next.js admite automáticamente los componentes de servidor. En el archivo next.config.js de tu proyecto, asegúrate de que las funciones experimentales de React estén habilitadas:

```
módulo.exports =
{ experimental:
  { serverComponents: true, }, };
```

Paso 3.1 Crear un componente de servidor

Paso 3.1.1 Definir un componente de servidor

Los componentes del servidor se crean como los componentes React normales, pero con .server.js como extensión de archivo.

Ejemplo: UserProfile.server.js

exportar función predeterminada UserProfile()

```
{ const user = { name: "Alice", age: 30 }; // Datos simulados del lado del servidor
```

```
  return
  ( <div>
    <h1>Perfil de usuario</h1>
    <p>Nombre: {user.name}</p>
    <p>Edad: {user.age}</p> </
  div> ); }
```

Paso 3.2 Utilizar el componente de servidor en un componente de cliente

Puede utilizar componentes de servidor en componentes de cliente sin problemas.

Ejemplo: App.js

```
importar dinámico desde 'next/dynamic';
```

```
// Importar dinámicamente el componente del servidor
```

```
const UserProfile = dynamic(() => import('./UserProfile.server'));
```

```
exportar función predeterminada
```

```
App()
{ return
  ( <div> <h1>Bienvenido</h1>
    <PerfilUsuario />
  </div> )
```

```
); }
```

Paso 4: Obtener datos en un componente del servidor

Los componentes del servidor pueden obtener datos directamente de las API o bases de datos, lo que elimina la necesidad de una capa de obtención de datos independiente.

Ejemplo: obtención de datos en un componente del servidor

```
importar fetch desde 'node-fetch';

exportar función asíncrona predeterminada UserProfile()
  { const respuesta = await
fetch('https://jsonplaceholder.typicode.com/users/1'); const usuario = await
  response.json();

  return
  ( <div>
    <h1>Perfil de usuario</h1>
    <p>Nombre: {user.name}</p>
    <p>Correo electrónico: {user.email}</p> </div> ); }
```

Paso 5: Combinar los componentes del servidor y del cliente

Los componentes de servidor a menudo se utilizan junto con los componentes de cliente para gestionar la interactividad.

Ejemplo: Combinación de componentes

```
// Counter.client.js (Componente cliente) importar
{ useState } desde 'react';

exportar función predeterminada Counter()
  { const [count, setCount] = useState(0);

  devolver (
```

```

    <div>
      <p>Conteo: {count}</p>
      <button      onClick={()      =>      setCount(contar      +
1))>Incremento</button> </div> ); }

```

```

// UserProfile.server.js (Componente del servidor) importa
Counter desde './Counter.client';

```

```

exportar función predeterminada UserProfile() { const
  usuario = { nombre: "Alice", edad: 30 };

  return
    ( <div>
      <h1>Perfil de usuario</h1>
      <p>Nombre: {user.name}</p> <p>Edad:
      {user.age}</p> <Counter /> </
      div> ); }

```

Paso 6 Optimizar el rendimiento de los componentes del servidor

Utilice la transmisión para una carga más rápida

Los componentes de React Server admiten la transmisión, lo que permite que partes de la interfaz de usuario se carguen progresivamente.

```

exportar función asíncrona predeterminada BlogPost()
  { const post = await
fetch('https://jsonplaceholder.typicode.com/posts/1').then(
  (res) => res.json() );

  devolver
    ( <div>

```

```
<h1>{título de la publicación}</h1> <p>{cuerpo de la publicación}</p> </div> ); }
```

Paso 1 Mejores prácticas para los componentes del servidor

1. Minimizar el JavaScript del lado del cliente: mantener la lógica interactiva en los componentes del cliente.
2. Utilice los recursos del servidor de forma eficiente: obtenga datos o realice cálculos pesados en los componentes del servidor.
3. Mantenga una separación clara: etiquete claramente los componentes como `.server.js` o `.client.js`.

• Mejoras en las herramientas para desarrolladores de React

React Developer Tools (DevTools) es una extensión del navegador que proporciona a los desarrolladores herramientas para depurar y optimizar aplicaciones React. Con React 19, DevTools introduce varias funciones nuevas que hacen que la depuración sea más eficiente y precisa.

Paso 1 Instalar las herramientas para desarrolladores de React

Instalación de la extensión

1. Abra Chrome Web Store o el navegador de su dispositivo. tienda de extensiones.
2. Busque herramientas para desarrolladores de React. Instale la extensión y actualice su navegador.
- 3.

Paso 2 Explora la interfaz actualizada

Nuevas características en React DevTools:

1. Mejoras del generador de perfiles:
 - visualice los tiempos de renderizado y los cuellos de botella en el rendimiento.
 - Analice patrones de renderizado con flamegraphs y gráficos.

2. Depuración de ganchos:

- Ver el estado y los valores de los ganchos.
- Realizar un seguimiento del comportamiento de los ganchos personalizados.

3. Razones para la representación de componentes:

- Identificar por qué se volvió a renderizar un componente.
- Depure representaciones innecesarias para optimizar el rendimiento.

Paso 3: Depurar componentes usando DevTools

Inspeccionar el árbol de componentes

1. Abra el panel DevTools en su navegador (por ejemplo, F12 o haga clic derecho → Inspeccionar).
2. Vaya a la pestaña React.
3. Explore el árbol de componentes para inspeccionar los accesorios, estado y ganchos.

Paso 4: Analizar el rendimiento con el generador de perfiles

Comenzar a perfilar:

1. Vaya a la pestaña Profiler en DevTools.
2. Haga clic en Grabar para comenzar a monitorear su aplicación.
rendimiento.
3. Interactúe con su aplicación para capturar el rendimiento
datos.

Analizar resultados:

- Flamegraph: muestra el tiempo empleado en renderizar cada componente.
- Vista clasificada: enumera los componentes por tiempo de renderizado.

Paso 5: Depurar ganchos

React 19 DevTools te permite inspeccionar los valores de los ganchos directamente:

1. Seleccione un componente en la pestaña React.
2. Expanda la sección Ganchos para ver el estado y efectos.

Paso 6 Optimizar la renderización con Render Reasons

1. Habilite las actualizaciones destacadas en la configuración de DevTools.
2. Interactúa con tu aplicación para ver qué componentes están volver a renderizar.
3. Utilice la función ¿Por qué se renderizó esto? para identificar nuevas renderizaciones innecesarias.

Paso 7 Funciones avanzadas

Actualizaciones de registros

Realice un seguimiento de las actualizaciones de un componente haciendo clic derecho sobre él en DevTools y seleccionando Registrar actualizaciones.

Componentes del servidor de depuración

Con React 19, DevTools también admite la depuración de componentes del servidor, lo que permite una integración perfecta de los flujos de trabajo cliente-servidor.

Los componentes de React Server y las herramientas de desarrollo actualizadas de React 19 ofrecen potentes funciones para crear y depurar aplicaciones web modernas. Siguiendo estas guías paso a paso, los desarrolladores pueden aprovechar al máximo estas funciones para optimizar el rendimiento y mejorar la experiencia de desarrollo.

Capítulo 5.

Componentes de React en profundidad

- Comprensión del ciclo de vida de los componentes

En React, el ciclo de vida del componente se refiere a la secuencia de eventos que ocurren durante la existencia de un componente en una aplicación React.

Desde que se crea un componente y se añade al DOM hasta que se elimina, React proporciona ganchos y métodos para gestionar las diferentes etapas de su ciclo de vida. Comprender el ciclo de vida del componente es esencial para optimizar el rendimiento, gestionar los efectos secundarios y garantizar una experiencia de usuario fluida.

1. ¿Qué es el ciclo de vida del componente? ?

El ciclo de vida del componente consta de tres fases principales:

1. Montaje: La fase en la que se crea un componente y añadido al DOM.
2. Actualización: La fase en la que un componente se vuelve a renderizar debido a cambios en los accesorios o el estado.
3. Desmontaje: La fase en la que se elimina un componente del DOM.

React proporciona métodos de ciclo de vida (para componentes de clase) y ganchos (para componentes funcionales) para interactuar con estas fases.

2. Fases del ciclo de vida en detalle

2.1 Fase de montaje

La fase de montaje es la etapa inicial donde se crea el componente y se inserta en el DOM. Esta fase incluye los siguientes métodos/ganchos del ciclo de vida:

Componentes de clase

1. constructor()
 - Se utiliza para inicializar el estado del componente y vincularlo.
- métodos.
- Se llama una vez antes de montar el componente.

Ejemplo:

```
clase MyComponent extiende React.Component {  
  constructor(props) {  
    super(props);  
    este.estado = { count: 0 };  
  }  
}
```

2. estática getDerivedStateFromProps()
 - Permite que el estado se actualice en función de los accesorios antes
- representación.
- Se utiliza rara vez y debe evitarse a menos que sea necesario.

Ejemplo:

```
estática getDerivedStateFromProps(props, estado) {  
  si (props.initialCount !== estado.count) {  
    retorna { count: props.initialCount };  
  }  
  devuelve nulo;  
}
```

3. prestar()
 - Método requerido en los componentes de la clase.
 - Describe la salida de UI del componente.

Ejemplo:

```
prestar() {  
  devolver <h1>¡Hola, React!</h1>;  
}
```

4. componentDidMount()

- Se llama después de que el componente se haya renderizado y agregado al DOM.
- Ideal para obtener datos o suscribirse a eventos.

Ejemplo:

```
componentDidMount()  
  { console.log('El componente se ha montado.')} }
```

Componentes funcionales

Los componentes funcionales utilizan el gancho `useEffect` para gestionar los efectos secundarios durante la fase de montaje.

Ejemplo:

```
importar React, { useEffect } de 'react';  
  
function MyComponent()  
  { useEffect(() =>  
    { console.log('El componente se ha montado.')}  
    return () =>  
      { console.log('Limpieza al desmontar.')} },  
  
    []); // La matriz de dependencias vacía garantiza que se ejecute solo una vez.  
  
  regresar <h1>¡Hola, React!</h1>; }
```

2.2 Fase de actualización

La fase de actualización ocurre cuando un componente se vuelve a renderizar debido a cambios en las propiedades, el estado o el contexto. Esta fase incluye los siguientes métodos/ganchos:

Componentes de clase

1. `estática getDerivedStateFromProps()`

- Se llama durante las fases de montaje y actualización.
- Actualiza el estado en función de los accesorios.

2. shouldComponentUpdate()

- Se utiliza para optimizar el rendimiento controlando si

El componente debería volver a renderizarse.

- Devuelve verdadero (predeterminado) o falso.

Ejemplo:

```
shouldComponentUpdate(nextProps, nextState) {  
  devuelve nextState.count !== this.state.count;  
}
```

3. prestar()

- Se llama durante cada actualización para volver a renderizar el componente.

4. obtenerInstantáneaAntesDeActualizar()

- Captura información (por ejemplo, posición de desplazamiento) antes de la El DOM se actualiza.

- El valor regresó es aprobado a componentDidUpdate().

Ejemplo:

```
obtenerInstantáneaAntesDeActualizar(prevProps, prevState) {  
  devolver documento.querySelector('.scroll-container').scrollTop;  
}
```

5. componentDidUpdate()

- Se invoca después de que el componente se haya vuelto a renderizar.
- Útil para interactuar con el DOM o crear API.

llamadas basadas en el estado/props actualizados.

Ejemplo:

```
componentDidUpdate(prevProps, prevState, instantánea) {  
  console.log('Componente actualizado. Posición de desplazamiento:', instantánea);  
}
```

Componentes funcionales

Los componentes funcionales manejan las actualizaciones mediante el gancho `useEffect`. La matriz de dependencia del gancho determina cuándo se ejecuta el efecto.

Ejemplo:

```
importar React, { useState, useEffect } de 'react';

función Contador({ initialCount }) { const
  [count, setCount] = useState(initialCount);

  useEffect(() =>
    { console.log(`Count actualizado: ${count}`); },
    [count]); // Se ejecuta solo cuando cambia `count`.

  devolver <botón al hacer clic={() => setCount(count +
1)}>Incrementar</button>; }
```

2.3 Fase de desmontaje

La fase de desmontaje ocurre cuando se elimina un componente del DOM. Esta fase incluye los siguientes métodos/ganchos:

Componentes de clase

1. `componentWillUnmount()`
 - Se llama inmediatamente antes de que se ejecute el componente. destruido.
 - Se utiliza para tareas de limpieza como cancelar la suscripción a eventos. o borrar temporizadores.

Ejemplo:

```
componentWillUnmount()
  { console.log('El componente se desmontará.')} }
```

Componentes funcionales

Los componentes funcionales utilizan la función de limpieza dentro de `useEffect` para desmontar la lógica.

Ejemplo:

```
useEffect(() =>
  { const timer = setInterval(() =>
    { console.log('Ejecutando...'); },
    1000);

  return () =>
    { clearInterval(timer); // Limpieza al desmontar.
      console.log('Componente desmontado.');
```

3. Combinación de métodos de ciclo de vida

En aplicaciones del mundo real, se pueden usar múltiples métodos o ganchos de ciclo de vida juntos para administrar el comportamiento del componente de manera eficiente.

Ejemplo: obtención de datos y limpieza

```
class DataFetcher extends React.Component { estado =
  { datos: null };

  componentDidMount()
    { fetch('https://api.example.com/data') .then((res)
      => res.json()) .then((data) =>
        this.setState({ data }));
    }

  componentWillUnmount()
    { console.log('Componente desmontado. Limpiar recursos aquí.');
```



```
  render()
    { return <div>{this.state.data ? this.state.data : 'Cargando...'}</div>; }
```

```
}}
```

Con componentes funcionales:

importar React, { useState, useEffect } de 'react'; función

```
DataFetcher() { const [data,  
  setData] = useState(null);
```

```
  useEffect(() => { const  
    fetchData = async () => { const respuesta  
      = await fetch('https://api.example.com/data'); const resultado = await  
        respuesta.json(); setData(resultado); };
```

```
  obtener datos();
```

```
  return () =>  
    { console.log('Limpieza al desmontar. '); }, []);
```

```
  devolver <div>{datos ? datos : 'Cargando...'}</div>; }
```

4. Mejores prácticas del ciclo de vida

1. Evite el uso excesivo de los métodos de ciclo de vida: utilice únicamente métodos/ganchos cuando sea necesario para evitar complejidad innecesaria.

2. Optimice el rendimiento: utilice `shouldComponentUpdate()` o `React.memo` para evitar re-renderizaciones innecesarias.

3. Limpiar recursos: limpie siempre las suscripciones, los temporizadores y los oyentes en la fase de desmontaje para evitar pérdidas de memoria.

4. Minimizar los efectos secundarios en el renderizado: evite realizar efectos secundarios (por ejemplo, llamadas API) dentro del método `render()`.

5. Transición de componentes de clase a componentes funcionales

Con la introducción de React Hooks, los componentes funcionales pueden manejar todas las fases del ciclo de vida, reduciendo la necesidad de componentes de clase. Los ganchos como `useEffect` unifican el manejo de los efectos secundarios en diferentes fases, simplificando la base del código.

El ciclo de vida de los componentes es un concepto fundamental en React, que permite a los desarrolladores gestionar el comportamiento de un componente y optimizar el rendimiento de la aplicación. Al comprender las fases del ciclo de vida (montaje, actualización y desmontaje), se pueden gestionar eficazmente los efectos secundarios, obtener datos y mantener una estructura de aplicación limpia. Ya sea que uses componentes de clase o componentes funcionales con ganchos, dominar el ciclo de vida es esencial para crear aplicaciones React sólidas y escalables.

• Escritura de componentes reutilizables y modulares

Los componentes reutilizables y modulares son la base de la arquitectura basada en componentes de React. Permiten a los desarrolladores crear aplicaciones escalables al promover la reutilización, la mantenibilidad y la consistencia del código. Esta guía ofrece un enfoque paso a paso para escribir componentes reutilizables y modulares en React.

Paso 1 **Comprender la reutilización y la modularidad**

¿Qué son los componentes reutilizables? ?

Los componentes reutilizables están diseñados para usarse en diferentes partes de una aplicación con mínimas modificaciones. Son genéricos y adaptables, lo que los hace ideales para gestionar patrones de interfaz de usuario comunes, como botones, campos de entrada y tarjetas.

¿Qué son los componentes modulares? ?

Los componentes modulares son autocontenidos, lo que significa que encapsulan su lógica, estilos y comportamiento. La modularidad garantiza la independencia de los componentes, lo que facilita el mantenimiento y la depuración del código.

Beneficios de los componentes reutilizables y modulares:

:

Consistencia: garantiza una apariencia uniforme en toda la aplicación.

2. Reducción de la duplicación de código: elimina el código repetitivo, lo que reduce la sobrecarga de mantenimiento.

3. Escalabilidad mejorada: facilita la expansión y modificar la aplicación.

Paso 2 Planifique la estructura de sus componentes

Antes de escribir un componente reutilizable, determine su propósito y alcance:

1. Defina su funcionalidad: ¿Qué se supone que debe hacer el componente?

Identificar oportunidades de reutilización: ¿Puede este componente 2?

¿Se puede utilizar en varios lugares?

Lista de propiedades: determina los datos o la configuración que necesitará el componente 3.

Ejemplo: Planificación de un componente de botón

1. Funcionalidad: Un botón en el que se puede hacer clic para realizar una acción. una acción.

2. Oportunidades de reutilización: se utilizan para diferentes acciones (por ejemplo, enviar formularios, abrir modales).

3. Propiedades: etiqueta, onClick, tipo, deshabilitado.

Paso 3 Escribe un componente básico

Comience escribiendo un componente funcional básico con un comportamiento predeterminado. Por ejemplo, vamos a crear un componente Botón.

Ejemplo: Componente de botón básico

```
importar React desde 'react';
```

```
función Botón({ etiqueta, onClick }) { return  
  <botón onClick={onClick}>{etiqueta}</botón>; }
```



```
}
```

Botón de exportación predeterminado;

Paso 4: Agregar accesorios para personalizar

Las propiedades permiten que los componentes sean flexibles y adaptables. Añada propiedades para gestionar variaciones como estilos, comportamientos y características adicionales.

Mejorar el componente Botón

```
importar React desde 'react';
```

```
función Botón({ etiqueta, onClick, tipo = 'botón', deshabilitado = falso }) {  
  return  
    ( <button type={tipo} deshabilitado={deshabilitado} onClick={onClick}> {etiqueta}  
      </  
      button> ); }  
}
```

Botón de exportación predeterminado;

Aquí:

- El tipo predeterminado es 'botón' pero se puede personalizar.
- Deshabilitado agrega funcionalidad para deshabilitar el botón cuando necesario.

Paso 5: Dar estilo al componente

Encapsule los estilos dentro del componente para mantener la modularidad. Puede usar CSS, módulos CSS o bibliotecas CSS-in-JS como los componentes con estilo.

Uso de módulos CSS

1. Cree un archivo Button.module.css:
-

```
.botón {
```

```
relleno: 10px 20px;  
borde: ninguno;  
radio del borde: 5px;  
color de fondo: #007bff; color:  
blanco; cursor:  
puntero; }
```

```
.button:disabled  
{ color de fondo: #cccccc; cursor:  
no permitido; }
```

2. Importar y aplicar los estilos en el componente Botón:

```
importar React desde 'react';  
importar estilos desde './Button.module.css';  
  
función Botón({ etiqueta, onClick, tipo = 'botón', deshabilitado = falso }) {  
  devolver  
    ( <button  
      className={styles.button}  
      tipo={tipo}  
      deshabilitado={deshabilitado}  
      onClick={onClick}  
    >  
      {etiqueta}  
    </botón> ); }
```

Botón de exportación predeterminado;

Paso 6: Gestionar variantes y temas

Para lograr la máxima reutilización, permita que los componentes admitan múltiples estilos o temas usando accesorios como variante o tema.

Agregar variantes al botón

```
importar React desde 'react';
importar estilos desde './Button.module.css';

función Botón({ etiqueta, onClick, tipo = 'botón', deshabilitado = falso, variante = 'primario' })
{ const buttonClass = `
  {styles.button} ${styles[variante]}`;

  devolver
  ( <button
    className={buttonClass}
    tipo={tipo}
    deshabilitado={deshabilitado}
    onClick={onClick}
    >
      {etiqueta}
    </botón> ); }
```

Botón de exportación predeterminado;

Actualización del módulo CSS para variantes

```
.button
{ relleno: 10px 20px;
  borde: ninguno;
  radio del borde: 5px;
  cursor: puntero; }

.primary
{ color de fondo: #007bff; color:
  blanco; }

.secundario
{ color de fondo: #6c757d;
```

```
color: blanco; }
```

```
.danger
```

```
{ color de fondo: #dc3545; color:  
  blanco; }
```

```
.button:disabled { color
```

```
  de fondo: #cccccc; cursor: no  
  permitido; }
```

Paso **P**ruebe el componente

Pruebe siempre los componentes reutilizables con distintos accesorios y casos extremos para asegurarse de que se comporten como se espera.

Prueba del componente del botón

```
importar React desde 'react';
```

```
importar Button desde './Button';
```

```
función App() { return
```

```
  ( <div>
```

```
    <Button label="Botón principal" variante="primario" onClick={() => alert('Botón principal  
    hecho clic')} />
```

```
    <Button label="Botón secundario" variante="secundario" onClick= {} => alert('Botón  
    secundario hecho clic')} />
```

```
    <Button label="Botón deshabilitado" deshabilitado /> </  
    div> ); }
```

```
exportar aplicación predeterminada;
```

Paso **E**scribir componentes de utilidad

Los componentes reutilizables a menudo incluyen funciones de utilidad, como íconos o indicadores de carga.

Agregar un icono al botón

```
función Botón({ etiqueta, onClick, tipo = 'botón', deshabilitado = falso, variante = 'principal',  
  icono }) { const buttonClass = `${  
  {styles.button} ${styles[variant]}`;  
  
  devolver  
  ( <button  
    className={buttonClass} tipo={tipo}  
  
    deshabilitado={deshabilitado}  
    onClick={onClick}  
  >  
  
    {icono && <span className={styles.icon}>{icono}} {etiqueta} </button> ); }
```

Botón de exportación predeterminado;

Paso 9 Organizar los archivos de componentes

Mantenga sus componentes reutilizables organizados en una estructura de carpetas dedicada.

Ejemplo de estructura de carpetas

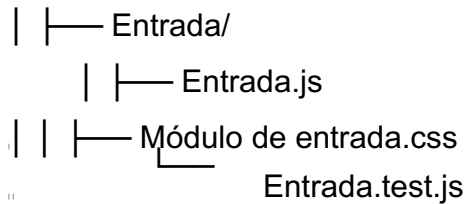
src/

|— componentes/ | —

Botón/ | — Botón.js | |

|— Botón.módulo.css | | |

└— Botón.test.js



Paso 10 Documentar el componente

Proporcione documentación clara para sus componentes reutilizables, incluido:

1. **Propósito:** Describir la función del componente.
2. **Propiedades:** enumera todas las propiedades admitidas con tipos y valores predeterminados valores.
3. **Ejemplos de uso:** demostración de casos de uso comunes. Incluir fragmentos de código

Ejemplo de documentación para el botón

- **Propósito:** Un componente de botón reutilizable con soporte para diferentes variantes y estados.
 - **Accesorios:**
 - **etiqueta (cadena):** el texto que se muestra en el botón.
- Requerido.
- **onClick (función):** Función a ejecutar al hacer clic.
- Opcional.
- **tipo (cadena):** Tipo de botón (botón, enviar). Predeterminado: botón.
 - **deshabilitado (booleano):** si el botón está deshabilitado. Predeterminado: falso.
 - **variante (cadena):** variante de estilo de botón (principal, secundaria). Predeterminado: primaria.

Paso 11 Utilice PropTypes para la validación

Utilice PropTypes para validar propiedades y detectar errores durante desarrollo.

Agregar PropTypes al botón

```
importar PropTypes desde 'prop-types';

Button.propTypes =
  { etiqueta: PropTypes.string.isRequired,
    onClick: PropTypes.func, tipo:
    PropTypes.oneOf(['botón', 'enviar', 'reiniciar']), deshabilitado:
    PropTypes.bool, variante:
    PropTypes.oneOf(['principal', 'secundario', 'peligro']), };

Button.defaultProps = { tipo:
  'botón',
  deshabilitado:
  falso, variante:
  'principal', };
```

Paso 12 Refactorizar para escalabilidad

A medida que su aplicación crece, refactorice los componentes reutilizables para adaptarlos a los nuevos requisitos sin afectar la funcionalidad existente. Utilice patrones de diseño como componentes de orden superior (HOC) o propiedades de renderizado si es necesario.

Los componentes reutilizables y modulares son esenciales para crear aplicaciones React escalables. Siguiendo esta guía paso a paso, podrás crear componentes adaptables, fáciles de mantener y consistentes. Comienza por definir el propósito del componente, añade flexibilidad con props, encapsula estilos y organiza tu código eficazmente. Al adoptar la reutilización y la modularidad, mejorarás significativamente la eficiencia de tu desarrollo y la calidad de tu código.

• Componentes con estado y sin estado

Los componentes de React se pueden clasificar en general en componentes con estado y sin estado según su capacidad para gestionar y mantener el estado. Comprender las diferencias, los casos de uso y las ventajas de...

Cada tipo es crucial para diseñar aplicaciones React escalables y fáciles de mantener. Esta guía completa explora estos componentes en profundidad.

1. Descripción general de componentes con estado y sin estado

~~Componentes con estado~~

Los componentes con estado mantienen y gestionan su propio estado interno. Pueden responder a las interacciones del usuario, llamadas a la API u otros eventos actualizando su estado, lo que activa una nueva renderización para actualizar la interfaz de usuario.

Características clave :

- Administrar su propio estado local utilizando `this.state` (clase componentes) o `useState/useReducer` (componentes funcionales).
 - Reaccionar a los cambios de datos dinámicos y volver a renderizar respectivamente.
 - A menudo se les denomina “inteligentes” o “contenedores”.
- componentes.

~~Componentes sin estado~~

Los componentes sin estado no gestionan el estado internamente. Se basan completamente en las propiedades transferidas desde los componentes principales para mostrar datos o gestionar el comportamiento. Estos componentes son simples, ligeros y se centran exclusivamente en la presentación.

Características clave :

- No mantener el estado.
 - Depende de los accesorios para recibir datos y activar acciones.
 - A menudo se les llama “tontos” o “presentativos”.
- componentes.

2. Ejemplos de componentes con estado y sin estado

Ejemplo de componente con estado

Un componente con estado que administra un contador:

```
importar React, { useState } de 'react';

función Contador()
  { const [count, setCount] = useState(0);

  return
    ( <div>
      <p>Recuento actual: {count}</p>
      <button      onClick={()      =>      setCount(contar      +
1))>Incremento</button> </
div> ); }

exportar contador predeterminado;
```

Aquí:

- useState inicializa el estado de conteo con un valor predeterminado de 0.
- La función setCount actualiza el estado, lo que activa una nueva representación.

Ejemplo de componente sin estado

Un componente sin estado que muestra un mensaje de saludo:

```
función Saludo({ nombre }) { devolver
  <h1>¡Hola, {nombre}!</h1>; }

exportar saludo predeterminado;
```

Aquí:

- El componente Saludo recibe el nombre de propiedad y lo muestra.
- No mantiene ni gestiona ningún estado interno.

4. Cuándo utilizar componentes con estado

Los componentes con estado son ideales para escenarios donde la interfaz de usuario depende de datos dinámicos o interacciones del usuario. Algunos casos de uso comunes incluyen:

1. Manejo de formularios:

gestión de entradas y validaciones de formularios.

```
función LoginForm() { const
  [nombre de usuario, establecerNombre de usuario] = useState("");
  const [contraseña, establecerContraseña] = useState("");

  devolver
  ( <form>
    <input
      type="text"
      value={nombreusuario}
      onChange={(e) => setUsername(e.target.value)} placeholder="Nombre
      de usuario" /> <input

      type="contraseña"
      value={contraseña}
      onChange={(e) => setPassword(e.target.value)}
      placeholder="Contraseña" /> </

    form> ); }
```

2. UI dinámica:

componentes que necesitan cambiar la apariencia o el comportamiento en función de las interacciones del usuario (por ejemplo, conmutadores, modales).

3. Obtención de datos de API:

obtención, visualización y actualización de datos de API externas.

5. Cuándo utilizar componentes sin estado

Los componentes sin estado son ideales para escenarios donde un componente solo necesita mostrar datos o gestionar funciones básicas. Algunos casos de uso comunes incluyen:

1. Componentes de presentación:

Componentes utilizados para la visualización de la interfaz de usuario, como encabezados, pies de página o tarjetas.

```
función Tarjeta({ título, descripción }) { return
  ( <div>

    <h2>{título}</h2>
    <p>{descripción}</p> </
    div> ); }
```

2. Componentes reutilizables:

Bloques de construcción como botones, entradas o íconos.

3. Componentes envolventes:

componentes que envuelven elementos secundarios con estilos o diseños.

6. Transición de componentes con estado a componentes sin estado

Con la introducción de React Hooks en React 16.8, los componentes funcionales ahora pueden administrar métodos de estado y ciclo de vida, combinando de manera efectiva las características de los componentes con estado y sin estado.

Ejemplo: Convertir un componente de clase en un componente funcional

Componente de clase:

```
clase Contador extiende React.Component { estado =
  { count: 0 };

  incremento = () =>
    { this.setState({ count: this.state.count + 1 }); };

  prestar() {
```

```
devolver
( <div>
  <p>Contar: {this.state.count}</p> <button
  onClick={this.increment}>Incremento</button> </div> ); } }
```

Componente funcional con ganchos:

```
función Contador() { const
  [count, setCount] = useState(0);

  devolver
  ( <div>
    <p>Conteo: {count}</p>
    <button      onClick={()      =>      setCount(contar      +
1)}>Incremento</button> </
    div> ); }
```

7. Ventajas y desventajas de los componentes con estado y sin estado

Componentes con estado

Ventajas:

- Habilite un comportamiento dinámico e interactivo.
- Gestione la lógica y el estado complejos internamente.

Contras:

- Menos reutilizable debido a la lógica específica del estado.
- Mayor complejidad en comparación con los sistemas sin estado

componentes.

Componentes sin estado

Ventajas:

- Simple y altamente reutilizable.
- Fácil de leer, probar y mantener.

Contras:

- Confíe completamente en los componentes principales para obtener datos y lógica.
- Funcionalidad limitada sin estado.

8. Mejores prácticas para el uso de componentes con estado y sin estado

1. Preocupaciones separadas:

utilice componentes con estado para administrar la lógica y los datos, y componentes sin estado para presentar los datos.

2. Minimizar estado:

Mantenga la gestión estatal mínima y centralizada siempre que sea posible.

3. Utilice componentes sin estado para la reutilización: cree componentes reutilizables manteniéndolos sin estado y pasando datos a través de accesorios.

4. Adopte componentes funcionales con ganchos: aproveche los ganchos de React para simplificar la gestión del estado en componentes funcionales.

Comprender las diferencias entre componentes con estado y sin estado es esencial para diseñar aplicaciones React eficientes.

Mientras que los componentes con estado son ideales para una funcionalidad dinámica e interactiva, los componentes sin estado se destacan por su simplicidad y reutilización.

Con la llegada de React Hooks, la línea entre ambos se ha difuminado, lo que permite a los desarrolladores disfrutar de las ventajas de ambos enfoques en componentes funcionales.

Mediante el uso estratégico de componentes con y sin estado, los desarrolladores pueden crear aplicaciones escalables, fáciles de mantener y de alto rendimiento.

- Mejores prácticas para el diseño de componentes

Diseñar componentes React es una habilidad crucial para crear aplicaciones eficientes, escalables y fáciles de mantener. Siguiendo las mejores prácticas,

Los desarrolladores pueden asegurarse de que sus componentes sean reutilizables, de alto rendimiento y estén alineados con la filosofía de React.

1. Mantenga los componentes pequeños y enfocados

Cada componente debe tener una única responsabilidad, lo que facilita su comprensión, prueba y reutilización.

Ejemplo: :

En lugar de crear un componente grande para un perfil de usuario, divídalo en componentes más pequeños:

```
función PerfilUsuario({ usuario }) { return
  ( <div>
    <Avatar
      src={usuario.avatar} /> <UserInfo
      nombre={usuario.nombre} bio={usuario.bio} /> </div> ); }
```

2. Utilice componentes funcionales

Prefiera los componentes funcionales a los de clase. Son más simples y, con Hooks, pueden gestionar métodos de estado y ciclo de vida.

3. Aprovechar los accesorios para la personalización

Pasar datos y configuración a través de accesorios para mantener los componentes flexibles y reutilizables.

Ejemplo: :

```
función Botón({ etiqueta, variante, onClick }) { return
  <botón className={`btn-${variante}`} onClick={onClick}> {etiqueta}</button>; }
```

4. Escribir componentes reutilizables

Diseñe componentes que se utilizarán en varias partes de su aplicación. Utilice nombres genéricos y evite codificar lógica específica.

5. Utilice PropTypes para la validación

Validar propiedades para detectar errores de forma temprana y garantizar que los componentes reciban los tipos de datos correctos.

6. Separar la lógica y la presentación

Mantenga la lógica empresarial separada de la interfaz de usuario mediante el uso de contenedores y componentes de presentación o ganchos personalizados.

7. Optimizar el rendimiento

Utilice técnicas como `React.memo` para evitar re-renderizaciones innecesarias y garantizar que sus componentes sean livianos.

8. Organizar archivos y carpetas

Adopte una estructura consistente para sus componentes, como agrupar los archivos relacionados en una sola carpeta.

Siguiendo estas prácticas recomendadas, podrá diseñar componentes React fáciles de mantener, escalables y usar. Céntrese en la reutilización, la simplicidad y el rendimiento para crear aplicaciones React robustas.

Capítulo 6.

Gestión del estado y del ciclo de vida

- Gestión del estado en React

La gestión de estados es fundamental para las capacidades dinámicas e interactivas de React. Permite que los componentes gestionen y respondan a la entrada del usuario, los datos de la API y otros eventos, garantizando así que la aplicación siga siendo responsiva y funcional.

1. ¿Qué es el estado en React? ?

El estado es un objeto de JavaScript que contiene datos dinámicos para un componente. A diferencia de las propiedades, que son inmutables y se transmiten desde los componentes principales, el estado es mutable y se controla dentro del componente. Cuando el estado cambia, React vuelve a renderizar automáticamente el componente para reflejar los datos actualizados.

Ejemplo de Estado:

```
importar React, { useState } de 'react';
```

```
función Contador()
```

```
{ const [count, setCount] = useState(0);
```

```
  devolver
```

```
  ( <div>
```

```
    <p>Conteo: {count}</p>
```

```
    <button      onClick={()      =>      setCount(contar      +
```

```
1))>Incremento</button> </
```

```
  div> ); }
```

2. Tipos de estado en React

2.1 Estado local

- Gestionado dentro de un único componente.
- Útil para administrar datos como entradas de formulario o alternar estados.

Ejemplo:

```
función Toggle() { const
  [isOn, setIsOn] = useState(false);

  devolver <button onClick={() => setIsOn(!isOn)}>{isOn ? 'ON' : 'OFF'} </button>; }
```

2.2 Estado global

- Compartido entre múltiples componentes.
- Administrado mediante Context API, Redux u otro estado bibliotecas de gestión.

Ejemplo (usando la API de contexto):

```
constante ThemeContext = React.createContext();

función App() { const
  [tema, setTheme] = useState('light');

  devolver
    ( <ThemeContext.Provider valor={{ tema, setTheme }}>
      <Barra de
        herramientas /> </
ThemeContext.Provider> ); }

función Barra de
  herramientas() { const { tema } = React.useContext(ThemeContext);
  return <div>Tema actual: {theme}</div>; }
```

```
}
```

2.3 Estado del servidor

- Representa datos obtenidos de un servidor.
- Se administra mediante bibliotecas como React Query o mediante la sincronización con API.

Ejemplo: :

función PerfilUsuario()

```
{ const [usuario, setUser] = useState(null);
```

```
useEffect(() =>
```

```
{ fetch('/api/
```

```
usuario') .then((res) =>
```

```
res.json()) .then((datos) =>
```

```
setUser(datos)); }, []);
```

```
devolver usuario ? <div>{user.name}</div> : <div>Cargando...</div>; }
```

2.4 Estado derivado

- Calculado a partir del estado o propiedades existentes.
- Evite la duplicación para evitar inconsistencias.

Ejemplo: :

función NombreCompleto({ Nombre, Apellido })

```
{ const NombreCompleto = `${Nombre} ${Apellido}`;
```

```
return <h1>{NombreCompleto}</
```

```
h1>; }
```

3. Técnicas para la gestión del Estado

3.1 Uso del gancho useState

El gancho useState es la forma más sencilla de administrar el estado local en componentes funcionales.

Ejemplo:

```
const [valor, valor_establecido] = useState(valor_inicial);
```

3.2 Uso del gancho useReducer

El gancho useReducer es ideal para gestionar lógica de estados compleja.

Ejemplo:

```
función reductor(estado, acción)
```

```
  { switch(acción.tipo) { caso  
    'incremento': return  
      { count: estado.count + 1 }; caso  
    'decremento': return  
      { count: estado.count - 1 };
```

```
    predeterminado: return estado;
```

```
  } }
```

```
función Contador() { const
```

```
  [estado, despacho] = useReducer(reductor, { count: 0 });
```

```
  devolver
```

```
  ( <div>
```

```
    <button onClick={() => despacho({ tipo: 'decremento' })}>- </button>
```

```
<span>{estado.count} <button onClick={()
```

```
  => despacho({ tipo: 'incremento' })}>+ </button> </div> ); }
```

3.3 Uso de la API de contexto

La API de contexto permite la gestión de estados globales sin bibliotecas de terceros.

Ejemplo:

```

constante AuthContext = React.createContext();

función App() {
  const [isAuthenticated, setIsAuthenticated] = useState(false);

  devolver (
    <AuthContext.Provider
      value={{
        isAuthenticated,
        establecerIsAuthenticated
      }}
    <Iniciar sesión />
    </AuthContext.Provider>
  );
}

función Iniciar sesión() {
  { const isAuthenticated, setIsAuthenticated } =
  React.useContext(AuthContext);

  devolver (
    <botón al hacer clic={() => setIsAuthenticated(!isAuthenticated)}>
      {isAuthenticated ? 'Cerrar sesión' : 'Iniciar sesión'}
    </botón>
  );
}

```

4. Mejores prácticas para la gestión estatal

1. Mantenga el estado mínimo: almacene solo lo necesario.
Evite duplicar el estado derivado.
2. Utilice una única fuente de verdad: asegúrese de que el estado sea centralizado y consistente.
3. Optimice el rendimiento: utilice `React.memo` y `useReducer` para componentes de rendimiento crítico.
4. Utilice bibliotecas externas con prudencia: utilice herramientas como `Redux` o `Consulta de React` para la gestión de estados complejos.

5. Separar la interfaz de usuario y la lógica: mantener la presentación
Componentes sin estado, que delegan la lógica a los componentes del contenedor.

• Uso de ganchos (useState, useEffect, useReducer)

Los ganchos son una característica poderosa en React que permite a los desarrolladores administrar el estado, los efectos secundarios y otras características de React en componentes funcionales.

1. useState: Gestión del estado local

El gancho useState se utiliza para gestionar el estado de los componentes funcionales. Permite definir variables de estado y actualizar sus valores.

Paso 1 Importar e inicializar useState

Importa useState desde React e inicializa una variable de estado con un valor predeterminado.

Ejemplo:

```
import React, { useState } from 'react';
```

```
función Counter() { const  
  [count, setCount] = useState(0); // count es la variable de estado, setCount la actualiza.
```

```
  devolver  
  ( <div>  
    <p>Conteo: {count}</p>  
    <button onClick={() => setCount(count +  
1)}>Incremento</button> </div> ); }
```

Paso 2 Actualizar el estado

Llama a la función de actualización de estado (setCount en el ejemplo) para modificar el estado. React renderiza automáticamente el componente para reflejar los cambios.

Paso 3 utilizar múltiples variables de estado

Puede utilizar múltiples llamadas useState para administrar diferentes partes del estado.

Ejemplo:

```
función Formulario()
  { const [nombre, establecerNombre] = useState(""); const
    [correo electrónico, establecerCorreo electrónico] = useState("");

    devolver

    ( <formulario> valor={nombre} onChange={(e) <input
      setName(e.target.value)} placeholder="Nombre" />
                                     valor={correo electrónico} onChange={(e) <input
      setEmail(e.target.value)} placeholder="Correo electrónico" /
      > </
form> ); }
```

2. useEffect: Manejo de efectos secundarios

El gancho useEffect se utiliza para administrar efectos secundarios como la obtención de datos, suscripciones o modificación manual del DOM.

Paso 1 Uso básico

Llame a useEffect con una función que realiza el efecto secundario.

Ejemplo: Registrar un mensaje cuando se procesa el componente:

```
importar React, { useEffect } de 'react'; función
```

```
Logger() { useEffect(()
  => {
```

```
console.log('¡Componente renderizado!'); });
```

```
return <p>Comprueba la consola.</p>; }
```

Paso 2 Agregar dependencias

Especifique las dependencias para controlar cuándo se ejecuta el efecto. Use una matriz vacía ([]) para ejecutar el efecto solo una vez.

Ejemplo: Obtención de datos en el montaje:

```
función DataFetcher()  
{ useEffect(() =>  
  { fetch('/api/  
    data') .then((res) =>  
      res.json()) .then((data) => console.log(data));  
  }, []); // Se ejecuta solo una vez en el montaje.  
  
  devolver <p>Obteniendo datos...</p>; }
```

Paso 3 Efectos de limpieza

Devuelve una función de limpieza de useEffect para manejar el desmontaje o el reinicio.

Ejemplo: Limpieza de un temporizador:

```
función Timer()  
{ useEffect(() =>  
  { const timer = setInterval(() =>  
    { console.log('Temporizador en  
      ejecución...'); }, 1000);  
  
    return () => clearInterval(timer); // Limpieza al desmontar. }, []);  
  
  return <p>El temporizador está en marcha. Revisa la consola.</p>; }
```

```
}
```

3. useReducer: Gestión de estados complejos

El gancho useReducer es una alternativa a useState para gestionar lógica de estados compleja. Funciona de forma similar a Redux, pero está integrado en React.

Paso 1 Definir una función reductora

La función reductora toma el estado actual y una acción, devolviendo un nuevo estado.

Ejemplo:

```
función reducer(estado, acción)
{ switch(acción.tipo) { caso
  'incremento': return
    { count: estado.count + 1 }; caso
  'decremento': return
    { count: estado.count - 1 };

  predeterminado: throw new Error('Tipo de acción desconocido');
}}
```

Paso 2 Inicializar useReducer

Pase el reductor y un estado inicial a useReducer.

Ejemplo:

```
función Contador()
{ const [estado, despacho] = useReducer(reductor, { count: 0 });

  devolver
  ( <div>
    <p>Conteo: {estado.conteo}</p>
```



```
    <button onClick={() => dispatch({ tipo: 'incremento'
  })}>Incrementar</button>
    <button onClick={() => dispatch({ tipo: 'decremento'
  })}>Decrementar</button> </
  div> ); }
```

Paso 3 Manejar la lógica de estados complejos

Utilice useReducer para escenarios donde múltiples variables de estado dependen unas de otras.

4. Combinando ganchos

Los ganchos se pueden combinar para gestionar tanto el estado como los efectos secundarios de manera eficiente.

Ejemplo: Obtener datos con useEffect y almacenarlos en estado con useState:

```
función DataFetcher() { const
  [datos, setData] = useState(null);

  useEffect(() =>
    { fetch('/api/
      data') .then((res) =>
        res.json()) .then((data) =>
          setData(data)); }, []);

  ¿devolver datos? <p>{data.title}</p> : <p>Cargando...</p>; }
```

Los Hooks de React (useState, useEffect, useReducer) ofrecen una forma eficaz de gestionar el estado y los efectos secundarios en componentes funcionales. Eliminan la necesidad de componentes de clase y ofrecen una API más concisa y flexible. Al comprender cómo usar estos hooks eficazmente, podrá crear aplicaciones React dinámicas, responsivas y fáciles de mantener.

5. Mejores prácticas para el uso de ganchos

Para maximizar la eficiencia y legibilidad de su código, siga estas prácticas recomendadas al usar ganchos:

5.1 Mantenga los ganchos en el nivel superior

Siempre llame a los ganchos en el nivel superior de su componente funcional. Evite llamarlos condicionalmente o dentro de bucles.

Ejemplo:

```
función MyComponent() { const
  [estado, setState] = useState(0); // Correcto si (verdadero) {

  // Evite llamar a los ganchos aquí } }
```

5.2 Usar matrices de dependencia en useEffect

Especifique siempre las dependencias en la matriz de dependencias useEffect para evitar renderizaciones repetidas innecesarias o efectos omitidos.

Ejemplo:

```
useEffect(() =>
  { console.log('Efecto activado'); },
[dependencia]); // Se ejecuta solo cuando cambia la `dependencia`
```

5.3 Mantener los efectos enfocados

Cada useEffect debe abordar una sola preocupación. Si un efecto se vuelve demasiado complejo, divídalos en varios.

Ejemplo:

```
usarEfecto(() => {
  // Manejar la obtención de
  datos } [])
```

```
}, []);
```

```
usarEfecto(() => {  
  // Manejar los oyentes de  
  eventos }, []);
```

5.4 Utilice ganchos personalizados para la reutilización

Extraiga la lógica común en ganchos personalizados para reutilizarlos en todos los componentes.

Ejemplo:

```
función useFetch(url) { const  
  [datos, setData] = useState(null);  
  
  useEffect(() =>  
  
    { fetch(url) .then((res) =>  
      res.json()) .then((data) => setData(data)); },  
    [url]);  
  
  devolver datos; }
```

```
función App() { const  
  datos = useFetch('/api/data'); devolver datos ?  
  <p>{data.title}</p> : <p>Cargando...</p>; }
```

5.5 Evite el uso excesivo del estado

Administre el estado únicamente de los datos dinámicos que afectan la interfaz de usuario.
Evite crear estados para datos derivados o estáticos.

Ejemplo:

```
//Evita esto
```

```
const [nombrecompleto, establecernombrecompleto] = useState(` ${nombre} ${apellido}`);  
  
// Utilice esta  
constante fullName = `${firstName} ${lastName}`;
```

6. Ganchos de depuración

React DevTools proporciona información sobre el comportamiento de los ganchos, lo que facilita la depuración de problemas:

1. Abra la pestaña React en DevTools.
Inspeccione el componente para ver su estado y efectos.
2. Utilice el generador de perfiles para analizar el rendimiento de la renderización e identificar cuellos de botella.

7. Transición a los ganchos

Si trabaja con una aplicación React heredada, considere refactorizar los componentes de clase a componentes funcionales con ganchos. Comience con componentes con un estado simple y vaya desarrollando gradualmente componentes más complejos.

Hooks como `useState`, `useEffect` y `useReducer` son revolucionarios para el desarrollo de React, ofreciendo una forma clara y eficiente de gestionar el estado y los efectos secundarios. Al dominar estos hooks y seguir las mejores prácticas, puedes crear aplicaciones React escalables y fáciles de mantener. Tanto si creas componentes sencillos como si gestionas lógica compleja, los hooks te permiten escribir código conciso y potente que se ajusta a los estándares modernos de React.

• Métodos del ciclo de vida de los componentes de clase

Los componentes de clase de React incluyen un conjunto bien definido de métodos de ciclo de vida que determinan su comportamiento durante su existencia en una aplicación. Estos métodos permiten a los desarrolladores controlar cómo se inician, actualizan y limpian los componentes.

lo que los hace esenciales para gestionar cambios de estado complejos, interacciones de API y gestión de recursos.

1. Descripción general de las fases del ciclo de vida

Los métodos del ciclo de vida de los componentes de clase se dividen en tres fases:

Montaje: El componente se inicializa y se inserta.

1. en el DOM.

Actualización: El componente se vuelve a renderizar debido a cambios de estado o 2. propiedad.

3. Desmontaje: El componente se retira del DOM y limpiado.

2. Fase de montaje

La fase de montaje ocurre cuando se crea un componente y se añade al DOM. Durante esta fase se activan los siguientes métodos:

2.1 constructor() ()

- Propósito: Inicializa el componente, establece el estado y vincula los controladores de eventos.
- Llamado primero en la fase de montaje.

Ejemplo:

```
clase MyComponent extiende React.Component
{ constructor(props)
  { super(props);
    this.state = { count: 0 }; } }
```

2.2 getDerivedStateFromProps() estático ()

- Propósito: Actualiza el estado en función de los cambios en los accesorios. Antes de renderizar.
- Se llama justo antes de render().

Ejemplo:

```
static getDerivedStateFromProps(props, estado) { if
  (props.initialCount !== estado.count) { return
    { count: props.initialCount }; } return null; }
```

2.3 render() ()

- Propósito: Define la estructura de UI del componente.
- Requerido en todos los componentes de la clase.

Ejemplo:

```
render()
{ return <h1>¡Hola, React!</h1>; }
```

2.4 componentDidMount() ()

- Propósito: ejecuta efectos secundarios como llamadas API o configuración activa los oyentes de eventos después de que se procesa el componente.
- Se llama una vez después de montar el componente.

Ejemplo:

```
componentDidMount()
{ console.log('Componente montado'); }
```

3. Fase de actualización

La fase de actualización ocurre cuando un componente se vuelve a renderizar debido a cambios en las propiedades, el estado o el contexto. Se activan los siguientes métodos:

3.1 `getDerivedStateFromProps()` estático `()`

- Llamado tanto en fase de montaje como de actualización a Actualizar el estado en función de los cambios de propiedad.

3.2 `shouldComponentUpdate()` `()`

- Propósito: Determina si el componente debe volver a renderizar.
- Devuelve verdadero (predeterminado) o falso.

Ejemplo:

```
shouldComponentUpdate(nextProps, nextState) {  
  devuelve nextState.count !== this.state.count;  
}
```

3.3 `render()` `()`

- Se llama durante cada actualización para definir la interfaz de usuario actualizada.

3.4 `obtenerInstantáneaAntesDeActualizar()`

- Propósito: Captura información (por ejemplo, posición de desplazamiento) antes de que se actualice el DOM.
- Devuelve un valor pasado a `componentDidUpdate()`.

Ejemplo:

```
obtenerInstantáneaAntesDeActualizar(prevProps, prevState) {  
  devuelve el documento.querySelector('.list').scrollHeight;  
}
```

3.5 `componentDidUpdate()` `()`

- Propósito: Ejecuta efectos secundarios después del componente. Vuelve a renderizar y actualiza el DOM.

Ejemplo:

```
componentDidUpdate(prevProps, prevState, instantánea) {  
  console.log('Componente actualizado');  
}
```

4. Fase de desmontaje

La fase de desmontaje se produce cuando se retira un componente del DOM. Se activa el siguiente método:

4.1 componentWillUnmount()

- **Propósito:** Limpia recursos como escuchas de eventos o temporizadores.

Ejemplo:

```
componenteSeDesmontará() {  
  console.log('El componente se desmontará');  
}
```

5. Métodos de ciclo de vida obsoletos

Algunos métodos de ciclo de vida, como `componentWillMount()`, `componentWillReceiveProps()` y `componentWillUpdate()` tienen han quedado obsoletos, favoreciendo alternativas como `getDerivedStateFromProps()` y `getSnapshotBeforeUpdate()`.

6. Mejores prácticas del método del ciclo de vida

1. Utilice `componentDidMount()` para efectos secundarios como API llamadas.
2. Optimice la actualización con `shouldComponentUpdate()`.
3. Limpie los recursos en `componentWillUnmount()` para Prevenir fugas de memoria.
4. Evite usarlo innecesariamente `obtenerEstadoDerivadoDeProps()` para minimizar la complejidad.
5. Utilice componentes funcionales con ganchos (`useEffect`, `useState`) para el desarrollo moderno de React.

• Mejores prácticas para actualizaciones estatales

Las actualizaciones de estado en React son un aspecto crucial para crear aplicaciones dinámicas y responsivas. Si se gestionan correctamente, garantizan interacciones fluidas con el usuario, una representación optimizada y un código fácil de mantener. Sin embargo, una gestión de estado inadecuada puede provocar errores, problemas de rendimiento y comportamientos no deseados.

1. Comprensión de las actualizaciones de estado de React

El estado en React es una estructura de datos inmutable que impulsa la renderización del componente. Cuando el estado cambia, React vuelve a renderizar el componente para reflejar los datos actualizados.

Puntos clave:

1. Las actualizaciones de estado son asincrónicas.
2. React procesa múltiples actualizaciones de estado para optimizar el rendimiento.
3. La función `setState` se utiliza para actualizar el estado en los componentes de clase, mientras que los ganchos como `useState` se utilizan en componentes funcionales.

2. Mejores prácticas para actualizar el estado

2.1 Utilizar actualizaciones funcionales para el estado dependiente

Cuando el nuevo estado depende del estado anterior, utilice siempre una actualización funcional.

Ejemplo:

```
este.setState((prevState) => ({ count: prevState.count + 1 }));
```

Con `useState`:

```
setCount((prevCount) => prevCount + 1);
```

2.2 Evitar la mutación de estado directa

Nunca modifique el estado directamente, ya que React no puede detectar cambios para volver a renderizar.

Ejemplo:

```
// Incorrecto
```

```
this.state.count = this.state.count + 1;
```

```
// Corrija
```

```
esto.setState({ count: this.state.count + 1 });
```

2.3 Actualizaciones de estado por lotes

React procesa automáticamente múltiples actualizaciones de estado para un mejor rendimiento.

Ejemplo:

```
establecerEstado((estadoanterior) => ({contar: estadoanterior.contar + 1 }));
```

```
establecerEstado((estadoanterior) => ({valor: estadoanterior.valor + 2 }));
```

3. Gestión de estados complejos

3.1 Usar objetos para el estado relacionado

Agrupe piezas de estado relacionadas en un solo objeto para una mejor organización.

Ejemplo:

```
const [formData, setFormData] = useState({ nombre: "", correo electrónico: "" });
```

```
const updateName = (nombre) => setFormData((prev) => ({ ...prev, nombre }));
```

3.2 Utilice useReducer para lógica de estados complejos

useReducer es ideal para gestionar estados con múltiples transiciones.

Ejemplo:

```
función reductor(estado, acción)
  { switch(acción.tipo) { caso
    'incremento': return
      { count: estado.count + 1 }; caso
    'decremento': return
      { count: estado.count - 1 };

    predeterminado: throw new Error('Acción desconocida');
  }}

const [estado, despacho] = useReducer(reductor, { count: 0 });
```

4. Optimización del rendimiento con actualizaciones de estado

4.1 Evitar estados redundantes

Almacene únicamente datos dinámicos que afecten directamente a la interfaz de usuario. Calcule datos derivados del estado o las propiedades existentes.

Ejemplo:

```
// Evite esto
const [fullName, setFullName] = useState(firstName + lastName);

// Utilice esta
constante fullName = `${firstName} ${lastName}`;
```

4.2 Evitar re-renderizados innecesarios

Utilice técnicas de memorización como `React.memo` o `useMemo` para evitar que los componentes se vuelvan a renderizar innecesariamente.

Ejemplo con `React.memo`:

```
constante MyComponent = React.memo(({ count }) => {
```

```
devuelve <div>{count}</div>; }));
```

5. Patrones de actualización de estado

5.1 Alternar estado

Utilice una actualización funcional para alternar un valor booleano.

Ejemplo:

```
setState((estadoprev) => !estadoprev);
```

5.2 /

Actualizar el estado numérico de forma incremental.

Ejemplo:

```
setState((estadoprev) => estadoprev + 1);
```

5.3 Estado de reinicio

Reiniciar el estado a su valor predeterminado.

Ejemplo:

```
setState(estadoinicial);
```

6. Manejo de actualizaciones de estado asincrónicas

React agrupa las actualizaciones de estado para mejorar el rendimiento. Para gestionar situaciones en las que se necesiten acciones tras la actualización del estado, utilice el gancho `useEffect` o las funciones de devolución de llamada en los componentes de clase.

Ejemplo con `useEffect`:

```
usarEfecto(() => {
```

```
console.log('Estado actualizado:', estado); },  
[estado]));
```

7. Depuración de actualizaciones de estado

1. Utilice React DevTools para inspeccionar el estado del componente.
2. Registre los cambios de estado en la consola para realizar un seguimiento de las actualizaciones.
3. Evite actualizar el estado en la representación para evitar bucles infinitos.

8. Transición a Hooks para la gestión de estados

Hooks como `useState` y `useReducer` simplifican la gestión de estados y reducen el código repetitivo. Considere refactorizar componentes de clase a componentes funcionales para aprovechar los hooks en aplicaciones React modernas.

Dominar los métodos del ciclo de vida y las actualizaciones de estado es esencial para crear aplicaciones React dinámicas, responsivas y eficientes. Al comprender el propósito de cada método del ciclo de vida y seguir las mejores prácticas para la gestión de estados, los desarrolladores pueden escribir código limpio y fácil de mantener que garantiza un rendimiento óptimo y experiencias de usuario fluidas.

Capítulo 7.

Manejo de eventos y formularios

• Agregar oyentes de eventos en React

React simplifica la gestión de eventos de forma declarativa, lo que facilita la adición de detectores de eventos y la gestión de las interacciones del usuario en la aplicación. A diferencia de JavaScript simple, React utiliza eventos sintéticos para garantizar la coherencia en diferentes navegadores.

1. ¿Qué son los oyentes de eventos en React? ?

Los oyentes de eventos en React son funciones que se activan mediante interacciones del usuario, como hacer clic en un botón, pasar el cursor sobre un elemento o escribir en un campo de texto. Ayudan a gestionar acciones como el envío de formularios, las llamadas a la API o las actualizaciones de la interfaz de usuario según la entrada del usuario.

Características principales del manejo de eventos de React:

Declarativo: Los eventos se especifican directamente en JSX usando 1. la sintaxis camelCase.

Eventos sintéticos: React envuelve eventos nativos en un envoltorio multiplataforma para mantener la coherencia.

Manejo en línea y externo: Los controladores de eventos pueden escribirse en línea o como funciones separadas.

2. Agregar escuchas de eventos en componentes funcionales

Paso 1. Agregar un detector de eventos en línea

React usa camelCase para los nombres de eventos, como `onClick`, `onChange` o `onKeyDown`. Puedes asociar un controlador directamente al elemento.

Ejemplo: Controlador de clic de botón

```
función App() {
```

```
return  
( <button onClick={() => alert('¡Botón hecho clic!')}>Haz clic en mí</button> ); }
```

Aquí:

- El evento onClick se adjunta al <button>.
- La función en línea se ejecuta cuando se hace clic en el botón.

hizo clic.

Paso 2 **Utilice** una función de controlador de eventos independiente

Para un código más limpio, defina el controlador de eventos como una función separada y páselo al detector de eventos.

Ejemplo: clic de botón con un controlador independiente

```
función App()  
{ const handleClick = () =>  
  { alert('¡Botón hecho clic!'); };  
  
return  
( <button onClick={handleClick}>Haz clic en mí</button> ); }
```

Ventajas de este enfoque:

- Mejora la legibilidad.
- Permite reutilizar el controlador para múltiples elementos.

Paso 3 **Pasar** parámetros a los controladores de eventos

Para pasar parámetros a los controladores de eventos, utilice una función de flecha.

Ejemplo: Paso de parámetros

```
función App() {
```

```
const handleClick = (mensaje) =>
  { alerta(mensaje); };

return
  ( <button onClick={() => handleClick('¡Hola, React!')}>Haz clic en mí</button> ); }
```

3. Agregar oyentes de eventos en componentes de clase

En los componentes de clase, los escuchas de eventos se agregan de manera similar, pero debes vincularlos para garantizar el contexto correcto.

Paso 1 Definir un controlador de eventos

Define el controlador de eventos como un método de la clase y vincúlalo en el constructor.

Ejemplo: vinculación de un controlador de eventos

clase App extiende React.Component

```
{ constructor(props)
  { super(props);
    this.handleClick = this.handleClick.bind(this); }
```

```
handleClick()
  { alert('¡Botón hecho clic!'); }
```

```
render()
  { return <button onClick={this.handleClick}>Haz clic en mí</button>; } }
```

Paso 2 Usar propiedades de clase (enfoque moderno)

Con JavaScript moderno, puedes definir el controlador como una propiedad de clase para evitar la vinculación manual.

Ejemplo: Controlador de eventos de propiedad de clase

```
clase App extiende React.Component { handleClick =  
  () => { alert('¡Botón hecho  
    clic!'); };  
  
  render()  
    { return <button onClick={this.handleClick}>Haz clic en mí</button>; } }
```

4. Manejo de tipos de eventos comunes

Paso 1 Manejo de eventos onChange (formularios) ()

El evento onChange se usa comúnmente en campos de entrada para rastrear la entrada del usuario.

Ejemplo: Campo de entrada

```
función App() { const  
  [texto, setText] = React.useState("");  
  
  const handleChange = (evento) =>  
    { setText(evento.objetivo.valor); };  
  
  devolver  
    ( <div>  
      <input type="text" value={texto} onChange={handleChange} /> <p>Entrada:  
      {texto}</p> </div> ); }
```

Paso 2 Manejo de eventos onSubmit (formularios) ()

El evento onSubmit se activa cuando se envía un formulario.

Ejemplo: Envío de formulario

```
función App() { const
  handleSubmit = (evento) =>
    { evento.preventDefault();
      alerta('¡Formulario enviado!'); };

  devolver
    ( <formulario onSubmit={handleSubmit}>
      <button type="submit">Enviar</button> </form> ); }
```

Paso 3 Manejo de eventos onMouseEnter y onMouseLeave

Estos eventos detectan cuando un usuario pasa el cursor sobre un elemento o lo abandona.

Ejemplo: Efecto de desplazamiento

```
función App() { const
  onMouseEnter = () => { console.log('Ratón
    ingresado'); };

  const onMouseLeave = () =>
    { console.log('Ratón izquierdo'); };

  devolver
    ( <div
      onMouseEnter={onMouseEnter}
      onMouseLeave={onMouseLeave} estilo={{ relleno:
        '20px', borde: '1px negro sólido' }}
    >
```

```
>  
  Pasa el cursor sobre  
  mí </
```

```
div> ); }
```

Paso 4 Manejo de eventos del teclado

React admite eventos como `onKeyDown`, `onKeyPress` y `onKeyUp`.

Ejemplo: Detección de pulsación de tecla

```
función App() { const  
  handleKeyDown = (evento) => { console.log(`Tecla  
    presionada: ${event.key}`); }; return <input type="text"  
  
  onKeyDown={handleKeyDown} />; }
```

5. Gestión de eventos sintéticos

Los eventos sintéticos de React envuelven eventos nativos para lograr consistencia entre navegadores.

Paso 1 Acceder a las propiedades del evento

Las propiedades del evento, como el objetivo, el tipo y la clave, son accesibles a través del objeto de evento sintético.

Ejemplo: Acceso al objetivo del evento

```
función App() { const  
  handleChange = (evento) => { console.log(`Valor  
    de entrada: ${event.target.value}`); };  
  
  devolver <input type="text" onChange={handleChange} />; }
```

Paso 2 Evitar la agrupación de eventos

React reutiliza objetos de evento sintéticos para mejorar el rendimiento, haciéndolos inaccesibles después de la ejecución del controlador. Usa `event.persist()` para conservar el evento.

Ejemplo: Persistir un evento

```
función App() { const
  handleClick = (evento) => { evento.persist();
    setTimeout(() =>
      { console.log(`Texto del
        botón: ${event.target.textContent}`); }, 1000); };

  regresar <button onClick={handleClick}>Haz clic en mí</button>; }
```

6. Propagación y burbujeo de eventos

Los eventos de React siguen el modelo de propagación de eventos del DOM, incluido el burbujeo y la captura.

Paso 1 Detener la propagación

Utilice `event.stopPropagation()` para evitar que el evento se propague a los elementos principales.

Ejemplo: Detener la propagación

```
función App() { const
  handleParentClick = () => { alert('El padre
    hizo clic'); };

  const handleChildClick = (evento) =>
    { evento.stopPropagation();
      alert('Niño hecho clic'); }
```

```
};  
  
devolver  
( <div onClick={handleParentClick}> <button  
  onClick={handleChildClick}>Haz clic en mí</button> </div> ); }
```

Paso 2 Captura de eventos

React admite la captura de eventos utilizando la propiedad capture.

Ejemplo: Fase de captura

```
función App() { const  
  handleCapture = () =>  
    { alert('¡Capturado!'); };  
  
  regresar <button onClickCapture={handleCapture}>Clic  
Yo</button>; }
```

7. Agregar oyentes de eventos personalizados

Para los escenarios donde el sistema de eventos de React es insuficiente, puedes agregar manualmente oyentes de eventos DOM nativos.

Paso 1 Adjuntar un detector de eventos nativo

Utilice useEffect para agregar y eliminar detectores de eventos en componentes funcionales.

Ejemplo: detector de cambio de tamaño de ventana

```
importar React, { useEffect } de 'react';  
  
función App() {
```

```
useEffect(() => { const
  handleResize = () =>
    { console.log(`Tamaño de la ventana: ${window.innerWidth}`); };

  ventana.addEventListener('resize', handleResize);

  return () =>
    { ventana.removeEventListener('resize', handleResize); }; }, []);

regresar <p>Redimensiona la ventana y verifica la consola.</p>; }
```

8. Mejores prácticas para los oyentes de eventos

1. Mantenga los controladores limpios: defina los controladores de eventos como funciones separadas para mejorar la legibilidad.
2. Evite las funciones anónimas: utilice funciones con nombre para evitar re-renderizaciones innecesarias.
3. Limpiar los oyentes: elimine siempre los oyentes en `useEffect cleanup` o `componentWillUnmount` para evitar pérdidas de memoria.
4. Usar memorización: use `useCallback` para memorizar los controladores de eventos al pasarlos como accesorios.

9. Depuración de oyentes de eventos

1. Utilice `console.log`: registre las propiedades del evento para depurar.
2. React DevTools: inspeccione los controladores de eventos adjuntos a los componentes.
3. Browser DevTools: Verifique los escuchas de eventos DOM en la pestaña "Escuchas de eventos".

Añadir detectores de eventos en React es un proceso sencillo, pero comprender cómo gestionarlos eficientemente es esencial para crear aplicaciones responsivas y fáciles de mantener. Siguiendo esta guía paso a paso, podrá gestionar las interacciones de los usuarios eficazmente, optimizar la gestión de eventos y garantizar una experiencia de usuario fluida.

• Trabajar con formularios y controlados

Componentes

Los formularios son una parte fundamental de cualquier aplicación web, ya que permiten a los usuarios introducir y enviar datos. En React, los formularios se gestionan mediante componentes controlados, donde los datos del formulario se controlan mediante el estado del componente. Este enfoque garantiza la sincronización entre la interfaz de usuario y el estado de la aplicación, lo que permite funciones como la validación en tiempo real y las interacciones dinámicas.

1. ¿Qué son los componentes controlados? ?

Un componente controlado en React es un elemento de formulario cuyo valor está totalmente controlado por el estado de React. En otras palabras, el estado del componente determina el valor de la entrada del formulario.

Características clave de los componentes controlados: :

- El valor del elemento de formulario lo establece el estado del componente. • El controlador `onChange` actualiza el estado cada vez que el usuario interactúa con el formulario.
- React se convierte en la única fuente de verdad para el formulario. datos.

2. ¿Por qué utilizar componentes controlados? ?

Los componentes controlados ofrecen varias ventajas sobre los componentes tradicionales: Manejo de formularios basado en DOM:

1. Sincronización en tiempo real: los datos del formulario siempre están sincronizados con el estado del componente.
Validación y formato: los valores de entrada se pueden 2. validar o formatear a medida que el usuario escribe.
3. Actualizaciones dinámicas: los componentes pueden reaccionar dinámicamente a la entrada del usuario, como habilitar o deshabilitar campos.
4. Comportamiento predecible: gestión centralizada del estado
Reduce errores y hace que el código sea más fácil de depurar.

3. Implementación de componentes controlados

3.1 Campos de entrada controlados

El ejemplo de componente controlado más simple implica un campo de entrada donde el valor está controlado por el estado.

Ejemplo: Entrada controlada

```
importar React, { useState } de 'react';

función ControlledInput() { const
  [valor, setValue] = useState("");

  const handleChange = (evento) =>
    { setValue(evento.objetivo.valor); };

  return
    ( <div>
      <input type="text" value={valor} onChange={handleChange} /> <p>Valor de
        entrada: {valor}</p> </div> ); }
```

```
exportar predeterminado ControlledInput;
```

3.2 Manejo de múltiples campos de entrada

Para formularios con múltiples campos, administre el estado utilizando un solo objeto donde cada campo corresponde a una clave.

Ejemplo: Gestión de múltiples campos

```
función MultiFieldForm() { const
  [formData, setFormData] = useState({ nombre de usuario: "", correo electrónico: "" });

  const handleChange = (evento) => {
```



```
const { nombre, valor } = evento.target;  
setFormData((prevData) => ({ ...prevData, [nombre]: valor })); };
```

```
devolver  
( <form>  
  <input  
    type="text"  
    name="nombreusuario"  
    value={formData.nombreusuario}  
    onChange={handleChange}  
    placeholder="Nombreusuario" /  
  >  
  <input  
    type="email"  
    name="email"  
    value={formData.email}  
    onChange={handleChange}  
    placeholder="Correo  
    electrónico" /> <button type="submit">Enviar</  
  button>  
</form> ); }
```

Aquí:

- El atributo de nombre de cada entrada coincide con una clave en el objeto de estado formData.
- La función handleChange actualiza dinámicamente el valor correspondiente.

3.3 Casilla de verificación controlada

Compruebe el estado de una casilla de verificación mediante su propiedad marcada.

Ejemplo: Casilla de verificación controlada

```

función ControlledCheckbox() { const
  [isChecked, setIsChecked] = useState(false);

  const handleChange = () =>
    { setIsChecked((prev) => !prev); };

  devolver

  ( <etiqueta> <tipo de entrada="casilla de verificación" marcada={está marcada}
onChange= {manejarCambio} />
    Aceptar términos y condiciones </
label> ); }

```

3.4 Botones de radio controlados

Los botones de opción permiten al usuario seleccionar una opción de un grupo.

Ejemplo: Botones de radio controlados

```

función ControlledRadioButtons() { const
  [opcionSeleccionada, setOpcionSeleccionada] = useState("");

  const handleChange = (evento) =>
    { setSelectedOption(evento.objetivo.valor); };

  devolver
  ( <div>
    <etiqueta>
      <tipo
        de
        entrada="radio"
        valor="Opción 1" marcada={OpciónSeleccionada
          === 'Opción 1'}
        onChange={manejarCambio} />
      Opción 1

```

```

</label>
<label>
  <input
    type="radio"
    value="Opción 2"
    marcada={opciónseleccionada === 'Opción 2'}
    onChange={manejarCambio} />

  Opción 2
</label>
<p>Opción seleccionada: {selectedOption}</p> </
div> ); }

```

3.5 Menú desplegable controlado

Utilice el atributo de valor para controlar un menú desplegable.

Ejemplo: Menú desplegable controlado

```

función ControlledSelect() { const
  [valorSeleccionado, establecerValorSeleccionado] = useState("");

  const handleChange = (evento) => {
    setSelectedValue(evento.objetivo.valor); };

  return
  ( <select value={selectedValue} onChange={handleChange}> <option
    value="">Seleccione una opción</option> <option
    value="Option 1">Opción 1</option> <option
    value="Option 2">Opción 2</option> </select> ); }

```

4. Manejo del envío del formulario

El controlador de eventos onSubmit captura los datos del formulario y evita el comportamiento de envío de formulario predeterminado.

Ejemplo: Envío de formulario

```
función SubmitForm() { const
  [formData, setFormData] = useState({ nombre de usuario: "", correo electrónico: " });

  const handleChange = (evento) => { const
    { nombre, valor } = evento.target;
    setFormData((prevData) => ({ ...prevData, [nombre]: valor })); };

  const handleSubmit = (evento) =>
    { event.preventDefault();
      console.log('Formulario enviado:', formData); };

  devolver
    ( <formulario onSubmit={handleSubmit}> <input

      tipo="texto"
      nombre="nombre de
        usuario" valor={formData.nombre de
          usuario} onChange={handleChange}
      marcador de posición="Nombre
        de
          usuario" /
      > <input
        tipo="correo
          electrónico" nombre="correo
            electrónico" valor={formData.correo
              electrónico}

        onChange={handleChange} marcador de

        posición="Correo electrónico" /> <botón tipo="enviar">Enviar</button> </form> );
    }
}
```

 }

5. Agregar validación

La validación garantiza que las entradas del usuario cumplan con los criterios específicos antes de su envío. Los componentes controlados facilitan la implementación de la validación en tiempo real.

Ejemplo: Validación básica de correo electrónico

```
función EmailValidationForm() { const [correo
  electrónico, establecerCorreo electrónico] =
  useState(""); const [error, establecerError] = useState("");

  const handleChange = (evento) => { const valor
    = evento.objetivo.valor; setEmail(valor);

    if (!value.includes('@'))
      { setError('Dirección de correo electrónico
        inválida'); }
    else

    { setError(""); } };

  devolver
    ( <div>
      <input type="email" value={email} onChange={handleChange} placeholder="Email" />
      {error && <p style={{ color:
        'red' }}>{error}</p>} </div> ); }
```

6. Mejores prácticas para trabajar con formularios y componentes controlados

1. Mantenga el estado mínimo: almacene solo datos esenciales del formulario en estado.

2. Utilice nombres descriptivos: utilice nombres significativos para las claves de estado y los atributos de nombre de entrada.
3. Agrupar estados relacionados: para formularios grandes, agrupe los campos relacionados en objetos de estado para una mejor organización.
4. Proporcionar retroalimentación en tiempo real: mostrar validación mensajes o comentarios inmediatamente a medida que los usuarios interactúan con el formulario.
5. Evite re-renderizados innecesarios: optimice el rendimiento mediante técnicas como `React.memo` o `useCallback`.
6. Casos límite de prueba: asegúrese de que los campos vacíos, las entradas no válidas y las interacciones inusuales se manejen con elegancia.

7. Depuración de formularios

1. Registros de la consola: registra actualizaciones de estado y datos de envío para depurar errores.
2. React DevTools: inspeccione el estado y las propiedades de los componentes controlados.
3. Pestaña Red: supervisa las llamadas API activadas por los envíos de formularios.

8. Uso de componentes no controlados

Si bien se prefieren los componentes controlados, los componentes no controlados (administrados por el DOM) pueden ser útiles para integrar bibliotecas de terceros o manejar formularios simples.

Ejemplo: Entrada no controlada

```
función UncontrolledInput() { const
  inputRef = React.useRef();

  const handleSubmit = () =>
    { alert(`Valor de entrada: ${inputRef.current.value}`); };

  devolver
    ( <div>
      <input type="text" ref={inputRef} /> <button
        onClick={handleSubmit}>Enviar</button>
```

</div>); }

Los componentes controlados en React proporcionan un marco robusto para gestionar formularios y la entrada de usuario. Al vincular los elementos del formulario al estado de React, los desarrolladores pueden implementar fácilmente funciones como validación, actualizaciones dinámicas y un manejo consistente de formularios. Seguir las mejores prácticas garantiza que los formularios se mantengan limpios, responsivos y fáciles de mantener, convirtiéndolos en una parte integral de cualquier aplicación React.

Dominar los componentes controlados es esencial para crear aplicaciones web modernas e interactivas.

• Técnicas de validación de formularios

Los formularios son una parte crucial de las aplicaciones web, ya que permiten a los usuarios introducir y enviar datos. Ya sea para registrarse, iniciar sesión o completar una encuesta, los formularios deben ser fiables e intuitivos. Para lograrlo, es fundamental implementar técnicas robustas de validación de formularios. Una validación adecuada garantiza la integridad de los datos, previene errores, mejora la experiencia del usuario y refuerza la seguridad.

1. La importancia de la validación de formularios

¿Por qué validar formularios?

1. Integridad de los datos: la validación garantiza que los datos enviados cumplan con los requisitos de la aplicación, como los formatos correctos para el correo electrónico o los números de teléfono.

2. Experiencia de usuario mejorada: al proporcionar retroalimentación instantánea, los usuarios pueden corregir errores antes del envío, reduciendo la frustración.

3. Seguridad: evite entradas maliciosas, como inyección SQL o secuencias de comandos entre sitios (XSS), validando y desinfectando datos.

4. Cumplimiento: garantizar el cumplimiento de estándares como el RGPD, que a menudo requieren ciertos formatos de datos y validación.

cheques.

2. Tipos de validación de formularios

2.1 Validación del lado del cliente

- Sucede en el navegador mediante JavaScript.
- Proporciona retroalimentación instantánea sin necesidad de un servidor pedido.
- Mejora la experiencia del usuario, pero no es infalible: es imprescindible estar respaldado por la validación del lado del servidor.

2.2 Validación del lado del servidor

- Se realiza en el servidor después del envío de datos.
- Garantiza una seguridad robusta al validar las entradas antes procesándolos.
- Más lento que la validación del lado del cliente, pero fundamental para Previendo ataques maliciosos.

2.3 Validación híbrida

- Combina la validación del lado del cliente y del lado del servidor.
 - Ofrece retroalimentación inmediata al tiempo que garantiza la seguridad de los datos.
- Manejo en el servidor.

3. Técnicas básicas de validación

3.1 Campos obligatorios

Asegúrese de que los campos obligatorios no queden vacíos.

Ejemplo:

```
si (!formData.nombre) {  
  console.error("El nombre es obligatorio.");  
}
```

3.2 Coincidencia de patrones

Utilice expresiones regulares para validar formatos específicos, como correos electrónicos o números de teléfono.

Ejemplo: Validación de correo electrónico

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/; if (!  
emailRegex.test(formData.email)) { console.error("Formato  
de correo electrónico no válido."); }
```

3.3 Longitud de los datos

Establecer límites mínimos y máximos de caracteres para campos como contraseñas o nombres de usuario.

Ejemplo:

```
if (formData.password.length < 8) { console.error("La  
contraseña debe tener al menos 8 caracteres."); }
```

3.4 Rango de valores

Validar las entradas numéricas para garantizar que se encuentren dentro de un rango específico.

Ejemplo:

```
if (formData.age < 18 || formData.age > 99) { console.error("La  
edad debe estar entre 18 y 99."); }
```

4. Técnicas de validación avanzadas

4.1 Validación en tiempo real

Proporciona retroalimentación instantánea a medida que el usuario escribe, mejorando la experiencia del usuario.

Implementación en React:

```
función validateEmail(correo electrónico) {  
  constante emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  devolver correo electrónicoRegex.test(correo electrónico) ? "" : "Formato de correo electrónico no válido.";  
}  
  
const [correo electrónico, establecerCorreo electrónico] = React.useState("");  
const [emailError, setEmailError] = React.useState("");  
  
const manejarCambioDeCorreoElectrónico = (e) => {  
  setEmail(e.objetivo.valor);  
  setEmailError(validateEmail(e.target.value));  
};
```

4.2 Validación condicional

Aplicar reglas basadas en los valores de otros campos.

Ejemplo:

```
if (formData.isEmployed &&!formData.employerName) {  
  console.error("Se requiere el nombre del empleador para el empleado  
individuos.");  
}
```

4.3 Validación asincrónica

Validar datos mediante API del lado del servidor, como verificar si un nombre de usuario ya esta tomado

Implementación en React:

```
función asíncrona validateUsername(nombre de usuario) {  
  const respuesta = await fetch(`/api/check-username?nombredeusuario=${nombredeusuario}`);  
  const resultado = await respuesta.json();  
  devolver resultado.isAvailable ? "" : "El nombre de usuario ya está tomado.";  
}
```

5. Validación con React y bibliotecas populares

5.1 Uso de las funciones integradas de React

React proporciona una forma flexible de gestionar la validación de formularios utilizando estados y controladores.

Ejemplo: Validación simple en React

```
const [formData, setFormData] = React.useState({ correo electrónico: "",
contraseña: "" });
const [errores, setErrors] = React.useState({});

const validate = () =>
  { const newErrors = {}; if
    (!formData.email) newErrors.email = "Se requiere correo electrónico.";
    if (formData.password.length < 8) newErrors.password = "Contraseña
debe tener al menos 8 caracteres.";
    setErrors(newErrors);
    return Object.keys(newErrors).length === 0; };

const handleSubmit = (e) =>
  { e.preventDefault(); if
    (validate())
    { console.log("¡Formulario enviado exitosamente!"); } };
```

5.2 Uso de Formik

Formik simplifica el manejo y la validación de formularios en aplicaciones React.

Ejemplo: Validación con Formik

```
importar { Formik, Formulario, Campo, MensajeDeError } de "formik";
importar *como Yup de "yup";

const validationSchema = Yup.object({
```

```
correo electrónico: Yup.string().email("Formato de correo electrónico no válido").required("Se requiere correo
```

```
electrónico"), contraseña: Yup.string().min(8, "La contraseña debe tener al menos 8 caracteres").required("Se requiere contraseña"), });
```

```
función MyForm() { return
```

```
( <Formik
```

```
  initialValues={{ email: "", password: "" }}
```

```
  validationSchema={validationSchema} onSubmit={({valores)
```

```
    => console.log(valores)}>
```

```
>
```

```
  <Formulario>
```

```
    <div>
```

```
      <Nombre del campo="correo electrónico" tipo="correo electrónico" />
```

```
      <ErrorMessage nombre="correo electrónico" />
```

```
    </div>
```

```
    <div>
```

```
      <Nombre del campo="contraseña" tipo="contraseña" />
```

```
      <ErrorMessage name="contraseña" /> </div> <button
```

```
        type="submit">Enviar</button>
```

```
    </Formulario>
```

```
</Formik> ); }
```

5.3 Uso del formulario de gancho de React

React Hook Form es una biblioteca eficiente y flexible para administrar formularios.

Ejemplo: Validación con formulario de gancho de React

```
importar { useForm } de "react-hook-form";
```

```
función MiFormulario() {
```

```
const { registrarse, manejarEnviar, formState: { errores } } = useForm();

const onSubmit = (datos) => console.log(datos);

devolver
( <formulario onSubmit={handleSubmit(onSubmit)}>
  <div>
    <input {...register("email", { required: "Se requiere correo electrónico" }}} />
    {errors.email && <p>{errors.email.message}</p>} </div> <div>
    <input
      {...register("password", { required: "Se requiere contraseña", minLength:
{ value: 8, message: "La contraseña debe tener al menos 8 caracteres" }}} } />
    {errors.password &&
      <p>{errors.password.message}</p>} </div> <button type="submit">Enviar</
button>
    </form> ); }
```

6. Mejorar la validación con las mejores prácticas de UX

6.1 Borrar mensajes de error

Escriba mensajes de error que sean concisos, específicos y procesables.

Ejemplo:

en lugar de “Entrada no válida”, utilice “La contraseña debe tener al menos 8 caracteres”.

6.2 Resaltar errores visualmente

Utilice colores, íconos o animaciones para llamar la atención sobre los campos no válidos.

Ejemplo:

- Borde rojo para campos no válidos.
- Mensajes de error en línea debajo de la entrada.

6.3 Evitar envíos duplicados

Deshabilite el botón de envío después del envío para evitar solicitudes múltiples.

7. Consideraciones de seguridad

La validación es fundamental para proteger las aplicaciones web. A continuación, se presentan algunas técnicas centradas en la seguridad:

1. Desinfectar entradas: eliminar código dañino, como scripts, de las entradas del usuario.
2. Validar en el servidor: nunca confíes en el lado del cliente validación solamente.
3. Utilice Captchas: proteja los formularios de bots y envíos de spam.

8. Prueba de validación del formulario

Asegúrese de que su lógica de validación sea sólida probando casos extremos y escenarios de usuario.

Ejemplo: Pruebas unitarias con Jest

```
test("debería devolver un error para un correo electrónico no válido",  
  () => { expect(validateEmail("invalid-email")).toBe("Formato de correo electrónico no válido."); });
```

Herramientas para pruebas:

- Cypress: Pruebe los flujos de formulario y los comportamientos en el navegador.
- Biblioteca de pruebas de React: simule interacciones del usuario y valide errores.

La validación de formularios es un aspecto esencial para crear aplicaciones web fiables e intuitivas. Al implementar técnicas eficaces, aprovechar bibliotecas modernas como Formik o React Hook Form y seguir las mejores prácticas, puede garantizar la robustez de sus formularios.

Seguro e intuitivo. Recuerda que la validación no se trata solo de detectar errores, sino de guiar a los usuarios y mejorar su experiencia general. ¡Empieza a integrar estas técnicas en tus proyectos para crear mejores formularios hoy mismo!

• Integración de bibliotecas de formularios de terceros

Las bibliotecas de formularios de terceros simplifican la gestión y validación de formularios en aplicaciones web, ahorrando tiempo y esfuerzo a los desarrolladores. Estas bibliotecas optimizan tareas como el manejo de entradas, la validación, la gestión de errores e incluso casos de uso complejos como los campos condicionales y la validación asíncrona. Esta guía ofrece una descripción detallada de las bibliotecas de formularios de terceros más populares, sus características e instrucciones paso a paso para integrarlas en aplicaciones React.

1. ¿Por qué utilizar bibliotecas de formularios de terceros?

Los formularios son parte fundamental de muchas aplicaciones, pero crearlos desde cero puede llevar mucho tiempo y ser propenso a errores. Las bibliotecas de formularios de terceros ofrecen:

- **Facilidad de uso:** herramientas prediseñadas para la gestión de formularios y validación.
- **Escalabilidad:** manejo simplificado de formularios complejos con múltiples campos y estructuras anidadas.
- **Rendimiento:** Manejo optimizado del estado del formulario y representación.
- **Soporte de la comunidad:** actualizaciones frecuentes, amplia documentación y foros comunitarios activos.

2. Bibliotecas de formularios populares para React

2.1 Formulario de gancho de React

- **Descripción general:** Biblioteca liviana con re-renderizados mínimos y una API intuitiva.
- **Características principales:**

- Validación incorporada utilizando estándares HTML.
- Integración con bibliotecas de validación de esquemas como Yup.
- Optimizado para un rendimiento controlado y

entradas no controladas

- Caso de uso: Ideal para proyectos simples a moderadamente complejos.
formularios.

2.2 Formik

- Descripción general: una biblioteca de formularios completa diseñada para
Manejar formularios de manera eficiente.

- Características principales:
- Gestión del estado del formulario.
- Soporte de validación con Yup o lógica personalizada.
- Fácil integración con componentes React.
- Caso de uso: Perfecto para formularios con validación compleja o

flujos de múltiples pasos.

2.3 Forma Redux

- Descripción general: Una biblioteca que aprovecha Redux para
Gestionar el estado del formulario.

- Características principales:
- Estado del formulario centralizado en la tienda Redux.
- Manejo simplificado de datos de formularios en todo el
solicitud.
- Caso de uso: ideal para aplicaciones que ya utilizan Redux.

2.4 Forma final de React

- Descripción general: una biblioteca de formularios flexible con soporte para
componentes controlados y no controlados.

- Características principales:
- Validación declarativa.
- Campos dinámicos y lógica condicional.
- Actualizaciones basadas en suscripción para un rendimiento óptimo.
- Caso de uso: Adecuado tanto para aplicaciones simples como dinámicas.
formularios.

3. Configuración del formulario de gancho de React

3.1 Instalación

Para instalar React Hook Form:

```
npm install react-hook-form
```

3.2 Integración básica

React Hook Form usa useForm para administrar el estado y la validación del formulario.

Ejemplo:

```
import { useForm } from "react-hook-form";

function SimpleForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = (datos) => console.log(datos);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <div>
        <input {...register("nombre de usuario", {
          requerido: "Se requiere el nombre de usuario"
        })} /> {errors.nombre de usuario && <p>{errors.nombre de usuario.mensaje}</p>} </div>
        <div>
          <input {...register("contraseña", {
            requerido: "Se requiere la contraseña",
            minLength: { valor: 8, mensaje: "Mínimo 8 caracteres" }
          })} /> {errors.contraseña && <p>{errors.contraseña.mensaje}</p>}
        </div>
        <button type="submit">Enviar</button> </form>
      );
}
```

 }

3.3 Validación del esquema con Yup

Integre la validación del esquema para requisitos más complejos:

```
npm install @hookform/resolvers sí
```

Ejemplo:

```
importar { useForm } de "react-hook-form"; importar
{ yupResolver } de "@hookform/resolvers/yup"; importar * como Yup
de "yup";

const schema = Yup.object().shape({ email:
  Yup.string().email("Formato de correo electrónico no válido").required("Se requiere
  correo
    electrónico"), Yup.string().min(8, password:      "Mínimo      8
  caracteres").required("Se requiere contraseña"), });
```

```
función SchemaForm() { const
  { register, handleSubmit, formState: { errors } } = useForm({ resolver:
    yupResolver(schema), });

  const onSubmit = (datos) => console.log(datos);

  devolver
    ( <formulario onSubmit={handleSubmit(onSubmit)}>
      <div>
        <input {...register("correo
          electrónico")} /> {errores.correo electrónico && <p>{errores.correo
            electrónico.mensaje}</p>} </div> <div>
          <input {...register("contraseña")} /> {errores.contraseña &&
            <p>{errores.contraseña.mensaje}</p>} </div>
```

```
<button type="submit">Enviar</button> </form> ); }
```

4. Uso de Formik

4.1 Instalación

Para instalar Formik:

```
npm instalar formik
```

4.2 Integración básica

Formik simplifica el manejo de formularios mediante su componente Formik o ganchos.

Ejemplo:

```
importar { Formik, Formulario, Campo, MensajeDeError } de "formik";
```

```
función BasicFormikForm() { return  
  ( <Formik
```

```
    initialValues={{ email: "", password: "" }}  
    validate={values => { const  
      errors = {}; if (!  
        values.email  
        { errors.email = "Se requiere correo  
        electrónico"; } else if (!/S+@\S+\.\S+/.test(values.email))  
        { errors.email = "Formato de correo electrónico  
  
        no válido"; } if (!  
        values.password) { errors.password = "Se requiere  
        contraseña"; } else if (values.password.length <  
        8) { errors.password = "Se requiere un mínimo de 8 caracteres"; }  
  
      return errors; }}
```

```

    }}
    onSubmit={({valores}) => console.log(valores)}
  >
  {() => (
    <Formulario>
      <div>
        <Nombre del campo="correo electrónico" tipo="correo electrónico" />
        <ErrorMessage nombre="correo electrónico" /
      > </div>
      <div>
        <Nombre del campo="contraseña" tipo="contraseña" />
        <ErrorMessage name="contraseña" /> </div>
      <button
        type="submit">Enviar</button>
    </Formulario> }}

</Formik> ); }

```

4.3 Integración avanzada con Yup

npm install sí

Ejemplo:

```

importar { Formik, Formulario, Campo, MensajeDeError } de "formik"; importar *
como Yup de "yup";

```

```

const validationSchema = Yup.object({ email:

```

```

  Yup.string().email("Se requiere un valor no
                                correo electrónico").required("Correo electrónico es
válido"),

```

```

  password: Yup.string().min(8, characters).required("Se
                                "Mínimo
                                8
requiere una contraseña"), });

```

```

función FormikWithYup() {

```

```
devolver
( <Formik
  initialValues={{ correo electrónico: "", contraseña:
  "" }} validationSchema={validationSchema}
  onSubmit={(valores) => console.log(valores)}
>
  <Formulario>
    <div>
      <Nombre del campo="correo electrónico" tipo="correo electrónico" />
      <ErrorMessage nombre="correo electrónico" /
    > </div>
    <div>
      <Nombre del campo="contraseña" tipo="contraseña" />
      <ErrorMessage name="contraseña" /> </div>
    <button
      type="submit">Enviar</button>
  </Formulario>
</Formik> ); }
```

5. Uso de React Final Form

5.1 Instalación

Para instalar React Final Form:

```
npm install final-form react-final-form
```

5.2 Integración básica

React Final Form proporciona una API sencilla para el manejo de formularios.

Ejemplo:

```
importar { Formulario, Campo } de "react-final-form";
```

```
función BasicFinalForm() {
```

```

const onSubmit = (valores) => console.log(valores);

return

( <Formulario
  onSubmit={onSubmit}
  validar={({valores}) =>
    { const errores = {}; si (!
      valores.nombreusuario) { errores.nombreusuario =

        "Se requiere el

nombre de usuario"; } devolver errores; }}
  render={({ handleSubmit }) => ( <formulario onSubmit={handleSubmit}>
    <div>
      <Field name="nombre de
        usuario"> {({ entrada,
          meta })
            => ( <div> <input {...input} placeholder="Nombre de
              usuario" /> {meta.error && meta.touched && <span>{meta.error}
            }
          </
            div> }}
        </Campo>
      </div>
      <button type="submit">Enviar</button> </
        form> )} /
    > ); }

```

6. Mejores prácticas para la integración de bibliotecas de formularios

1. Elija la biblioteca adecuada:

- Utilice bibliotecas ligeras como React Hook Form para formas simples.

- Opte por Formik o Redux Form para un uso más complejo casos.
- 2. Validación del esquema de apalancamiento:
 - Utilice Yup o bibliotecas similares para una validación consistente lógica.
- 3. Optimizar el rendimiento:
 - Minimizar las repeticiones de renderizados mediante el uso de componentes controlados juiciosamente.
 - Evite sobrecargar formularios con campos innecesarios o lógica.
- 4. Centrarse en la experiencia del usuario:
 - Proporcionar mensajes de error claros.
 - Utilice la validación en tiempo real para obtener mejores comentarios.

Integración de bibliotecas de formularios de terceros como React Hook Form, Formik o

React Final Form simplifica el proceso de gestión y validación

Formularios en aplicaciones React. Al elegir la biblioteca adecuada según

las necesidades de su proyecto y aprovechar funciones avanzadas como el esquema

Con la validación y los campos dinámicos, puede crear formularios robustos y fáciles de usar.

formularios con el mínimo esfuerzo. Experimente con estas bibliotecas y adopte

Mejores prácticas para garantizar que sus formularios sean eficientes y fáciles de mantener.

y agradable para que los usuarios interactúen con ellos.resolvers';

importar * como Yup de 'yup';

Capítulo 8.

Enrutamiento y navegación de React

• Introducción a React Router

React Router es una biblioteca popular para gestionar el enrutamiento y la navegación en aplicaciones React. Proporciona una forma declarativa de definir y gestionar rutas, lo que permite transiciones fluidas entre diferentes páginas o vistas dentro de una aplicación de página única (SPA). Con React Router, los desarrolladores pueden crear sistemas de navegación dinámicos del lado del cliente, manteniendo una experiencia de usuario consistente.

1. ¿Qué es React Router? ?

React Router es una biblioteca estándar para el enrutamiento en React. Permite a los desarrolladores crear estructuras de navegación que gestionan:

- Enrutamiento basado en URL: asigna URL a direcciones específicas de componentes.
- Enrutamiento dinámico: adapta las rutas según el estado de la aplicación o la entrada del usuario.
- Enrutamiento anidado: admite rutas jerárquicas para diseños complejos.

A diferencia de los sitios web tradicionales de varias páginas, React Router funciona en el contexto de una aplicación de una sola página. Manipula la API del historial del navegador para actualizar la URL y renderizar los componentes correspondientes sin necesidad de recargar la página completa.

2. ¿Por qué utilizar React Router? ?

React Router simplifica la navegación y mejora la experiencia del usuario en las SPA. Estas son sus principales ventajas:

1. Enrutamiento declarativo:

Las rutas se definen de forma declarativa mediante JSX, lo que hace que el código sea fácil de leer y mantener.

2. Manejo de URL dinámico: React

Router admite parámetros dinámicos, cadenas de consulta y coincidencia de rutas para una navegación flexible.

3. Transiciones fluidas:

Actualiza la URL y renderiza los componentes sin actualizar la página, lo que da como resultado transiciones más rápidas y fluidas.

4. Rutas anidadas:

Los desarrolladores pueden definir rutas secundarias, lo que permite diseños modulares y jerárquicos.

5. Integración con el historial del navegador:

React Router proporciona soporte integrado para la API de historial del navegador, lo que permite la navegación, la funcionalidad de avance/retroceso y el enrutamiento programático.

6. División de código:

Admite carga diferida, lo que permite la división de código para reducir el tiempo de carga inicial de una aplicación.

3. Conceptos básicos de React Router

3.1 Enrutador del navegador

BrowserRouter es el componente de nivel superior que encapsula toda la aplicación y gestiona la navegación. Utiliza la API de historial de HTML5 para mantener la interfaz de usuario sincronizada con la URL.

Ejemplo:

```
import { BrowserRouter } from 'react-router-dom';
```

función App()

```
  { return (  
    <Enrutador del navegador>  
      { /*Las rutas se definirán aquí*/}  
    </BrowserRouter> ); }
```

3.2 Rutas y Ruta

- El componente Rutas actúa como un contenedor para todas las rutas.
- Cada Ruta define un camino y el componente a seguir.

Ejemplo:

```
importar { BrowserRouter, Routes, Route } desde 'react-router-dom';
```

```
función Inicio()
```

```
  { return <h1>Página de inicio</h1>; }
```

```
función Acerca de()
```

```
  { devolver <h1>Página Acerca de</h1>; }
```

```
función App()
```

```
  { return (  
    <Enrutador del navegador>  
      <Rutas>  
        <Ruta de ruta="/" elemento={<Inicio />} />  
        <Route path="/about" element={<Acerca de />} />  
      </Rutas>  
    </BrowserRouter> ); }
```

En este ejemplo:

- Al navegar hasta / se representa el componente Inicio.
- Al navegar a /about se muestra el componente Acerca de.

3.3 Enlace

El componente Enlace proporciona navegación declarativa sin recargar la página. Reemplaza las etiquetas de anclaje tradicionales (<a>).

Ejemplo:

```
importar { Enlace } desde 'react-router-dom';
```

```
función Navbar()
```

```
  { return (
```

```
    <navegación>
```

```
      Inicio
```

```
      <Link to="/about">Acerca de</Link> </
```

```
      nav> ); }
```

3.4 useNavigate

El gancho useNavigate permite la navegación programática dentro de la aplicación.

Ejemplo:

```
importar { useNavigate } desde 'react-router-dom';
```

```
función Iniciar
```

```
  sesión() { const navegar = useNavigate();
```

```
  const handleLogin = () => {
```

```
    // Después de iniciar sesión, redirigir al panel
```

```
    de control browse('/
```

```
    dashboard'); };
```

```
  regresar <button onClick={handleLogin}>Iniciar sesión</button>; }
```

4. Configuración del enrutador React

4.1 Instalación

Instalar React Router usando npm o yarn:

npm instalar react-router-dom

4.2 Agregar rutas

Una vez instalado, envuelva el componente raíz con BrowserRouter y defina rutas utilizando Routes y Route.

Ejemplo: Configuración básica

```
importar React desde 'react';
importar { BrowserRouter, Routes, Route } desde 'react-router-dom';

función App()
{ return (
  <Enrutador del navegador>
    <Rutas>
      <Ruta de ruta="/" elemento={<Inicio />} />
      <Route path="/about" element={<Acerca de />} />
    </Rutas>
  </BrowserRouter> ); }
```

5. Funciones avanzadas

5.1 Rutas anidadas

React Router admite rutas anidadas para diseños jerárquicos.

Ejemplo:

```
función App()
{ return (
  <Enrutador del navegador>
    <Rutas>
      <Ruta de ruta="/" elemento={<Diseño />}>
        <Elemento de índice de ruta={<Inicio />} />
        <Route path="about" element={<Acerca de />} />
      </Ruta de ruta="/" elemento={<Diseño />}>
    </Rutas>
  </BrowserRouter> ); }
```

```
    </Ruta>
  </Rutas>
</BrowserRouter> ); }
```

En esta configuración:

- El componente Diseño es la ruta principal.
- Los componentes Inicio y Acerca de están anidados debajo de él.

5.2 Rutas dinámicas

Las rutas dinámicas utilizan parámetros para manejar URL variables.

Ejemplo:

```
importar { useParams } de 'react-router-dom';

función Producto()
{ const { id } = useParams();
  return <h1>ID del producto: {id}</h1>; }

función App()
{ return (
  <Enrutador del navegador>
    <Rutas>
      <Route path="/product/:id" element={<Product />} />
    </Rutas>
  </BrowserRouter> ); }
```

Aquí:

- Al navegar a /product/123 se muestra el producto con id igual a 123.

5.3 Manejo de errores con rutas de respaldo

React Router proporciona una forma de manejar rutas no coincidentes usando un comodín (*).

Ejemplo:

```
función NotFound() { return  
  <h1>Página no encontrada</h1>; }
```

```
función App()  
{ return (  
  <Enrutador del navegador>  
    <Rutas>  
      <Ruta de ruta="/" elemento={<Inicio />} />  
      <Ruta de ruta="*" elemento={<No encontrado />} />  
    </Rutas>  
  </BrowserRouter> ); }
```

6. Mejores prácticas para React Router

1. Organice las rutas de forma lógica: agrupe las rutas relacionadas para lograr una mejor legibilidad y facilidad de mantenimiento.
2. Utilice la carga diferida: optimice el rendimiento cargando rutas dinámicamente usando React.lazy y Suspense.
3. Manejar la autenticación: proteja las rutas privadas con envoltorios de autenticación personalizados.
4. Optimización SEO: utiliza react-helmet o similar
Bibliotecas para gestionar metadatos para una mejor visibilidad en los motores de búsqueda.
5. Límites de error: manejar errores inesperados con gracia, con componentes de respaldo.

React Router es una herramienta indispensable para crear sistemas de navegación dinámicos e intuitivos en aplicaciones web modernas. Su sintaxis declarativa, flexibilidad y robustas funciones lo convierten en la opción ideal para gestionar rutas en aplicaciones de una sola página. Al comprender sus conceptos básicos y aprovechar las funciones avanzadas,

Al igual que las rutas anidadas y los parámetros dinámicos, los desarrolladores pueden crear experiencias de navegación fluidas y eficientes para sus usuarios.

• Configuración de la navegación

La navegación es fundamental en cualquier aplicación web, ya que permite a los usuarios navegar entre diferentes páginas o secciones sin problemas. En React, la navegación se gestiona habitualmente mediante la biblioteca React Router, que proporciona herramientas para gestionar el enrutamiento y la navegación en aplicaciones de página única (SPA). Esta guía explica cómo configurar la navegación en React paso a paso.

1. Instalación de React Router

React Router es la biblioteca más utilizada para la navegación en aplicaciones React. Para empezar, instala react-router-dom con npm o yarn.

Paso 1. Instalar React Router

Ejecute el siguiente comando en el directorio de su proyecto:

```
npm instalar react-router-dom
```

Para hilo:

```
hilo agregar react-router-dom
```

Paso 2. Verificar la instalación

Verifique el archivo package.json para asegurarse de que react-router-dom esté listado como una dependencia.

2. Configuración de la estructura del proyecto

Antes de configurar la navegación, es esencial organizar los archivos del proyecto para una mejor escalabilidad y legibilidad.

Estructura de carpetas recomendada

src/

├── componentes/ # Componentes compartidos o reutilizables | ├── Header.js | ├──

Footer.js ├── páginas/ #

Componentes para diferentes

páginas | ├── Home.js | ├── About.js | ├── Contact.js ├── App.js #

Componente principal de la

aplicación ├── index.js #

Punto de entrada

3. Envoltiendo la aplicación con BrowserRouter

El componente BrowserRouter proporciona el contexto de enrutamiento a tu aplicación. Debe contener toda la aplicación, normalmente en el archivo index.js o App.js.

Paso 1 Importar BrowserRouter

Importar BrowserRouter desde react-router-dom.

Paso 2 Envuelva la aplicación

En App.js o index.js, envuelva el componente principal con BrowserRouter.

Ejemplo: Envolver la aplicación

```
importar React desde 'react';
```

```
importar ReactDOM desde 'react-dom'; importar
```

```
{ BrowserRouter } desde 'react-router-dom'; importar App desde './
```

```
App';
```

```
ReactDOM.render( <Enrutador  
  del navegador>
```

```
    <Aplicación />
```



```
</BrowserRouter>,  
document.getElementById('root') );
```

4. Definición de rutas

Las rutas asignan URL a los componentes. Usa las rutas y los componentes de ruta de React Router para definir la estructura de navegación.

Paso 1 Crear páginas

Cree componentes básicos para cada página en la carpeta pages/:

- Inicio.js
-

```
función Inicio() { return  
  <h1>Bienvenido a la página de inicio</h1>; }
```

```
exportar predeterminado Inicio;
```

- Acerca de.js
-

```
función Acerca de()  
  { return <h1>Acerca de nosotros</h1>; }
```

```
exportar predeterminado Acerca de;
```

- Contacto.js
-

```
función Contacto() { return  
  <h1>Contáctenos</h1>; }
```

```
exportar contacto predeterminado;
```

Paso 2 Importar páginas a .

```
importar Inicio desde './pages/Inicio';  
importar Acerca de desde './pages/  
Acerca de'; importar Contacto desde './pages/Contacto';
```

Paso 3: Configurar rutas

Envuelva las rutas dentro del componente Rutas en App.js.

Ejemplo: Configuración de rutas

```
importar { Rutas, Ruta } desde 'react-router-dom'; función  
App() { return ( <div>  
  
  <Rutas>  
    <Ruta de ruta="/" elemento={<Inicio />} />  
    <Route path="/about" element={<Acerca de />} />  
    <Route path="/contact" element={<Contacto />} />  
  </Rutas> </  
div> ); }
```

```
exportar aplicación predeterminada;
```

5. Agregar un menú de navegación

Para navegar entre páginas, agregue un menú de navegación usando el componente Enlace de React Router.

Paso 1: Crear un componente de encabezado

Cree un archivo Header.js en la carpeta componentes/.

Ejemplo: Menú de navegación

```
importar { Enlace } desde 'react-router-dom';
```

```
función Encabezado()  
{ return (  
  <navegación>  
  <ul>  
    <li><Link to="/">Inicio</Link></li> <li><Link to="/  
    about">Acerca de</Link></li> <li><Link to="/  
    contact">Contacto</Link></li> </ul> </nav> ); }
```

```
exportar encabezado predeterminado;
```

Paso 2. Incluir el encabezado en `App.js`.

Importe e incluya el componente Header en App.js:

```
importar encabezado desde './componentes/Header';
```

```
función App() { return  
  ( <div>  
  
    <Encabezado />  
    <Rutas>  
      <Ruta de ruta="/" elemento={<Inicio />} />  
      <Route path="/about" element={<Acerca de />} />  
      <Route path="/contact" element={<Contacto />} />  
    </Rutas> </div> ); }
```

```
exportar aplicación predeterminada;
```

6. Cómo darle estilo al menú de navegación

Utilice CSS para diseñar el menú de navegación para una mejor experiencia del usuario.

Paso 1 Crear un archivo CSS

Cree un archivo Header.css en la carpeta componentes/:

```
nav ul
  { tipo-estilo-lista: ninguno;
    relleno: 0;
    visualización:
    flex; espacio:
    10px; }

nav ul li
  { display: en
  línea; }

nav ul li a
  { decoración de texto:
    ninguna;
  color: azul; }

nav ul li a: hover
  { decoración de texto: subrayado; }
```

Paso 2: Importar el archivo CSS

Importa el archivo CSS en Header.js:

```
importar './Header.css';
```

7. Navegación dinámica y programática

React Router admite la navegación dinámica basada en acciones o condiciones del usuario mediante el gancho `useNavigate`.

Paso 1 Navegar programáticamente

Utilice el gancho `useNavigate` para activar la navegación en una función.

Ejemplo: Navegación de inicio de sesión

```
importar { useNavigate } desde 'react-router-dom';

función Iniciar sesión()
  { const navegar = useNavigate();

  const handleLogin = () => {
    // Después de iniciar sesión correctamente, navegue a la página de
    inicio browse('/'); };

  regresar <button onClick={handleLogin}>Iniciar sesión</button>; }
```

```
exportar inicio de sesión predeterminado;
```

8. Manejo de rutas no encontradas (404)

Para manejar URL no válidas, defina una ruta alternativa con un comodín (*).

Paso 1 Crear un componente NotFound

```
función NotFound() { return
  <h1>404: Página no encontrada</h1>; }
```

```
exportar predeterminado NotFound;
```

Paso 2 Agregar ruta de respaldo

Incluya la ruta de respaldo en App.js:

```
importar NotFound desde './pages/NotFound';

<Rutas>
  <Ruta de ruta="/" elemento={<Inicio />} />
  <Route path="/about" element={<Acerca de />} />
  <Route path="/contact" element={<Contacto />} />
  <Ruta de ruta="*" elemento={<No encontrado />} />
</Rutas>
```

9. Rutas anidadas

Para diseños complejos, puede definir rutas anidadas.

Paso 1 Crear un componente de diseño

función Diseño()

```
{ return  
  ( <div>  
    <Encabezado />  
    <Salida /> </div> ); }
```

exportar diseño predeterminado;

Paso 2 Actualizar rutas con estructura anidada

importar diseño desde './componentes/Layout';

<Rutas>

```
<Ruta de ruta="/" elemento={<Diseño />}>  
  <Elemento de índice de ruta={<Inicio />} />  
  <Route path="about" element={<Acerca de />} />  
  <Route path="contact" element={<Contacto />} />  
</Ruta>
```

</Rutas>

10. Mejores prácticas para la navegación

1. Organice las rutas de forma lógica: agrupe las rutas relacionadas.
2. Componentes de carga diferida: use React.lazy y Suspense para cargar componentes solo cuando sea necesario.
3. Proporcionar comentarios sobre los estados de carga: Mostrar cargadores para rutas más lentas.

4. Rutas privadas seguras: proteja las rutas sensibles con lógica de autenticación.

Configurar la navegación en React con React Router es un proceso sencillo pero potente. Siguiendo esta guía paso a paso, puedes crear una estructura de navegación robusta que mejore la experiencia del usuario.

Desde definiciones de rutas simples hasta funciones avanzadas como rutas anidadas y navegación programática, React Router proporciona a los desarrolladores todas las herramientas necesarias para crear SPA dinámicas y sin inconvenientes.

• Enrutamiento dinámico y parámetros de ruta

El enrutamiento dinámico y los parámetros de ruta son características esenciales para crear aplicaciones web flexibles, interactivas y basadas en datos. React Router permite a los desarrolladores crear rutas dinámicas que se adaptan a las entradas del usuario y al estado de la aplicación, lo que facilita la creación de aplicaciones de página única (SPA) complejas y escalables.

1. Comprensión del enrutamiento dinámico

El enrutamiento dinámico se refiere a la creación de rutas que pueden gestionar segmentos variables en una URL. En lugar de definir rutas estáticas como `/home` o `/about`, el enrutamiento dinámico permite rutas como `/product/:id`, donde `:id` es un parámetro que cambia según el producto específico al que se accede.

Beneficios del enrutamiento dinámico

1. Flexibilidad: Maneje fácilmente datos específicos del usuario o específicos del usuario. rutas.

Reutilización: utilice una única definición de ruta para varias instancias dinámicas.

Escalabilidad: simplifica la navegación en aplicaciones grandes 3. con contenido dinámico.

2. ¿Qué son los parámetros de ruta? ?

Los parámetros de ruta son marcadores de posición en una ruta URL que capturan valores dinámicos. Se puede acceder a estos parámetros y usarlos para obtener datos o renderizar componentes específicos dinámicamente.

Ejemplo de parámetros de ruta:

```
<Route path="/product/:id" element={<Product />} />
```

En este ejemplo:

- `:id` es un parámetro de ruta que captura valores dinámicos desde la URL, como `/producto/123`.

3. Configuración del enrutamiento dinámico en React

Para implementar el enrutamiento dinámico, utilizará las funciones de React Router, como `Route`, `useParams` y rutas anidadas.

Paso 1. Instalar React Router

Asegúrese de que React Router esté instalado en su proyecto:

```
npm instalar react-router-dom
```

Paso 2. Definir rutas dinámicas

Define una ruta con parámetros utilizando el componente `Ruta`.

Ejemplo: Definición de una ruta dinámica

```
importar { BrowserRouter, Routes, Route } desde 'react-router-dom'; importar Home  
desde './Home'; importar Product  
desde './Product';
```

```
función App() { return  
  (  
    <Enrutador del navegador>  
      <Rutas>  
        <Ruta de ruta="/" elemento={<Inicio />} />
```



```
      <Route path="/product/:id" element={<Product />} />
    </Rutas>
  </BrowserRouter> ); }
```

exportar aplicación predeterminada;

4. Acceso a parámetros de ruta con useParams

React Router proporciona el gancho useParams para acceder a los parámetros de ruta en el componente renderizado por una ruta dinámica.

Paso 1 Importar y usar useParams

Importa el gancho useParams y úsalo dentro del componente.

Ejemplo: Acceso al parámetro id

```
importar { useParams } de 'react-router-dom';
```

```
función Producto()
```

```
  { const { id } = useParams();
```

```
    devolver <h1>ID del producto: {id}</h1>; }
```

exportar producto predeterminado;

Si la URL es /product/123, el valor del id será 123.

5. Rutas dinámicas anidadas

Las rutas anidadas le permiten crear estructuras jerárquicas, lo que facilita la gestión de diseños complejos o contenido basado en datos.

Paso 1 Definir rutas anidadas

Utilice el componente Outlet para definir una relación padre-hijo para las rutas.

Ejemplo: Rutas dinámicas anidadas

```
importar { BrowserRouter, Routes, Route, Outlet } desde 'react-router-dom';
```

```
función Categorías() { return  
  ( <div>
```

```
    <h1>Categorías</h1>  
    <Salida />  
  </
```

```
div> ); }
```

```
función SubCategoría() { const  
  { categoría } = useParams();
```

```
  devolver <h1>Subcategoría: {categoría}</h1>; }
```

```
función App()
```

```
  { return (
```

```
    <Enrutador del navegador>
```

```
    <Rutas>
```

```
      <Route path="/categories" element={<Categorías />}>
```

```
        <Route path=":category" element={<SubCategoría />} />
```

```
      </Ruta>
```

```
    </Rutas>
```

```
  </BrowserRouter> ); }
```

```
exportar aplicación predeterminada;
```

Cómo funciona:

- El componente Categorías actúa como el componente principal ruta.
- El componente SubCategoría se representa en función del parámetro de categoría (por ejemplo, /categorías/electrónica).

6. Parámetros opcionales

React Router admite parámetros opcionales, que pueden hacer que una ruta sea más flexible.

Paso 1 Definir parámetros opcionales

Utilice la sintaxis del signo de interrogación (?) para definir un parámetro opcional.

Ejemplo: Parámetro opcional

```
<Route path="/profile/:nombreusuario?" elemento={<Perfil />} />
```

Esta ruta coincide con /profile y /profile/johndoe.

Paso 2 Manejar parámetros opcionales

Utilice useParams para acceder al parámetro opcional.

Ejemplo: Componente de perfil

```
función Perfil() { const
  { nombre de usuario } = useParams();

  devolver <h1>{nombreusuario ? `Hola, ${nombreusuario}` : 'Hola, Invitado'} </h1>; }
```

```
exportar perfil predeterminado;
```

7. Parámetros de consulta

Los parámetros de consulta son una forma alternativa de pasar datos dinámicos en la URL. A diferencia de los parámetros de ruta, los parámetros de consulta no forman parte de la ruta, sino que se añaden a la URL con un signo de interrogación (?).

Paso 1 Acceder a los parámetros de consulta

Utilice el gancho useSearchParams para leer los parámetros de consulta.

Ejemplo: Acceso a parámetros de consulta

```
importar { useSearchParams } de 'react-router-dom'; función Buscar()

{ const [searchParams] =
  useSearchParams(); const palabra clave =
  searchParams.get('palabra clave');

  devolver <h1>Resultados de búsqueda para: {palabra clave}</h1>; }
```

```
exportar búsqueda predeterminada;
```

Si la URL es /search?keyword=laptop, el valor de la palabra clave será laptop.

Paso 2 Actualizar los parámetros de consulta

Utilice el segundo parámetro de useSearchParams para establecer o actualizar los parámetros de consulta.

Ejemplo: Actualización de parámetros de consulta

```
función SearchInput() { const
  [searchParams, setSearchParams] = useSearchParams();

  const handleSearch = (evento) =>
    { setSearchParams({ palabra clave: evento.objetivo.valor }); };

  devolver <input type="text" onChange={handleSearch}
  placeholder="Buscar..." />; }
```

```
exportar predeterminado SearchInput;
```

8. Navegación programática con rutas dinámicas

React Router proporciona el gancho useNavigate para la navegación programática, lo que le permite navegar dinámicamente en función de eventos o interacciones del usuario.

Ejemplo: Redireccionamiento con parámetros

```
importar { useNavigate } desde 'react-router-dom';

función ListaProductos()
  { const navegar = useNavigate();

  const goToProduct = (id) =>
    { browse(`/producto/${id}`); };

  devolver
    ( <div>
      <button onClick={() => goToProduct(1)}>Producto 1</button> <button
        onClick={() => goToProduct(2)}>Producto 2</button> </div> ); }

exportar lista de productos predeterminada;
```

9. Enrutamiento dinámico con obtención de datos

Las rutas dinámicas a menudo implican la obtención de datos según el parámetro de la URL.

Ejemplo: obtención de datos con parámetros

```
importar { useParams } de 'react-router-dom'; importar
{ useEffect, useState } de 'react';

función Producto()
  { const { id } = useParams();
  const [producto, setProduct] = useState(null);

  useEffect(() =>
    { fetch(`/api/products/${id}`)
      .then((respuesta) =>
        respuesta.json()) .then((datos) => setProduct(datos));
```

```
}, [identificación]);
```

```
si (!producto) devuelve <p>Cargando...</p>;
```

```
devolver
```

```
( <div>
```

```
  <h1>{producto.nombre}</h1>
```

```
  <p>{producto.descripcion}</p> </div> ); }
```

```
exportar producto predeterminado;
```

10. Mejores prácticas para el enrutamiento dinámico

1. Validar parámetros: garantizar que los parámetros de ruta sean Válido antes del procesamiento.
2. Proporcionar alternativas: manejar parámetros no válidos o faltantes con elegancia.
3. Utilice componentes modulares: separe la lógica de ruta en componentes dedicados para lograr claridad y reutilización.
4. Optimice la obtención de datos: minimice las llamadas a la API almacenando en caché o precargando datos cuando sea posible.
5. Rutas sensibles seguras: proteja las rutas con lógica de autenticación y autorización.

El enrutamiento dinámico y los parámetros de ruta son herramientas potentes para crear aplicaciones responsivas y orientadas al usuario en React. Con React Router, los desarrolladores pueden crear rutas que se adaptan a datos dinámicos, gestionan rutas anidadas y admiten parámetros de consulta. Al dominar estas funciones y seguir las mejores prácticas, pueden crear SPA escalables y fáciles de mantener que ofrecen una experiencia de usuario fluida.

- Manejo de rutas anidadas

Las rutas anidadas son una potente función de React Router que permite a los desarrolladores crear una jerarquía de rutas dentro de una aplicación. Permiten diseños complejos, navegación estructurada y una representación fluida de componentes, lo que las hace especialmente útiles para aplicaciones con contenido altamente anidado o páginas modulares.

1. ¿Qué son las rutas anidadas? ?

Las rutas anidadas permiten una estructura de enrutamiento jerárquica, donde las rutas secundarias se definen dentro de una ruta principal. Son especialmente útiles para crear páginas con diseños compartidos o navegación multinivel.

Ejemplo de rutas anidadas

- URL: /panel de control/configuración
- Ruta principal: /dashboard
- Ruta secundaria: /dashboard/settings

2. ¿Por qué utilizar rutas anidadas? ?

Las rutas anidadas mejoran la organización y la escalabilidad de su aplicación al:

1. Reducir la repetición: evite duplicar diseños o elementos de navegación en múltiples componentes.
2. Optimización de diseños: renderizar diseños principales y secundarios componentes dentro de la misma vista.
3. Mejorar la modularidad: crear estructuras de rutas modulares y reutilizables.

3. Configuración de React Router para rutas anidadas

Para implementar rutas anidadas, usará el componente Outlet de React Router. Este componente actúa como un marcador de posición para renderizar los componentes secundarios.

Paso 1. Instalar React Router

Asegúrese de que react-router-dom esté instalado:

npm instalar react-router-dom

Paso 2 Estructura básica de carpetas

Organice sus archivos para una mejor mantenibilidad:

src/

```
|— componentes/ | —  
Layout.js | — Navbar.js  
|— páginas/ | —  
Dashboard.js | —  
Profile.js | — Settings.js —  
App.js
```

4. Definición de rutas anidadas

Utilice el componente Outlet para definir una relación de ruta padre-hijo.

Paso 1 Crear un diseño principal

El diseño principal contiene componentes compartidos como encabezados o barras laterales.

Ejemplo: Layout.js

```
importar { Salida, Enlace } desde 'react-router-dom';
```

```
función Diseño()
```

```
{ devolver
```

```
( <div>
```

```
  <navegación>
```

```
  <ul>
```

```
    <li><Link to="profile">Perfil</Link></li> <li><Link  
    to="settings">Configuración</Link></li> </ul>
```



```

    </nav>

    <Outlet /> { /* Marcador de posición para componentes secundarios
    */ } </

div> ); }

exportar diseño predeterminado;

```

Paso 2 Definir rutas

Configurar rutas anidadas en App.js.

Ejemplo: Configuración de ruta anidada

```

importar { BrowserRouter, Routes, Route } desde 'react-router-dom'; importar Layout
desde './components/Layout'; importar Dashboard
desde './pages/Dashboard'; importar Profile desde './
pages/Profile'; importar Settings desde './
pages/Settings';

función App() { return
(
  <Enrutador del navegador>
    <Rutas>
      <Route path="/dashboard" element={<Layout />}>
        <Elemento de índice de ruta={<Dashboard />} /> { /* Ruta secundaria
predeterminada */ }
        <Route path="perfil" elemento={<Perfil />} />
        <Route path="settings" element={<Configuración />} />
      </Ruta>
    </Rutas>
  </BrowserRouter> ); }

```

exportar aplicación predeterminada;

Paso 3 Crear componentes secundarios

Define componentes individuales para las rutas secundarias.

Ejemplo: Dashboard.js .

```
función Dashboard() { return  
  <h1>Bienvenido al Dashboard</h1>; }
```

```
exportar tablero predeterminado;
```

Ejemplo: Profile.js .

```
función Perfil() { return  
  <h1>Página de perfil</h1>; }
```

```
exportar perfil predeterminado;
```

Ejemplo: Settings.js .

```
función Configuración()  
  { return <h1>Página de configuración</  
h1>; }
```

```
exportar configuración predeterminada;
```

5. Rutas anidadas con parámetros

Los parámetros permiten que se pasen datos dinámicos a través de rutas anidadas.

Paso 1 Definir rutas parametrizadas

Añade parámetros a las rutas secundarias.

Ejemplo: Ruta anidada parametrizada

```
<Route path="perfil/:userId" elemento={<Perfil />} />
```

Paso 2 Acceder a los parámetros en los componentes secundarios

Utilice el gancho useParams para acceder a los parámetros de ruta.

Ejemplo: Profile.js con parámetros

```
importar { useParams } de 'react-router-dom';

función Perfil() { const
  { userId } = useParams();

  devolver <h1>Perfil para el ID de usuario: {userId}</h1>; }

exportar perfil predeterminado;
```

- Si la URL es /dashboard/profile/123, el valor de userId será 123.

6. Uso de rutas de índice

Las rutas de índice son rutas secundarias predeterminadas que se representan cuando coincide con la ruta principal.

Paso 1 Definir una ruta de índice

Utilice el atributo de índice en la ruta secundaria.

Ejemplo: Ruta de índice

```
<Route path="/dashboard" element={<Layout />}>
  <Elemento de índice de ruta={<Dashboard />} /> { /* Representado por defecto */ }

  <Route path="perfil" elemento={<Perfil />} />
  <Route path="settings" element={<Configuración />} />
</Ruta>
```

- Navegar a /dashboard mostrará el Dashboard componente.

7. Manejo de errores en rutas anidadas

React Router permite rutas de respaldo para manejar rutas no coincidentes.

Paso 1 Crear un componente NotFound

```
función NotFound() { return  
  <h1>404: Página no encontrada</h1>; }
```

```
exportar predeterminado NotFound;
```

Paso 2 Agregar una ruta de respaldo

Incluya una ruta comodín (*) para capturar rutas no coincidentes.

Ejemplo: Ruta de respaldo

```
<Route path="/dashboard" element={<Layout />}>  
  <Elemento de índice de ruta={<Panel de control />} />  
  <Route path="perfil" elemento={<Perfil />} />  
  <Route path="settings" element={<Configuración />} />  
  <Route path="*" element={<NotFound />} /> {/* Ruta general */}  
</Ruta>
```

8. Navegación entre rutas anidadas

Utilice el componente Link o el gancho useNavigate para la navegación.

Paso 1 Navegación con enlaces

En el componente Diseño, agregue enlaces para navegar entre rutas secundarias.

Ejemplo: Barra de navegación en Layout.js

```
<nav>  
  <ul>  
    <li><Link to="profile">Perfil</Link></li> <li><Link  
    to="settings">Configuración</Link></li> </ul> </nav>
```

Paso 2 Navegación programática

Utilice el gancho `useNavigate` para navegar dinámicamente.

Ejemplo: Redireccionamiento con `useNavigate`

```
importar { useNavigate } desde 'react-router-dom';

función Dashboard() { const
  navegar = useNavigate();

  const goToProfile = () => {
    browse('/dashboard/profile'); }; return

  <button onClick={goToProfile}>Ir al perfil</button>; }

exportar tablero predeterminado;
```

9. Gestión del estado en rutas anidadas

Pase el estado a través de la navegación o use el contexto de React para datos compartidos.

Pasando Estado con navegación

```
navegar('/tablero/perfil', { estado: { userId: 123 } });
```

Acceso al estado en componentes secundarios

Utilice el gancho `useLocation` para recuperar el estado.

Ejemplo: Acceso al estado

```
importar { useLocation } de 'react-router-dom'; función

Perfil() { const
  ubicación = useLocation(); const
  { userId } = ubicación.estado || {};
```

devolver <h1>Perfil para el ID de usuario: {userId}</h1>; }

exportar perfil predeterminado;

10. Mejores prácticas para rutas anidadas

1. Organice los archivos de forma lógica: agrupe las rutas y los componentes relacionados en carpetas para mayor claridad.
2. Utilice rutas de índice: simplifique la navegación para los componentes secundarios predeterminados.
3. Maneje los errores con elegancia: incluya rutas de respaldo para rutas no coincidentes.
4. Optimice el rendimiento: utilice la carga diferida (React.lazy y Suspense) para los componentes secundarios.
5. Evite la anidación profunda: limite los niveles de anidación para mantener la legibilidad y la simplicidad.

Las rutas anidadas son una herramienta versátil para crear una navegación estructurada y modular en aplicaciones React. Al aprovechar funciones como Outlet, rutas de índice y parámetros, los desarrolladores pueden crear diseños escalables e intuitivos con componentes compartidos y contenido dinámico.

Seguir las mejores prácticas garantiza que las rutas anidadas sigan siendo mantenibles y eficientes, incluso en aplicaciones complejas.

Capítulo 9.

Dar estilo a las aplicaciones React

- Estilos en línea vs. CSS en JS

El estilo juega un papel crucial en la creación de aplicaciones React visualmente atractivas e intuitivas. Entre los diversos enfoques para diseñar componentes React, los estilos en línea y CSS en JS son dos métodos populares. Ambas técnicas ofrecen ventajas y desventajas únicas, lo que las hace adecuadas para diferentes casos de uso.

1. Comprensión de los estilos en línea

Los estilos en línea implican aplicar estilos directamente a un elemento de React mediante el atributo `style`. Estos estilos se especifican como un objeto JavaScript donde los nombres de las propiedades siguen la sintaxis `camelCase`.

Ejemplo de estilos en línea

```
función InlineExample() { const
  buttonStyle =
    { backgroundColor: 'azul',
      color: 'blanco',
      relleno: '10px',
      borde: 'ninguno',
      borderRadius: '5px', };

  regresar <button style={buttonStyle}>Haz clic en mí</button>; }
```

Ventajas de los estilos en línea

1. Alcance de los componentes:

Los estilos en línea tienen un alcance inherentemente limitado al elemento al que se aplican, lo que evita conflictos de estilos.

2. Estilo dinámico:

Los estilos se pueden manipular fácilmente utilizando variables o lógica de JavaScript, lo que permite actualizaciones dinámicas.

3. Simple y rápido:

Ideal para pequeños cambios de estilo localizados sin crear archivos CSS separados.

Desventajas de los estilos en línea

1. Características limitadas:

Los estilos en línea no admiten funciones CSS avanzadas como pseudoclases (:hover, :focus) o consultas de medios.

2. Problemas de rendimiento: los

estilos en línea pueden generar problemas de rendimiento en aplicaciones grandes debido a declaraciones repetidas.

3. Menor capacidad de mantenimiento:

administrar una gran cantidad de estilos en línea puede resultar engorroso y menos legible.

2. Entendiendo CSS en JS

CSS-in-JS es un enfoque de estilo moderno donde CSS se escribe dentro de archivos JavaScript. Este método permite a los desarrolladores definir estilos como objetos JavaScript y aplicarlos dinámicamente a los componentes de React.

Bibliotecas populares de CSS en JS

- Componentes con estilo
- Emoción
- JSS

Ejemplo de CSS en JS con componentes con estilo

importar con estilo desde 'styled-components';

```
const Button = styled.button` color  
  de fondo: azul;
```



```
color: blanco;
relleno: 10px;
borde: ninguno;
radio del borde: 5px;

&:hover
{ color de fondo: azul oscuro; } `;
```

```
función CSSinJSExample() {
  regresar <Button>Haz clic en mí</Button>; }
```

Ventajas de CSS en JS

1. Estilos con alcance:

Asigna automáticamente estilos a los componentes, eliminando el riesgo de conflictos de estilos globales.

2. Estilo dinámico y condicional: permite que los

estilos se adapten dinámicamente en función de las propiedades del componente o el estado de la aplicación. 3.

Compatibilidad completa con

funciones CSS: admite funciones CSS avanzadas, incluidas pseudoclases, fotogramas clave y consultas de medios.

4. Mantenibilidad mejorada: fomenta el

estilo modular, lo que hace que las bases de código sean más organizadas y fáciles de mantener.

Desventajas de CSS en JS

1. Sobrecarga de rendimiento: el

procesamiento de estilos en tiempo de ejecución puede generar cuellos de botella en el rendimiento en aplicaciones grandes.

2. Curva de aprendizaje:

Requiere comprensión de la sintaxis de la biblioteca y de JavaScript para el estilo. 3.

Agrega dependencias adicionales, aumentando potencialmente el tamaño del paquete.

3. Comparación: Estilos en línea vs. CSS en JS

Estilos en línea de funciones CSS en JS

Sintaxis del objeto JavaScript Objetos de JavaScript o plantillas etiquetadas

Estilo dinámico Compatible Totalmente compatible con props/estado

Pseudoclases No soportadas Soportadas

Consultas de medios No compatible Compatible

Mantenibilidad Baja para aplicaciones a gran escala Alta debido a modularización

Rendimiento Ligero para componentes pequeños Puede introducir sobrecarga de rendimiento en tiempo de ejecución

4. Cuándo usar estilos en línea

- Componentes pequeños con requisitos de estilo mínimos.
- Prototipado rápido o prueba de elementos individuales.
- Estilos que son altamente dinámicos y dependientes de

Lógica de JavaScript.

5. Cuándo usar CSS en JS

- Aplicaciones a gran escala que requieren diseño modular y estilos mantenibles.
- Componentes que necesitan funciones CSS avanzadas como animaciones o consultas de medios.
 - Equipos que adoptan una arquitectura moderna centrada en JavaScript flujo de trabajo de desarrollo.

6. Combinando estilos en línea y CSS en JS

En algunos casos, se puede utilizar una combinación de estilos en línea y CSS en JS. eficaz. Por ejemplo:

- Utilice CSS-in-JS para componentes y diseños reutilizables.
- Utilice estilos en línea para un estilo único y altamente dinámico

ajustes.

Ejemplo: Combinación de enfoques

```
importar con estilo desde 'styled-components';
```

```
const Contenedor = styled.div` display:
```

```
  flex; justify-
```

```
  content: center; align-items:
```

```
  center; height: 100vh; `;
```

```
función CombinedExample() { const
```

```
  dynamicStyle = { color: 'rojo',
```

```
    fontSize:
```

```
    '20px', };
```

```
  devolver (
```

```
    <Contenedor>
```

```
      <p style={dynamicStyle}>Estilo dinámico en línea</p>
```

```
    </Contenedor> ); }
```

Los estilos en línea y CSS-in-JS ofrecen formas únicas de diseñar aplicaciones React, cada una con sus ventajas y desafíos. Los estilos en línea son ideales para elementos pequeños y dinámicos, mientras que CSS-in-JS destaca por su modularidad y sus capacidades CSS avanzadas. Elegir el enfoque adecuado depende de los requisitos del proyecto, las preferencias del equipo y la escala de la aplicación. Al comprender sus diferencias y casos de uso, podrá tomar decisiones informadas para ofrecer aplicaciones React bien diseñadas y fáciles de mantener.

• Uso de módulos CSS

Los módulos CSS ofrecen un enfoque modular y con alcance para el diseño en React, lo que permite a los desarrolladores escribir CSS limpio, reutilizable y fácil de mantener. Cada clase CSS de un módulo tiene alcance local.

evitando conflictos de nombres y garantizando que los estilos se apliquen únicamente a los componentes previstos.

1. ¿Qué son los módulos CSS? ?

Los módulos CSS son una función de CSS que localiza estilos generando nombres de clase únicos. Al importarlos a un archivo JavaScript o TypeScript, permiten aplicar estilos con alcance específico a los componentes donde se utilizan.

2. Ventajas de los módulos CSS

1. Estilos con alcance:

evita la contaminación del espacio de nombres global y los conflictos de estilo.

2. Reutilización:

permite el diseño modular al localizar estilos para componentes específicos.

3. Sintaxis simple:

Utiliza sintaxis CSS normal sin necesidad de bibliotecas de estilo basadas en JavaScript.

4. Agnóstico del marco: se puede

utilizar con cualquier marco o biblioteca que admita módulos CSS.

3. Configuración de módulos CSS

Paso 1: crear una aplicación React

Asegúrate de tener un proyecto React configurado. Si no, crea uno con "Crear aplicación React":

Ejemplo de módulos CSS de npx para crear una aplicación React

Paso 2: habilitar módulos CSS

La aplicación Create React viene con módulos CSS habilitados por defecto. Los archivos con la extensión `.module.css` se consideran módulos CSS.

Paso 3: crear un módulo CSS

En su carpeta `src/`, cree un archivo de módulo CSS llamado `Button.module.css`:

```
/* Button.module.css
*/ .button
{ color de fondo: azul; color:
  blanco; relleno:
  10px 20px; borde:
  ninguno; radio
  del borde: 5px; cursor:
  puntero; }

.button:hover
{ color de fondo: azul oscuro; }
```

Paso 4 Importar y usar el módulo CSS

Importe el módulo CSS en el componente React y aplique la clase.

Ejemplo: Componente de botón

```
importar React desde 'react';
importar estilos desde './Button.module.css';

función Botón() { return
  <button className={styles.button}>Haz clic en mí</button>; }
```

Botón de exportación predeterminado;

Aquí:

- El objeto de estilos asigna nombres de clase del módulo CSS a nombres de clase únicos.

- `estilos.button` hace referencia a la clase `.button` definida en `Button.module.css`.

4. Estilo dinámico con módulos CSS

Los módulos CSS admiten estilos dinámicos combinando múltiples clases o aplicando lógica condicional.

Paso 1 Combinación de clases

Utilice la biblioteca de nombres de clase para combinar clases dinámicamente:

```
npm install nombres de clase
```

Ejemplo: Combinación de clases

```
importar React desde 'react';  
importar classNames desde 'classnames'; importar  
estilos desde './Button.module.css';
```

```
función Botón({ isPrimary }) { const  
  buttonClass = classNames(styles.button, {  
    [estilos.primario]: esPrimario, });  
  
  devolver <button className={buttonClass}>Haz clic en mí</button>; }
```

Botón de exportación predeterminado;

`Botón.module.css`

```
.button  
{ color de fondo: gris; color:  
  blanco; }
```

```
.primary  
{ color de fondo: azul; }
```

5. Uso de módulos CSS con estilos anidados

Los módulos CSS pueden manejar estilos anidados utilizando selectores secundarios o convenciones de nombres BEM.

Ejemplo: estilos anidados

```
/* Tarjeta.module.css */ .card
```

```
{ relleno:  
  20px; borde: 1px  
  sólido #ccc; }
```

```
.cardTitle  
  { tamaño de fuente:  
    18px; peso de fuente:  
    negrita; }
```

```
.cardContent  
  { tamaño de fuente:  
    14px; }
```

Ejemplo: Componente de tarjeta

```
importar React desde 'react';  
importar estilos desde './Card.module.css';
```

```
función Tarjeta()  
  { return  
    ( <div className={styles.card}> <h1  
      className={styles.cardTitle}>Título de la tarjeta</h1> <p  
        className={styles.cardContent}>Contenido de la tarjeta</p> </div> ); }
```

```
exportar tarjeta predeterminada;
```

6. Módulos CSS con animaciones

Los módulos CSS admiten animaciones definidas en fotogramas clave CSS.

Ejemplo: Estilos de animación

```
/* Animación.módulo.css */
@keyframes fadeIn { desde

    { opacidad: 0; }

    hasta { opacidad:

1; } }

.fadeIn
{ animación: fadeIn 2s entrada suave; }
```

Ejemplo: uso de animación

```
importar React desde 'react';
importar estilos desde './Animation.module.css';

función AnimatedComponent() { return <div
    className={styles.fadeIn}>¡Hola, mundo!</div>; }

exportar componente animado predeterminado;
```

7. Mejores prácticas para módulos CSS

1. Utilice nombres de clase significativos: siga una convención de nomenclatura como BEM para mayor claridad y facilidad de mantenimiento.
2. Modularizar estilos:
Cree módulos CSS separados para cada componente.
3. Evite los estilos globales: utilice módulos CSS para delimitar los estilos en lugar de aplicarlos globalmente.

4. Utilice variables:

Define variables CSS reutilizables para colores, fuentes y espaciado.

5. Combine de forma responsiva:

utilice consultas de medios dentro de los módulos CSS para lograr diseños responsivos.

Los módulos CSS proporcionan una excelente manera de diseñar aplicaciones React con CSS con alcance, modular y fácil de mantener.

• Componentes estilizados y emoción

El estilo en React ha evolucionado con bibliotecas modernas como Styled Components y Emotion, que se incluyen en el paradigma CSS-in-JS.

Estas bibliotecas le permiten escribir CSS directamente dentro de sus archivos JavaScript, proporcionando estilos con alcance, dinámicos y fáciles de mantener para sus aplicaciones React.

1. ¿Qué son los componentes con estilo? ?

Styled Components es una biblioteca que permite aplicar estilos a componentes de React mediante literales de plantilla etiquetados. Esta biblioteca aplica estilos a los componentes, garantizando la ausencia de conflictos de espacios de nombres globales y permitiendo aplicar estilos dinámicos mediante props.

Instalación

Para comenzar a utilizar componentes con estilo, instale la biblioteca:

```
npm install componentes con estilo
```

Ejemplo: Componente con estilo básico

```
importar con estilo desde 'styled-components';
```

```
const Button = styled.button` color
```

```
  de fondo: azul; color: blanco;
```

```
  relleno: 10px;
```

borde: ninguno;

radio del borde: 5px;

&:hover

{ color de fondo: azul oscuro; } `;

función App() { return

<Button>Haz clic en mí</Button>; }

exportar aplicación predeterminada;

Ventajas de los componentes estilizados

1. Estilos con alcance: cada componente con estilo tiene nombres de clase únicos, lo que evita conflictos.

2. Estilo dinámico: los estilos pueden cambiar según las propiedades del componente.

3. Componentes reutilizables: combina estilo y estructura en un único componente reutilizable. 4.

Compatibilidad total con CSS: los componentes con estilo son compatibles pseudoclasas, animaciones y consultas de medios.

2. ¿Qué es la emoción? ?

Emotion es otra biblioteca CSS-in-JS para aplicar estilos a aplicaciones React. Ofrece características similares a las de los componentes con estilo, pero con mayor flexibilidad.

Emotion permite escribir estilos como componentes con estilo o directamente en la propiedad CSS.

Instalación

Para usar Emotion, instale su núcleo y los paquetes específicos de React:

npm instalar @emotion/react @emotion/styled

Ejemplo: Componente de emoción básica

```
/** @jsxImportSource @emotion/react */ importar
{ css } desde '@emotion/react';

const buttonStyle = css` color
  de fondo: azul; color: blanco;
  relleno: 10px;
  borde: ninguno;
  radio del borde:
  5px;

  &:hover
    { color de fondo: azul oscuro; } `;

función App()
  { return <button css={buttonStyle}>Haz clic en mí</button>; }
```

exportar aplicación predeterminada;

Ventajas de la emoción

1. Sintaxis múltiples: elija entre propiedades CSS, componentes con estilo o estilos de objeto.

Alto rendimiento: optimizado para el tiempo de ejecución

2. rendimiento.

3. Estilo dinámico: admite accesorios y temas para estilos dinámicos.

Flexibilidad: se integra bien con el CSS 4 existente.

marcos o bibliotecas.

3. Casos de uso comunes

Estilo dinámico con accesorios

Ambas bibliotecas admiten estilos dinámicos basados en accesorios.

Ejemplo de componentes con estilo

```
constante Botón = styled.button`  
  color de fondo: ${props => (props.primary ? 'azul' : 'gris')}; color: blanco; `;
```

```
función App()  
  { return (  
    <>  
    <Botón principal>Primario</Botón>  
    <Button>Secundario</Button> </  
  
  > ); }
```

Ejemplo de emoción

```
const buttonStyle = (primary) => css` color  
  de fondo: ${primary ? 'azul' : 'gris'}; color: blanco; `;
```

```
función App()  
  { return (  
    <>  
    <button css={buttonStyle(true)}>Principal</button> <button  
    css={buttonStyle(false)}>Secundario</button> </> ); }
```

Uso de temas

Los temas permiten la gestión centralizada de estilos como colores, fuentes y espaciado.

Ejemplo de componentes con estilo

```
importar { ThemeProvider } de 'styled-components';
```

```
const theme =
  { colores:
    { primario: 'azul',
      secundario:
        'gris', }, };

const Button = styled.button`
  color de fondo: ${({props}) => props.theme.colors.primary}; color:
  blanco; `;

función App()
  { return (
    <ThemeProvider theme={tema}>
      Haz clic en mí
    </Proveedor de
tema> ); }
```

Ejemplo de emoción

```
importar { ThemeProvider } de '@emotion/react'; const
theme = { colores:
  { primario:
    'azul',
    secundario:
      'gris', }, };

const buttonStyle = (tema) => css`
  color de fondo: ${theme.colors.primary}; color:
  blanco; `;

función App()
  { return (
    <ThemeProvider theme={tema}>
```

```
        <botón          css={{(tema)          =>          buttonStyle(tema)}}>Clic  
Yo</button>  
    </Proveedor de tema> ); }
```

Consultas de medios

Ambas bibliotecas permiten un estilo responsivo mediante consultas de medios.

Ejemplo de componentes con estilo

```
constante Contenedor = styled.div`  
    relleno: 20px;  
  
    @media (ancho máximo: 600px)  
    { relleno: 10px; } `;
```

Ejemplo de emoción

```
const containerStyle = css` relleno:  
    20px;  
  
    @media (ancho máximo: 600px)  
    { relleno: 10px; } `;
```

5. Mejores prácticas

1. Utilice temas para lograr coherencia: centralice los estilos compartidos en un objeto de tema.
2. Mantenga los estilos modulares: evite sobrecargar un solo archivo con demasiados componentes con estilo.

3. Optimice el rendimiento: evite re-renderizaciones innecesarias

mediante el uso de estilos memorizados.

4. Integración gradual: introducir bibliotecas CSS en JS

gradualmente en proyectos existentes.

Los componentes con estilo y Emotion son herramientas potentes para gestionar estilos en aplicaciones React. Mientras que los componentes con estilo son fáciles de usar y priorizan la simplicidad, Emotion ofrece mayor flexibilidad y optimizaciones de rendimiento. La elección de la biblioteca adecuada depende de los requisitos del proyecto y las preferencias del equipo.

Independientemente de su elección, estas herramientas pueden ayudarle a crear interfaces de usuario escalables, mantenibles y dinámicas.

• Tematizar tu aplicación React

La creación de temas en React permite a los desarrolladores gestionar los estilos de toda la aplicación de forma eficiente, garantizando la consistencia del diseño y un mantenimiento más sencillo. Con la creación de temas, puedes centralizar variables como colores, fuentes y espaciado, lo que permite que tu aplicación se adapte a cambios de marca o a los modos claro/oscuro.

1. ¿Por qué utilizar temas??

La temática simplifica la gestión de estilos globales al:

1. Centralizar variables de estilo: defina colores, fuentes y espaciado en un solo lugar.

Consistencia: asegúrese de que el aspecto sea uniforme en todos los 2. solicitud.

3. Ajustes dinámicos: permite realizar cambios en el tiempo de ejecución, como cambiar temas o modos.

2. Configuración de un tema con componentes con estilo

Paso 1 **1** Instalar componentes con estilo

```
npm install componentes con estilo
```

Paso 2 Definir un objeto de tema

Crea un objeto de tema que contenga tus variables de estilo.

Ejemplo: `theme.js` .

```
const theme =  
  { colores:  
    { principal: '#3498db',  
      secundario: '#2ecc71',  
      fondo: '#ecf0f1', texto:  
        '#2c3e50', },  
  
    fuentes:  
    { principal: 'Arial, sans-serif', }, };
```

```
exportar tema predeterminado;
```

Paso 3 Utilice ThemeProvider

Envuelva su aplicación con ThemeProvider y pase el objeto de tema.

Ejemplo: `:` .

```
importar { ThemeProvider } desde 'styled-components'; importar  
tema desde './theme'; importar  
Home desde './Home';
```

```
función App()  
  { return (  
    <ThemeProvider theme={tema}>  
      <Inicio />  
    </Proveedor de  
  
tema> ); }
```



```
exportar aplicación predeterminada;
```

Paso 4 ~~A~~ Acceder a las variables del tema

Utilice la propiedad del tema para acceder a las variables.

Ejemplo: `Home.js` .

```
importar con estilo desde 'styled-components';
```

```
const Contenedor = styled.div`
```

```
  color de fondo: ${({props}) => props.theme.colors.background}; color: ${({props}) =>
  props.theme.colors.text}; familia de fuentes: ${({props}) =>
  props.theme.fonts.primary}; relleno: 20px; `;
```

```
función Inicio() { return
```

```
  <Container>¡Bienvenido a la aplicación temática!</Container>; }
```

```
exportar predeterminado Inicio;
```

3. Establecer un tema con emoción

Paso 1 ~~I~~ Instalar Emotion

```
npm instalar @emotion/react
```

Paso 2 ~~D~~ Definir un objeto de tema

Crea un objeto de tema similar a los componentes con estilo.

Ejemplo: `theme.js` .

```
tema constante =
```

```
{ colores:
  { principal: '#3498db',
    secundario: '#2ecc71',
```

```
fondo: '#ecf0f1', texto:  
'#2c3e50', },
```

fuentes:

```
{ principal: 'Arial, sans-serif', }, };
```

exportar tema predeterminado;

Paso 3 Utilizar ThemeProvider

Envuelva su aplicación con ThemeProvider de Emotion.

Ejemplo: : .

```
importar { ThemeProvider } desde '@emotion/react'; importar  
tema desde './theme'; importar  
Home desde './Home';
```

función App()

```
{ return (  
  <ThemeProvider theme={tema}>  
    <Inicio />  
  </Proveedor de
```

```
tema> ); }
```

exportar aplicación predeterminada;

Paso 4 Acceder a las variables del tema

Acceda a las variables del tema mediante la función css.

Ejemplo: Home.js .

```
/** @jsxImportSource @emotion/react */ importar  
{ css } desde '@emotion/react';
```

```
const containerStyle = (tema) => css`
```

```
color de fondo: ${theme.colors.background}; color: $  
{theme.colors.text}; familia de  
fuentes: ${theme.fonts.primary}; relleno: 20px;  
`;  
`;
```

```
función Inicio() { return  
  <div css={containerStyle}>¡Bienvenido a la aplicación temática! </div>; }
```

```
exportar predeterminado Inicio;
```

4. Implementación de un modo *bscuro*/claro

Paso 1 Ampliar el objeto de tema

Agregue temas separados para los modos claros y oscuros.

Ejemplo: theme.js .

```
exportar const lightTheme =  
  { colores:  
    { fondo: '#ffffff', texto:  
      '#000000', }, },
```

```
exportar const darkTheme =  
  { colores:  
    { fondo: '#000000', texto:  
      '#ffffff', }, },
```

Paso 2 Agregar un selector de temas

Alternar entre temas claros y oscuros.

Ejemplo: App.js

```
importar { ThemeProvider } de 'styled-components'; importar
{ lightTheme, darkTheme } de './theme'; importar Home de
'./Home'; importar { useState } de
'react';

función App()
  { const [isDarkMode, setIsDarkMode] = useState(false);

  devolver
    ( <ThemeProvider theme={isDarkMode ? darkTheme : lightTheme}
  > <button
    onClick={() => setIsDarkMode(!isDarkMode)}>Alternar tema</button>
  <Inicio /> </
    ThemeProvider> ); }

exportar aplicación predeterminada;
```

5. Mejores prácticas para la temática

Centralizar variables: define todos los estilos compartidos en el objeto de tema 1.

2. Utilice nombres significativos: utilice claves descriptivas para colores, fuentes y espaciado.

3. Evite las definiciones de temas en línea: mantenga los objetos del tema Separado para mantenibilidad.

4. Capacidad de respuesta de la prueba: asegúrese de que los temas funcionen bien en diferentes tamaños de pantalla.

La creación de temas en React con componentes con estilo o Emotion simplifica la gestión global de estilos, garantiza la coherencia y permite una personalización dinámica. Tanto si implementas actualizaciones de marca como el modo oscuro, estas herramientas ofrecen la flexibilidad y la potencia necesarias para ofrecer una experiencia de usuario cohesiva.

Capítulo 10.

Obtención de datos y API

• Obtención de datos con Fetch y Axios

Obtener datos de las API es fundamental en el desarrollo web moderno. En React, puedes recuperar y gestionar datos mediante dos métodos populares: la API de obtención integrada y la biblioteca de terceros Axios. Esta guía ofrece un enfoque completo y paso a paso para obtener datos con ambas herramientas, destacando sus características, uso y prácticas recomendadas.

1. Introducción a Fetch y Axios

¿Qué es Fetch? ?

fetch es una API de JavaScript integrada para realizar solicitudes HTTP. Está disponible de forma nativa en navegadores modernos y ofrece una interfaz sencilla basada en promesas.

¿Qué es Axios? ?

Axios es una biblioteca de terceros que simplifica las solicitudes HTTP. Ofrece funciones como cancelación de solicitudes, gestión de tiempos de espera y transformación automática de JSON, lo que la convierte en una opción popular para la obtención de datos.

Diferencias clave

FeatureFetch Axios

Soporte integrado	Nativo de los navegadores	Requiere instalación
Manual de transformación de datos	Se requiere análisis de JSON	Analiza JSON automáticamente
Manejo de errores	Debe manejarse manualmente	Manejo de errores integrado

Características Funcionalidad básica Funciones avanzadas (por ejemplo, interceptores, cancelación)

2. Configuración del proyecto

Paso 1 Crear una nueva aplicación React

Crea un nuevo proyecto React usando Create React App:

Ejemplo de obtención de datos de `npx create-react-app`

Navegue hasta el directorio del proyecto:

ejemplo de obtención de datos de `cd`

Paso 2 Instalar Axios

Si planea utilizar Axios, instálelo mediante npm o yarn:

`npm instalar axios`

3. Obtener datos con Fetch

La API de obtención es una interfaz basada en promesas para realizar solicitudes HTTP. Requiere una URL y devuelve una promesa que se resuelve en el objeto de respuesta.

Paso 1 Crear una función de obtención de datos

Crea un nuevo componente `FetchExample.js`:

```
importar React, { useEffect, useState } de 'react';
```

```
función FetchExample() { const  
  [datos, establecerDatos] = useState([]); const  
  [error, establecerError] = useState(null);
```

```
  usarEfecto(() => {  
    // Obtener datos de una API
```

```

fetch('https://jsonplaceholder.typicode.com/posts') .then((response)
=> { if (!response.ok) { throw
  new Error('La respuesta
    de la red no fue correcta'); } return response.json(); }) .then((data)

=> setData(data)) .catch((error)

=> setError(error.message)); }, []);

si (error)
  { devolver <p>Error: {error}</p>; }

devolver
  ( <ul>
    {datos.mapa((elemento) =>
      ( <li clave={elemento.id}>{elemento.título}</
        li> )}}
    </
  ul> ); }

exportar predeterminado FetchExample;

```

Paso 2 Explicación

1. Obtención de datos:
 - fetch se utiliza para realizar una solicitud GET a la API. • La respuesta se analiza como JSON mediante .json().
2. Manejo de errores:
 - verifique si la respuesta es correcta antes de analizarla para detectarla Errores HTTP.
3. Visualización de datos:
 - Los datos recuperados se asignan a una lista.

Paso 3 Manejo de solicitudes POST

Modifique el componente para enviar datos mediante una solicitud POST:

```
función PostExample() { const
  [mensaje, setMessage] = useState("");

  const handleSubmit = () =>
    { fetch('https://jsonplaceholder.typicode.com/posts', { method:
      'POST', headers:
        { 'Content-
          Type': 'application/json', }, body:

      JSON.stringify({ title: 'foo', body: 'bar', userId: 1 }), }) .then((response)

=> response.json())) .then((data) =>
  setMessage(`Publicación creada con ID: ${data.id}`)); };

  return
  ( <div>
    <button onClick={handleSubmit}>Crear publicación</button> {mensaje
    && <p>{mensaje}</p> } </div> ); }

exportar predeterminado PostExample;
```

4. Obtención de datos con Axios

Axios simplifica las solicitudes HTTP con su interfaz rica en funciones.

Paso 1 Crear una función de obtención de datos

Crea un nuevo componente AxiosExample.js:

```
importar React, { useEffect, useState } de 'react'; importar
axios de 'axios';
```

```
función AxiosExample() {
```



```
const [datos, setData] = useState([]); const
[error, setError] = useState(nulo);

useEffect(() =>

  { axios .get('https://jsonplaceholder.typicode.com/
    posts') .then((respuesta) =>
      setData(respuesta.datos)) .catch((error) =>
        setError(error.mensaje)); }, []);

si (error)
  { devolver <p>Error: {error}</p>; }

devolver
  ( <ul>

    {datos.mapa((elemento) =>
      ( <li clave={elemento.id}>{elemento.título}</

    li> )}}
  </
ul> ); }
```

exportar predeterminado AxiosExample;

Paso 2: Explicación

1. Obtención de
 - datos: axios.get se utiliza para realizar una solicitud GET.
 - Axios analiza automáticamente la respuesta JSON.
2. Manejo de errores:
 - Axios proporciona detalles de errores en un formato consistente.
3. Visualización de datos:
 - de forma similar al ejemplo de búsqueda, los datos se muestran utilizando .mapa.

Paso 3: Manejo de solicitudes POST

Modifique el componente para enviar datos mediante una solicitud POST:

```

función PostExample() { const
  [mensaje, setMessage] = useState("");

  const handleSubmit = () =>

    { axios .post('https://jsonplaceholder.typicode.com/posts', { título: 'foo',
      cuerpo:
        'bar', userId:

        1, }) .then((response) => setMessage(`Publicación creada con ID: $
{response.data.id}
`)) .catch((error) => console.error(error));
  };

  return
  ( <div>
    <button onClick={handleSubmit}>Crear publicación</button> {mensaje
    && <p>{mensaje}</p> } </div> ); }

```

exportar predeterminado PostExample;

5. Uso de / para una mejor legibilidad

Tanto fetch como Axios se pueden usar con async/await para obtener un código más limpio y legible.

Async/Await con búsqueda

```

función AsyncFetchExample() { const
  [datos, setData] = useState([]); const [error,
  setError] = useState(null);

  useEffect(() =>
    { const fetchData = async () => { try {

```

```
constante respuesta = esperar
fetch('https://jsonplaceholder.typicode.com/posts'); if (!response.ok)
  throw new Error('La respuesta de red no fue
OK');
const data = await respuesta.json();
setData(datos); }
catch(error)
{ setError(error.mensaje); } };

obtenerDatos(); },
[]);

si (error)
{ devolver <p>Error: {error}</p>; }

devolver
( <ul>
  {datos.mapa((elemento) =>
    ( <li clave={elemento.id}>{elemento.título}</
li> )}}
</
ul> ); }
```

Async/Await con Axios

```
función AsyncAxiosExample() { const [datos,
  establecerDatos] = useState([]); const [error,
  establecerError] = useState(null);

  useEffect(() => { const
    fetchData = async () => { try { const
```

```
      respuesta =
      axios.get('https://jsonplaceholder.typicode.com/posts');
```

```
        setData(respuesta.datos); }  
      catch(error)  
        { setError(error.mensaje); } }  
  
    obtenerDatos(); },  
    []);  
  
    si (error)  
      { devolver <p>Error: {error}</p>; }  
  
    devolver  
      ( <ul>  
        {datos.mapa((elemento) =>  
          ( <li clave={elemento.id}>{elemento.título}</li>  
        ))}  
      </ul>  
    ); }
```

7. Mejores prácticas para la obtención de datos

1. Utilice controladores de aborto:

Evite fugas de memoria cancelando solicitudes en curso cuando se desmonta el componente.

2. Maneje los errores con elegancia:

muestre mensajes de error significativos al usuario.

3. Optimice el rendimiento: utilice

almacenamiento en caché y paginación para conjuntos de datos grandes.

4. Proteja su API:

Nunca exponga datos confidenciales en solicitudes API.

Obtener datos en React es una habilidad crucial, y tanto fetch como Axios ofrecen soluciones robustas para solicitudes HTTP. Si bien fetch es ligero e integrado en el navegador, Axios proporciona funciones adicionales como interceptores, cancelación de solicitudes y transformación automática de datos.

Al comprender sus capacidades y mejores prácticas, puede tomar decisiones.

decisiones informadas para crear aplicaciones eficientes, confiables y fáciles de usar.

• Uso de React Query para el estado del servidor

Gestión

Gestionar el estado del servidor en aplicaciones web modernas puede ser complicado debido a problemas como el almacenamiento en caché, la sincronización y la gestión de errores. React Query simplifica la gestión del estado del servidor al proporcionar herramientas para obtener, almacenar en caché, sincronizar y actualizar datos del servidor en aplicaciones React.

1. ¿Qué es React Query?

React Query es una biblioteca de obtención de datos para React que gestiona el estado del servidor. Gestiona la sincronización, el almacenamiento en caché y las actualizaciones de datos, reduciendo el código repetitivo y mejorando el rendimiento de la aplicación.

Características principales de React Query

1. Almacenamiento en caché de datos: almacena en caché automáticamente los datos para minimizar las llamadas API redundantes.
2. Actualizaciones en segundo plano: actualiza datos obsoletos en segundo plano.
3. Manejo de errores: proporciona soporte integrado para manejar errores de manera elegante.
4. Recuperación automática: mantiene los datos actualizados mediante la recuperación automática basado en condiciones específicas.
5. Paginación y desplazamiento infinito: simplifica el manejo de grandes conjuntos de datos.

2. Configuración de la consulta de React

Paso 1. Instalar React Query

Para usar React Query, instale el paquete `@tanstack/react-query`:

```
npm install @tanstack/react-query npm install  
@tanstack/react-query-devtools
```

Paso 2. Configurar el cliente de consulta

Cree un cliente de consulta utilizando QueryClient y envuelva su aplicación con QueryClientProvider.

Ejemplo: Configuración del cliente de consultas

```
importar React desde 'react';  
importar ReactDOM desde 'react-dom'; importar  
{ QueryClient, QueryClientProvider } desde '@tanstack/react-query'; importar App desde './App';
```

```
constante queryClient = nuevo QueryClient();
```

```
ReactDOM.render( <QueryClientProvider cliente={queryClient}  
  > <App />  
  </QueryClientProvider>,  
  document.getElementById('root') );
```

3. Obtención de datos con React Query

La característica principal de React Query es su capacidad de obtener y almacenar en caché datos. Utilice el gancho useQuery para obtener datos de una API.

Paso 1. Ejemplo básico

Componente: Obtención de publicaciones

```
importar React desde 'react';  
importar { useQuery } desde '@tanstack/react-query';  
  
constante fetchPosts = async () => {
```

```
constante respuesta = esperar
fetch('https://jsonplaceholder.typicode.com/posts');
if (!response.ok) { throw
  new Error('La respuesta de la red no fue correcta'); } return

response.json()); };

función Posts() { const
  { datos, isLoading, error } = useQuery(['posts'], fetchPosts);

  si (isLoading) retorna <p>Cargando...</p>; si (error)
  retorna <p>Error: {error.message}</p>;

  devolver
  ( <ul>
    {datos.mapa((publicación) =>
      ( <li clave={publicación.id}>{publicación.título}</
    li> )}}
    </
  ul> ); }

exportar publicaciones predeterminadas;
```

Explicación

1. `useQuery(['posts'], obtenerPosts)`
 - Obtiene datos y los almacena en caché debajo de las publicaciones clave.
2. Estado de carga: el indicador `isLoading` indica si se están obteniendo datos.

Manejo de errores: los errores se detectan y se muestran. 3.

4. Representación de datos: los datos se representan una vez que están disponibles.

4.Estados de consulta de React

React Query proporciona múltiples estados para administrar los datos del servidor:

1. Cargando: Indica que se están obteniendo datos (`isLoading`).

2. Éxito: Los datos se han obtenido correctamente (isSuccess). 3. 4.

Error: Hubo un error durante la obtención (isError).

Obsoleto: los datos están obsoletos y es necesario volver a buscarlos.

5. Almacenamiento en caché y tiempo de inactividad

React Query almacena en caché los datos y los considera obsoletos por defecto. Usa la opción `staleTime` para definir cuánto tiempo deben mantenerse actualizados.

Ejemplo: Establecer tiempo de inactividad

```
const { datos } = useQuery(['posts'], fetchPosts, {
  staleTime: 5000, // Los datos permanecen actualizados durante 5
  segundos });
```

6. Recuperación de datos

React Query puede recuperar datos de forma manual o automática.

Recarga manual

Llame a la función de recuperación devuelta por `useQuery` para recuperar los datos.

Ejemplo: Recarga manual

```
const { datos, recuperar } = useQuery(['posts'], fetchPosts);

devolver
( <div>
  <button onClick={() => refetch()}>Recuperar datos</button> <ul>

    {data.map((post) => ( <li
      key={post.id}>{post.title}</li> )))} </ul> </div> );
```

Recarga automática

Utilice la opción `refetchInterval` para recuperar datos automáticamente a intervalos regulares.

Ejemplo: Recarga automática

```
const { datos } = useQuery(['posts'], fetchPosts, {  
  refetchInterval: 10000, // Recuperar cada 10 segundos });
```

7. Mutaciones con React Query

Las mutaciones se utilizan para crear, actualizar o eliminar datos. Utilice el gancho `useMutation` para gestionarlos.

Paso 1 Definir una función de mutación

Componente: Creación de una publicación

```
importar React, { useState } de 'react'; importar  
{ useMutation, useQueryClient } de '@tanstack/react-query';
```

```
const createPost = async (newPost) => {  
  constante          respuesta          =          esperar  
  fetch('https://jsonplaceholder.typicode.com/posts', {  
    método: 'POST',  
    encabezados:  
      { 'Content-Type': 'application/json', },  
  
    cuerpo: JSON.stringify(newPost), }); if  
  (!  
    response.ok) { throw  
      new Error('Error al crear la publicación'); } return  
  
  response.json(); };
```

```
función CreatePost() { const
  [título, setTitle] = useState(""); const
  queryClient = useQueryClient();

  const mutation = useMutation(createPost,
    { onSuccess: () => { //
      Invalidar la consulta de publicaciones para
      actualizar los datos

    }

  });

  queryClient.invalidateQueries(['posts']); }, });

const handleSubmit = () =>
  { mutación.mutate({ título, cuerpo: 'Cuerpo de muestra', usuariold:
    1 });
  setTitle(""); };

return
  ( <div>
    <input
      value={title}
      onChange={(e) => setTitle(e.target.value)}
      placeholder="Ingresar título de la

    publicación" /> <button onClick={handleSubmit}>Crear publicación</
    button> {mutation.isLoading && <p>Creando publicación...</
    p>} {mutation.isError && <p>Error: {mutation.error.message}</p>}
    {mutation.isSuccess && <p>¡Publicación creada exitosamente!</p>} </
    div> ); }
```

exportar predeterminado CreatePost;

Explicación

1. useMutation(createPost)
 - Maneja la creación de una nueva publicación.
2. La devolución de
 - llamada onSuccess invalida la consulta de publicaciones para actualizar la lista.

3. Función de mutación

- Activa la mutación con los datos proporcionados.

8. Invalidación de consultas

Invalidar una consulta garantiza que se vuelvan a obtener los datos almacenados en caché. Utilice `queryClient.invalidateQueries` para invalidar consultas específicas.

Ejemplo: Consulta invalidada

```
queryClient.invalidateQueries(['publicaciones']);
```

9. Paginación y desplazamiento infinito

React Query simplifica el manejo de grandes conjuntos de datos con soporte integrado para paginación y desplazamiento infinito.

Ejemplo de paginación

Componente: Publicaciones paginadas

```
función PaginatedPosts({ página }) { const
  fetchPosts = async ({ clave de consulta }) => { const [,
    página] = clave de consulta; const
    respuesta = await fetch( `https://
      jsonplaceholder.typicode.com/posts? _page=${page}
&_limit=10`
    );
    devolver respuesta.json(); };

const { datos } = useQuery(['posts', página], fetchPosts);

devolver
( <ul>
  {datos.mapa((publicación) =>
    ( <li clave={publicación.id}>{publicación.título}</
    li> ))}
  </
ul> ); }
```

```
}
```

10. Uso de React Query Devtools

React Query Devtools proporciona una interfaz visual para inspeccionar consultas, mutaciones y cachés.

Instalación

Instalar las herramientas de desarrollo:

```
npm installa @tanstack/react-query-devtools
```

Configuración

Envuelva su aplicación con el componente ReactQueryDevtools:

```
importar { ReactQueryDevtools } desde '@tanstack/react-query-devtools';
```

```
<QueryClientProvider cliente={queryClient}>  
  <Aplicación />  
  <ReactQueryDevtools initialIsOpen={falso} />  
</ProveedorClienteDeConsulta>;
```

11. Mejores prácticas para consultas de React

1. Utilice claves de consulta de forma eficaz: utilice claves de consulta únicas y descriptivas para evitar conflictos.
2. Aprovechar la caché:
 - definir tiempos de caché adecuados para reducir reajustes innecesarios.
3. Centralizar las funciones de API:
 - cree un archivo separado para todas las funciones de API para mantener su código organizado.
4. Maneje los errores con elegancia:
 - proporcione mensajes de error significativos y alternativas.

React Query es una potente herramienta para gestionar el estado del servidor en aplicaciones React. Al simplificar la obtención de datos, el almacenamiento en caché y la sincronización, reduce el código repetitivo y mejora el rendimiento de la aplicación. Con funciones como mutaciones, paginación y recarga en segundo plano, React Query es imprescindible para el desarrollo moderno de React.

• Manejo de carga, errores y almacenamiento en caché

Gestionar eficientemente los estados de carga, los errores y el almacenamiento en caché es esencial para crear aplicaciones React fluidas e intuitivas. Estos aspectos garantizan interacciones fluidas, una retroalimentación adecuada durante la obtención de datos y un rendimiento óptimo. Esta guía explica cómo gestionar eficazmente los estados de carga, los errores y el almacenamiento en caché en aplicaciones React mediante herramientas como React Query, Axios y la API de obtención nativa.

1. Comprensión de los conceptos básicos

Estado de carga

Un estado de carga representa el periodo durante el cual una aplicación espera datos de un servidor. Garantiza que los usuarios sepan que un proceso está en curso.

Estado de error

Un estado de error se produce cuando una solicitud falla debido a problemas de red, errores del servidor o solicitudes no válidas. La gestión adecuada de errores es crucial para la depuración y la retroalimentación del usuario.

Almacenamiento en caché

El almacenamiento en caché almacena temporalmente los datos obtenidos, lo que reduce las solicitudes de red redundantes y mejora el rendimiento de la aplicación.

2. Configuración de la aplicación

Paso 1 **1** Instalar dependencias

Si usa React Query o Axios, instale los paquetes necesarios:

```
npm instalar @tanstack/react-query axios
```

Paso 2 Crear la estructura de la aplicación

Organiza tu proyecto:

src/

├── componentes/ | ──

LoadingIndicator.js | ── ErrorMessage.js

├── api/ | ── apiClient.js ── App.js

3. Manejo de estados de carga

3.1 Visualización de un indicador de carga

Utilice un indicador de carga, como un spinner o un texto, para informar a los usuarios que se están obteniendo datos.

Ejemplo: LoadingIndicator.js

```
función LoadingIndicator() { return  
  <p>Cargando...</p>; }
```

```
exportar predeterminado LoadingIndicator;
```

3.2 Integración con useState

Para escenarios de búsqueda simples, administre los estados de carga manualmente con useState.

Ejemplo: Cargar con Fetch

```
importar React, { useEffect, useState } de 'react';
importar LoadingIndicator de './LoadingIndicator';

función FetchData()
{ const [datos, setData] = useState(null);
  const [cargando, setLoading] = useState(true);

  useEffect(() =>
    { fetch('https://jsonplaceholder.typicode.com/
      posts') .then((respuesta) =>

        respuesta.json()) .then((datos) => { setData(datos); setLoading(false); }); }, []);

    si (cargando)
      { return <IndicadorDeCarga />; }

    devolver
      ( <ul>
        {datos.mapa((elemento) =>
          ( <li clave={elemento.id}>{elemento.título}</

            li> )}}
        </
      ul> ); }

exportar FetchData predeterminado;
```

3.3 Uso de React Query para cargar el estado

React Query simplifica la gestión del estado de carga con su propiedad `isLoading`.

Ejemplo: Carga con React Query

```
importar { useQuery } desde '@tanstack/react-query';
```

```
const fetchPosts = async () => { const
  respuesta = await fetch('https://
jsonplaceholder.typicode.com/posts'); return respuesta.json(); };
```

```
función Posts() { const
  { datos, isLoading } = useQuery(['posts'], fetchPosts);

  if (isLoading)
    { return <p>Cargando...</p>; }
```

```
  devolver
    ( <ul>

      {datos.mapa((publicación) =>
        ( <li clave={publicación.id}>{publicación.título}</

        li> )}}
    </
  ul> ); }
```

```
exportar publicaciones predeterminadas;
```

4. Manejo de errores

4.1 Visualización de mensajes de error

Proporciona mensajes de error claros y útiles para informar a los usuarios cuando algo sale mal.

Ejemplo: ErrorMessage.js

```
función ErrorMessage({ mensaje }) { devolver <p
  estilo={{ color: 'rojo' }}>Error: {mensaje}</p>; }
```

```
exportar mensaje de error predeterminado;
```

4.2 Manejo de errores con Fetch

Maneje manualmente los errores verificando el estado de la respuesta.

Ejemplo: Manejo de errores con Fetch

```
función FetchDataWithErrorHandling() { const
  [datos, setData] = useState(null); const [error,
  setError] = useState(null);

  useEffect(() =>
    { fetch('https://jsonplaceholder.typicode.com/
      posts') .then((response)
        => { if (!response.ok)
            { throw new Error('Error al obtener los datos'); }

          return

          response.json(); }) .then((data) =>
            setData(data)) .catch((error) =>
              setError(error.message)); }, []);

  si (error)
    { devolver <ErrorMessage mensaje={error} />; }

  devolver
    ( <ul>
      {datos.mapa((elemento) =>
        ( <li clave={elemento.id}>{elemento.título}</
        li> )}}
      </
    ul> ); }
```

4.3 Manejo de errores con React Query

React Query captura automáticamente errores y proporciona un indicador `isError`.

Ejemplo: Manejo de errores con consultas de React

```
función PostsWithErrorHandling() { const { datos,
  isLoading, isError, error } = useQuery(['posts'], fetchPosts);

  si (estáCargando)
    { devolver <p>Cargando...</p>; } si

  (estáError)
    { devolver <ErrorMessage mensaje={error.message} />; }

  devolver
    ( <ul>
      {datos.mapa((publicación) =>
        ( <li clave={publicación.id}>{publicación.título}</
          li> )}}
      </
    ul> ); }
```

5 Implementación del almacenamiento en caché

El almacenamiento en caché garantiza que los datos se almacenen localmente para su reutilización, lo que reduce las llamadas de red redundantes y mejora el rendimiento.

5.1 Almacenamiento en caché de consultas de React

React Query almacena en caché automáticamente los datos bajo la clave de consulta.

Ejemplo: Almacenamiento en caché de datos

```
función PostsWithCaching() { const { data,
  isLoading } = useQuery(['posts'], fetchPosts, { staleTime: 5000, // Los datos permanecen
    actualizados durante 5 segundos cacheTime: 300000, // El caché dura 5
    minutos });

  si (estáCargando) {
```

```
    regresar <p>Cargando...</p>;
  }

  devolver (
    <ul>
      {datos.map((publicación) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

5.2 Uso de Axios para el almacenamiento en caché

Axios admite el almacenamiento en caché con bibliotecas de terceros como axios-cache-adapter.

Instalación

```
npm install axios adaptador de caché axios
```

Ejemplo: Almacenamiento en caché con Axios

```
importar axios desde 'axios';
importar { setupCache } desde 'axios-cache-adapter';

constante cache = setupCache({
  Edad máxima: 15 60 1000, // Almacenar datos en caché durante 15 minutos
});

constante api = axios.create({
  adaptador: cache.adapter,
});

constante fetchPostsWithAxios = async () => {
  constante respuesta =
    api.get('https://jsonplaceholder.typicode.com/posts');
  devolver respuesta.data;
};
```

```
función AxiosCachedPosts() { const { datos,  
  isLoading } = useQuery(['cachedPosts'], fetchPostsWithAxios);  
  
  if (isLoading) { return  
    <p>Cargando...</p>; }  
  
  devolver  
  ( <ul>  
    {datos.mapa((publicación) =>  
      ( <li clave={publicación.id}>{publicación.título}</li>  
    ))}  
  </ul> ); }  
  
exportar predeterminado AxiosCachedPosts;
```

6. Combinación de estados de carga, error y almacenamiento en caché

React Query gestiona la carga, los errores y el almacenamiento en caché sin problemas, lo que le permite concentrarse en la lógica empresarial.

Ejemplo: Componente de obtención de datos completo

```
función PostsComplete() { const { datos,  
  se está cargando, se está produciendo un error, error } = useQuery(['posts'], fetchPosts,  
{ staleTime: 5000,  
  cacheTime: 300000, });  
  
  if (isLoading) { return  
    <p>Cargando...</p>; }  
  
  si (isError)  
    { devolver <ErrorMessage mensaje={error.message} />; }
```

```
devolver (  
  <ul>  
    {datos.map((publicación) => (  
      <li key={post.id}>{post.title}</li>  
    ))}  
  </ul>  
);  
}
```

exportar por defecto PostsComplete;

7. Mejores prácticas para la carga, errores y almacenamiento en caché

1. Estados de carga:
 - Utilice cargadores de esqueleto o hilanderos para un mejor uso. experiencia.
 - Mostrar indicadores de carga en áreas donde se almacenan datos esperado.
2. Estados de error:
 - Proporcionar mensajes de error significativos.
 - Registrar errores para depuración y análisis.
3. Almacenamiento en caché:
 - Establezca tiempos de caché adecuados para equilibrar el rendimiento y frescura de los datos.
 - Utilice claves de consulta de forma eficaz para evitar conflictos.
4. Combinar estados:
 - Utilice bibliotecas como React Query para el estado integrado gestión.

Gestionar la carga, los errores y el almacenamiento en caché de manera eficiente es vital para crear Aplicaciones React robustas. Herramientas como React Query simplifican estos procesos, ofreciendo soporte integrado para gestionar estados del servidor, minimizando las llamadas API redundantes y proporcionando retroalimentación clara a usuarios.

• Integración de API RESTful

La integración de APIs RESTful es una habilidad fundamental para los desarrolladores de React, ya que permite la interacción entre una aplicación frontend y un servicio backend. Las API permiten obtener, crear, actualizar y eliminar datos, proporcionando la funcionalidad necesaria para las aplicaciones modernas.

1. Comprensión de las API RESTful

Una API RESTful es un servicio web que se rige por los principios REST (Transferencia de Estado Representacional). Utiliza métodos HTTP para realizar operaciones con los recursos:

- GET: Obtener recursos.
- POST: Crear nuevos recursos.
- PUT/PATCH: Actualizar recursos existentes.
- ELIMINAR: Eliminar recursos.

Para este tutorial, utilizaremos la API JSONPlaceholder, un servicio gratuito para probar API RESTful.

2. Configuración de la aplicación React

Paso 1 Crear un nuevo proyecto de React

Para comenzar, crea una nueva aplicación React:

```
npx create-react-app integración-de-api-restful cd
integración-de-api-restful
```

Paso 2 Instalar Axios (opcional)

Si prefiere utilizar Axios en lugar de la API de búsqueda nativa:

```
npm instalar axios
```

Paso 3 Organizar el proyecto

Crea la siguiente estructura para una mejor organización:

fuentes/

```
|— componentes/ |  
|— PostList.js | |—  
PostForm.js |— api/ | |—  
apiClient.js  
|— App.js
```

3. Obtención de datos de una API

Usando Fetch

La API de obtención es un método integrado para realizar solicitudes HTTP en JavaScript. Aquí se explica cómo obtener datos en React.

Ejemplo: Obtener publicaciones

```
importar React, { useEffect, useState } de 'react';
```

```
función PostList() { const
```

```
  [posts, setPosts] = useState([]); const [loading,  
  setLoading] = useState(true); const [error, setError] =  
  useState(null);
```

```
  useEffect(() =>
```

```
    { fetch('https://jsonplaceholder.typicode.com/  
      posts') .then((response)
```

```
      => { if (!response.ok)
```

```
        { throw new Error('La respuesta de la red no fue correcta'); }
```

```
      return
```

```
        response.json(); }) .then((data) => { setPosts(data); setLoading(false); }) .catch((error) => { setError
```

```
    setLoading(falso); }); },
  []);
```

```
si (cargando) retorna <p>Cargando...</p>; si (error) retorna <p>Error: {error}</p>;
```

```
devolver
( <ul>
  {posts.map((post) => ( <li
    key={post.id}>{post.title}</li> ))} </ul> ); }
```

```
exportar lista de publicaciones predeterminada;
```

Usando Axios

Axios simplifica las solicitudes HTTP con transformación JSON automática y manejo de errores integrado.

Ejemplo: Obtener publicaciones con Axios

```
importar React, { useEffect, useState } de 'react';
importar axios de 'axios';
```

función PostList()

```
{ const [posts, setPosts] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

```
useEffect(() =>
```

```
{ axios .get('https://jsonplaceholder.typicode.com/
posts') .then((response)
=> { setPosts(response.data);
setLoading(false);

})
```



```

    }) .catch((error) =>
      { setError(error.mensaje);
        setLoading(falso); }); },
    []);

```

si (cargando) retorna <p>Cargando...</p>; si (error) retorna <p>Error: {error}</p>;

```

devolver
( <ul>
  {posts.map((post) => ( <li
    key={post.id}>{post.title}</li> ))} </ul> ); }

```

exportar lista de publicaciones predeterminada;

4. Envío de datos a una API

Usando Fetch

Para crear un nuevo recurso, utilice el método POST.

Ejemplo: Enviar una publicación

```

función PostForm()
{ const [título, setTitle] = useState("");
  const [mensaje, setMessage] = useState("");

  const handleSubmit = (e) =>
    { e.preventDefault();

      fetch('https://jsonplaceholder.typicode.com/posts',
        { método: 'POST',
          encabezados: { 'Tipo de contenido': 'application/
            json' }, cuerpo: JSON.stringify({ título, cuerpo: 'Cuerpo de muestra',
            userId: 1 }), })

```

```
    }) .then((respuesta) =>
      respuesta.json()) .then((datos) => setMessage(`Publicación creada con ID: ${data.id}`)) .catch((error) => setMessage(`Error: ${error.message}`)); });
```

devolver

```
( <formulario onSubmit={handleSubmit}>
  <valor
    de
    entrada={título} onChange={(e) =>
      setTitle(e.target.value)}

    marcador de posición="Título" /> <botón
      tipo="enviar">Enviar</button> {mensaje &&
```

```
<p>{mensaje}</p> } </form> ); }
```

Usando Axios

Axios simplifica las solicitudes POST con transformación JSON incorporada.

Ejemplo: Enviar una publicación con Axios

```
función PostForm() { const
  [título, setTitle] = useState(""); const
  [mensaje, setMessage] = useState("");
```

```
const handleSubmit = (e) =>
  { e.preventDefault();
```

```
    axios .post('https://jsonplaceholder.typicode.com/posts', { título,
      cuerpo: 'Cuerpo de
      muestra',
      usuario: 1, })
```

```

        .then((response) => setMessage(`Publicación creada con ID: ${response.data.id}`))
    )).catch((error) =>
        setMessage(`Error: ${error.message}`)); });

```

devolver

```

    ( <formulario onSubmit={handleSubmit}>
        <valor
            de
            entrada={título} onChange={(e) =>
                setTitle(e.target.value)}

            marcador de posición="Título" /> <botón
                tipo="enviar">Enviar</button> {mensaje
            &&

        <p>{mensaje}</p> } </form> ); }

```

5. Actualización y eliminación de datos

Actualizar un recurso

Para actualizar un recurso, utilice el método PUT o PATCH.

Ejemplo: Actualizar una publicación con Axios

función updatePost(id, updatedData) {

```

    axios .put(`https://jsonplaceholder.typicode.com/posts/${id}`,
        updatedData) .then((response) => console.log('Publicación actualizada:',
            response.data)) .catch((error) => console.error('Error al actualizar la publicación:', error));
    }

    updatePost(1, { title: 'Título actualizado', body: 'Cuerpo actualizado' });

```

Eliminar un recurso

Para eliminar un recurso, utilice el método DELETE.

Ejemplo: Eliminar una publicación con Axios

función deletePost(id)

```
{ axios.delete(`https://jsonplaceholder.typicode.com/posts/${id}`) .then(() =>
  console.log('Publicación eliminada')) .catch((error)
    => console.error('Error al eliminar publicación:', error));
}
```

eliminarPost(1);

6. Uso de React Query para la integración de API

React Query simplifica la obtención de datos y la gestión del estado mediante el control del almacenamiento en caché, el manejo de errores y la recuperación en segundo plano.

Paso 1. Instalar React Query

npm instalar @tanstack/react-query

Paso 2. Configurar el cliente de consulta

```
importar { QueryClient, QueryClientProvider } de '@tanstack/react-query'; importar PostList de './
PostList';
```

```
constante queryClient = nuevo QueryClient();
```

```
función App() { return
  (
    <QueryClientProvider cliente={queryClient}>
      <Lista de publicaciones />
    </ProveedorDeClienteDeConsulta> ); }
```

```
exportar aplicación predeterminada;
```

Paso 3 Obtener datos con una consulta de React

```
importar { useQuery } desde '@tanstack/react-query';

const fetchPosts = async () => { const
  respuesta = await fetch('https://
jsonplaceholder.typicode.com/posts'); return respuesta.json(); };

función PostList() { const
  { datos, isLoading, error } = useQuery(['posts'], fetchPosts);

  si (isLoading) retorna <p>Cargando...</p>; si
  (error) retorna <p>Error: {error.message}</p>;

  devolver
  ( <ul>
    {datos.mapa((publicación) =>
      ( <li clave={publicación.id}>{publicación.título}</
    li> )))
  </
ul> ); }
```

7. Mejores prácticas para la integración de API

1. Centralizar las llamadas

API: cree un archivo dedicado para las solicitudes API para mantener su código organizado.

```
exportar const fetchPosts = () => fetch('/posts').then((res) => res.json());
```

2. Maneje los errores con elegancia:

proporcione mensajes de error significativos a los usuarios.

3. Optimice el rendimiento: utilice

bibliotecas de almacenamiento en caché como React Query para una gestión eficiente del estado.

4. API seguras:

Evite exponer información confidencial, como claves API, en su código frontend.

Integrar APIs RESTful en React es una habilidad crucial para crear aplicaciones dinámicas basadas en datos. Ya sea que se use la API nativa de búsqueda, Axios o React Query, comprender cómo interactuar con las API para operaciones CRUD es esencial.

Capítulo 11.

Gestión del estado más allá de React

• API de contexto para la gestión de estados globales

La gestión del estado es un aspecto fundamental en la creación de aplicaciones React.

Si bien el estado local es suficiente para gestionar el estado de pequeña escala, gestionar el estado global en múltiples componentes requiere una solución sólida.

La API de contexto es una función integrada de React para administrar el estado global de manera eficiente sin depender de bibliotecas de terceros.

1. ¿Qué es la API de contexto? ?

La API de contexto permite crear estados globales que pueden compartirse entre componentes sin necesidad de pasar propiedades manualmente a cada nivel del árbol de componentes.

Resulta especialmente útil para gestionar:

- Datos de autenticación del usuario
- Temas y preferencias de interfaz de usuario
- Configuración de idioma
- Configuraciones de toda la aplicación

Características clave de la API de contexto

1. Evita la perforación de los puntales (pasar los puntales a través de varias capas de componentes). 2.

Ligero e integrado en React, sin bibliotecas externas son necesarios.

3. Adecuado para aplicaciones de tamaño moderado.

2. ¿Cómo funciona la API de contexto? ?

La API de contexto consta de tres componentes principales:

1. `React.createContext`: crea un objeto Context.

2. Proveedor: proporciona el valor de contexto a sus hijos.
3. Consumidor: consume el valor del contexto.

3. Configuración de la API de contexto

Paso 1 Crear un contexto

Crea un archivo para definir el contexto y su proveedor.

Ejemplo: ThemeContext.js

```
importar React, { createContext, useState } de 'react';

exportar const ThemeContext = createContext();

exportar const ThemeProvider = ({ children }) => { const
  [theme, setTheme] = useState('light');

  const toggleTheme = () =>
    { setTheme((prevTheme) => (prevTheme === 'claro' ? 'oscuro' : 'claro')); };

  devolver
    ( <ThemeContext.Provider valor={{ tema, toggleTheme }}> {hijos}

    </ThemeContext.Provider> ); };


```

Paso 2 Envuelva la aplicación con el proveedor

Envuelva su árbol de componentes con el proveedor para que el contexto esté disponible.

Ejemplo: App.js

```
importar React desde 'react';
importar { ThemeProvider } desde './ThemeContext'; importar
ThemedComponent desde './ThemedComponent';

función App() {
```



```
devolver (  
  <Proveedor de temas>  
    <Componente temático />  
  </Proveedor de tema> ); }
```

exportar aplicación predeterminada;

Paso 3 **Utilizar** el contexto en los componentes

Consuma los valores de contexto utilizando el gancho useContext.

Ejemplo: ThemedComponent.js

```
importar React, { useContext } de 'react'; importar  
{ ThemeContext } de './ThemeContext';  
  
función ThemedComponent() { const { tema,  
  toggleTheme } = useContext(ThemeContext);  
  
  const estilo =  
    { backgroundColor: tema === 'claro' ? '#fff' : '#333', color: tema ===  
      'claro' ? '#000' : '#fff', relleno: '20px', textAlign:  
        'center', };  
  
  return  
    ( <div style={estilo}> <p>El  
      tema actual es {theme}</p> <button  
        onClick={toggleTheme}>Alternar tema</button> </div> ); }
```

exportar componente temático predeterminado;

4. Uso de contextos múltiples

Puede administrar múltiples contextos para diferentes estados globales, como autenticación de usuarios y temas.

Ejemplo: Combinación de contextos

```
importar { ThemeProvider } desde './ThemeContext'; importar  
{ AuthProvider } desde './AuthContext';
```

```
función App() { return  
  (  
    <Proveedor de Autorización>  
      <Proveedor de temas>  
        <SuComponente />  
      </Proveedor de temas>  
    </Proveedor de
```

```
Autorización> ); }
```

```
exportar aplicación predeterminada;
```

5. Ventajas y desventajas de la API de contexto

Ventajas

1. Integrado en React: no se requieren bibliotecas adicionales.
2. Reduce la perforación de la hélice.
3. Simple e intuitivo para aplicaciones pequeñas y medianas.

Limitaciones

1. No optimizado para actualizaciones frecuentes; pueden producirse repeticiones de renderizados excesivas.
2. La depuración puede ser un desafío para componentes profundamente anidados.
3. Escalabilidad limitada para aplicaciones grandes.

6. Mejores prácticas

1. Evite el uso excesivo del contexto: utilice la API de contexto para obtener un estado verdaderamente global; confíe en el estado local cuando sea adecuado.
2. Modularizar contextos: crear contextos separados para dominios de estado distintos (por ejemplo, tema, autenticación).
3. Combinar con otras herramientas: para aplicaciones complejas, Utilice Context API junto con bibliotecas como Redux o Zustand.

La API de Contexto es una solución potente y ligera para gestionar el estado global en aplicaciones React. Al reducir la necesidad de realizar pruebas de rendimiento y al integrarse a la perfección con el ecosistema de React, ofrece una excelente opción para proyectos de tamaño moderado. Sin embargo, para aplicaciones a gran escala, considere combinar la API de Contexto con otras herramientas de gestión de estado para garantizar el rendimiento y la escalabilidad.

• Kit de herramientas Redux: un enfoque simplificado para Redux

Redux es una biblioteca de gestión de estados ampliamente utilizada para aplicaciones React. Si bien Redux es potente, su configuración verbosa y su código repetitivo suelen resultar complejos. Redux Toolkit (RTK) aborda estos desafíos ofreciendo un enfoque simplificado y más eficiente para usar Redux.

1. ¿Qué es Redux Toolkit? ?

Redux Toolkit es un paquete oficial del equipo Redux que simplifica la configuración de Redux y reduce el código repetitivo. Proporciona herramientas para la gestión de estados, acciones asíncronas e integración de middleware.

Características principales de Redux Toolkit

1. `configureStore`: simplifica la configuración de la tienda.
2. `createSlice`: combina acciones y reductores en una unidad única.
3. Middleware integrado: integra Redux Thunk para lógica asíncrona.
4. Inmutabilidad con Immer: simplifica las actualizaciones de estado.

2. Configuración del kit de herramientas Redux

Paso 1: Instalar Redux Toolkit

Instalar Redux Toolkit y React-Redux:

```
npm instalar @reduxjs/toolkit react-redux
```

Paso 2: Crea una porción de Redux

Una porción contiene la lógica del reductor y las acciones para una característica específica.

Ejemplo: counterSlice.js

```
importar { createSlice } desde '@reduxjs/toolkit';

const counterSlice = createSlice({ nombre:
  'contador',
  estadoInicial: { valor: 0 },
  reductores:
    { incremento: (estado) =>
      { estado.valor +=
        1; }, decremento: (estado)
        => { estado.valor
          -= 1; }, reinicio:
            (estado) =>
              { estado.valor = 0; }, }, });

export const { incremento, decremento, reinicio } = counterSlice.actions; export
default counterSlice.reducer;
```

Paso 3: Configurar la tienda

Utilice configureStore para configurar la tienda Redux.

Ejemplo: store.js

```
importar { configureStore } desde '@reduxjs/toolkit'; importar  
counterReducer desde './counterSlice';
```

```
exportar const store = configureStore({ reductor:  
  { contador:  
    counterReducer, }, });
```

Paso **4** Proporcionar la tienda a la aplicación

Envuelva su aplicación con el componente Proveedor.

Ejemplo: index.js

```
importar React desde 'react';  
importar ReactDOM desde 'react-dom'; importar  
{ Provider } desde 'react-redux'; importar { store }  
desde './store'; importar App desde './  
App';
```

```
ReactDOM.render( <Provider  
  store={tienda}> <App /> </  
  Provider>,
```

```
  document.getElementById('root') );
```

3. Uso del kit de herramientas Redux en componentes

Paso **1** Seleccionar estado

Utilice el gancho useSelector para acceder al estado.

Ejemplo: CounterDisplay.js

```
importar React desde 'react';  
importar { useSelector } desde 'react-redux';
```

```
función CounterDisplay() { const
  contador = useSelector((estado) => estado.contador.valor); devolver
  <h1>Contador: {contador}</h1>; }
```

```
exportar predeterminado CounterDisplay;
```

Paso 2 Acciones de despacho

Utilice el gancho useDispatch para enviar acciones.

Ejemplo: CounterControls.js

```
importar React desde 'react';
importar { useDispatch } desde 'react-redux'; importar
{ incrementar, decrementar, reiniciar } desde './counterSlice';

función CounterControls() { const
  despacho = useDispatch();

  devolver
  ( <div>
    <button onClick={() dispatch(increment())}>Incrementar</
    button> <button onClick={() dispatch(decrement())}
    >Decrementar</button> <button onClick={() =>
    dispatch(reset())}>Restablecer</button> </div> ); }
```

```
exportar CounterControls predeterminados;
```

4. Manejo de acciones asincrónicas

Redux Toolkit incluye createAsyncThunk para administrar la lógica asincrónica.

Ejemplo: Obtención de datos

```
importar { createSlice, createAsyncThunk } desde '@reduxjs/toolkit';

exportar const fetchPosts = createAsyncThunk('posts/fetchPosts', async () =>
{ const
  respuesta = await fetch('https://
jsonplaceholder.typicode.com/posts'); devolver respuesta.json(); });

const postsSlice = createSlice({ nombre:
  'posts', initialState:
  { posts: [], estado: 'idle', error: null }, reductores: {},
  extraReducers:
  (builder) =>

    { builder .addCase(fetchPosts.pending, (state) =>
      { state.status =

        'loading'; }) .addCase(fetchPosts.fulfilled, (state, action) =>
        { state.status = 'succeeded';
          state.posts =

            action.payload; }) .addCase(fetchPosts.rejected, (state,
              action) => { state.status =

                'failed'; state.error =

                  action.error.message; }); }, });

exportar publicaciones predeterminadasSlice.reducer;
```

Mejores prácticas con Redux Toolkit

1. Utilice porciones para la modularidad: cree porciones para diferentes características o dominios.
2. Aproveche los Thunks asíncronicos: simplifique la integración de API con `createAsyncThunk`.

3. Combinar con RTK Query: para almacenamiento en caché y sincronización de API avanzados.

4. Organice los archivos: mantenga los reductores, las acciones y la tienda configuración modular.

Redux Toolkit ha revolucionado la gestión de estados en React al simplificar la compleja configuración de Redux y reducir el código repetitivo. Sus potentes utilidades, como `createSlice` y `createAsyncThunk`, facilitan la gestión de estados síncronos y asíncronos.

• Comparación de Redux y Context API

La gestión de estados es fundamental en la creación de aplicaciones React, y los desarrolladores suelen debatir si usar Redux o la API de Context. Ambas herramientas tienen sus ventajas y desventajas, y la elección depende en gran medida de la complejidad y los requisitos de la aplicación. En esta guía, exploraremos las diferencias entre Redux y la API de Context, cuándo usar cada una y los escenarios prácticos en los que destacan.

Entendiendo Redux

Redux es una popular biblioteca de gestión de estados que proporciona un almacén centralizado para gestionar el estado de una aplicación. Suele asociarse con React, pero también puede usarse con otros frameworks o bibliotecas de JavaScript.

Conceptos básicos de Redux

1. Tienda: una única fuente de verdad para la aplicación estado.
Acciones: Objetos JavaScript simples que describen qué
2. sucedió.
3. Reductores: Funciones que toman el estado actual y una acción, luego devuelven el siguiente estado.
4. Middleware: intercepta acciones para realizar efectos secundarios como el registro, las llamadas API o las operaciones asíncronas.

Comprensión de la API de contexto

La API de contexto es una función integrada de React que permite gestionar y compartir el estado globalmente sin necesidad de realizar perforaciones. Es ligera y no requiere bibliotecas adicionales.

Conceptos básicos de la API de contexto

1. Objeto de contexto: creado utilizando `React.createContext`.

Proveedor: suministra el estado a sus componentes secundarios. 2.

3. Consumidor: consume el estado del proveedor, ya sea directamente o a través del gancho `useContext`.

Cuándo usar Redux

1. Grandes aplicaciones con estados complejos

Redux es ideal para aplicaciones que requieren:

- Gestión de estados profundamente anidados.
- Compartir estado entre componentes distantes en el árbol de componentes.
- Manejo de proyectos a gran escala y con múltiples funcionalidades.

2. Gestión predecible del estado

La estricta estructura de Redux facilita la depuración y el seguimiento de los cambios de estado a lo largo del tiempo, especialmente en aplicaciones con múltiples contribuyentes.

3. Middleware para acciones asíncronas

Si su aplicación implica operaciones asincrónicas complejas, como llamadas API, el middleware de Redux (por ejemplo, Redux Thunk o Redux Saga) simplifica su manejo.

Ejemplo de caso de uso: Aplicación de comercio electrónico

Una plataforma de comercio electrónico puede utilizar Redux para gestionar:

- Autenticación de usuario.
- Estado del carrito de compras.